

BCSE205L

Computer Architecture and Organization

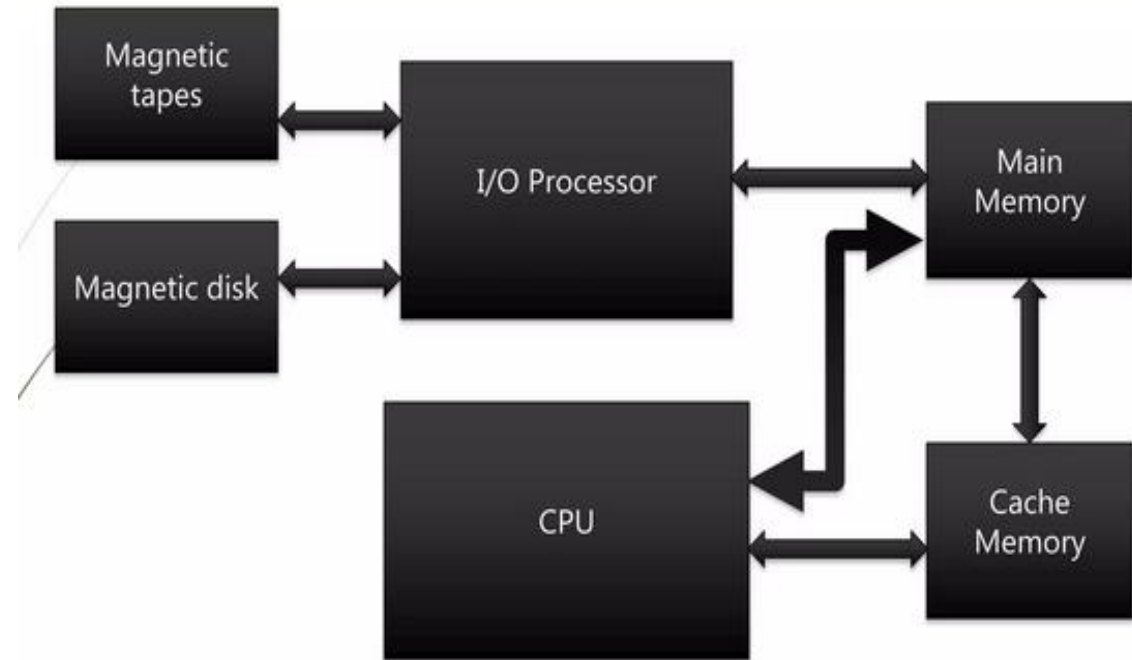
Module 4 – Memory System Organization and Architecture

Module:4	Memory System Organization and Architecture	7 Hours
Memory systems hierarchy: Characteristics, Byte Storage methods, Conceptual view of memory cell - Design of scalable memory using RAM's- ROM's chips - Construction of larger size memories - Memory Interleaving - Memory interface address map- Cache memory: principles, Cache memory management techniques, Types of caches, caches misses, Mean memory access time evaluation of cache.		

Dr. Erapaneni Gayatri
Assistant Professor
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore

Memory Unit

- **Memory unit** is essential component of digital computer since it is needed for storing programs and data.
 - Memory unit that communicates directly with CPU is called **Main memory**.
 - Only programs and data currently needed by processor reside in the main memory
- Devices that provide backup storage is called **Auxiliary (Secondary) Memory**
 - All other information is stored in auxiliary memory and transferred to main memory when needed.
- **The binary information may be instructions and data**



Memory Capacity

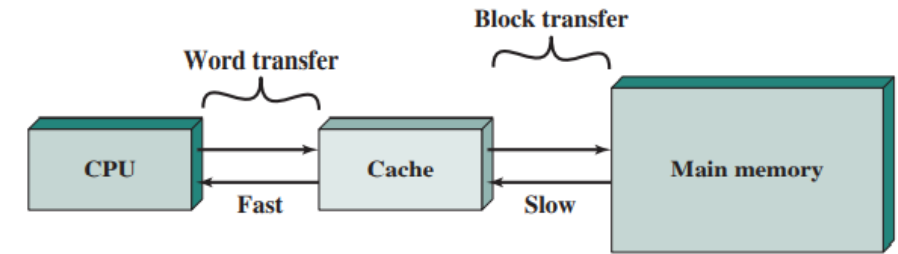
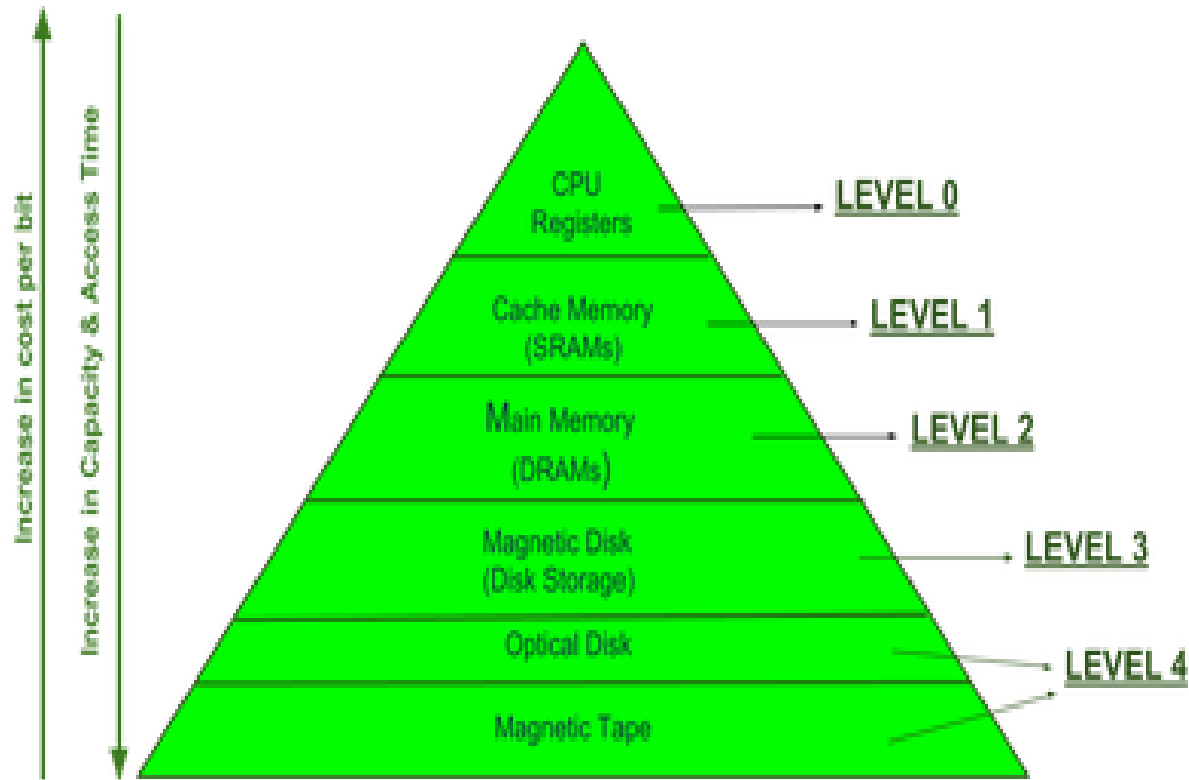
- Number of bytes that can be stored

Term	Normal Usage	Usage as Power of 2
K (Kilo)	10^3	$2^{10} = 1,024$
M (Mega)	10^6	$2^{20} = 1,048,576$
G (Giga)	10^9	$2^{30} = 1,073,741,824$
T (Tera)	10^{12}	$2^{40} = 1,099,511,627,776$

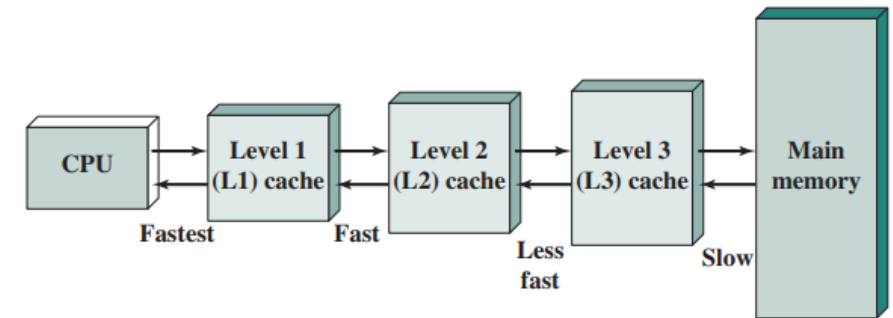
Units for Measuring Memory (Data Storage) Capacity:

1 Bit	= 1 Binary Digit
4 Bits	= 1 Nibble
8 Bits	= 1 Byte
$2^{10} = 1024$ Bytes	= 1 Kilobyte
$2^{20} = 1024$ Kilobyte	= 1 Megabyte
$2^{30} = 1024$ Megabyte	= 1 Gigabyte
$2^{40} = 1024$ Gigabyte	= 1 Terabyte
$2^{50} = 1024$ Terabyte	= 1 Petabyte
$2^{60} = 1024$ Petabyte	= 1 Exabyte

Memory Hierarchy



(a) Single cache



(b) Three-level cache organization

Figure 4.3 Cache and Main Memory

Memory Characteristics

- Location
 - CPU
 - Internal (main)
 - External (secondary)
- Capacity
 - Word size (in Bytes)
 - Number of words (Blocks)
- Unit of transfer
 - Word
 - Block
- Access methods
 - Sequential access
 - Direct access
 - Random access
 - Associative access
- Performance
 - Access time
 - Cycle time
 - Transfer rate
- Physical Type
 - Semiconductor
 - Magnetic surface
 - Optical
- Physical Characteristics
 - Volatile / Non-Volatile
 - Erasable / Non-erasable

Memory Characteristics

1. Location of memories

- CPU
 - Registers – used by CPU as its local memory
- Internal memory
 - Main memory
 - Cache memory
- External memory
 - Peripheral devices – disk, tape – accessible to CPU via I/O controllers



Memory Characteristics

2. Capacity & 3. Unit of Transfer

- **Word (Internal Memory)**

The **capacity of the internal memory** is typically expressed in terms of bytes or words

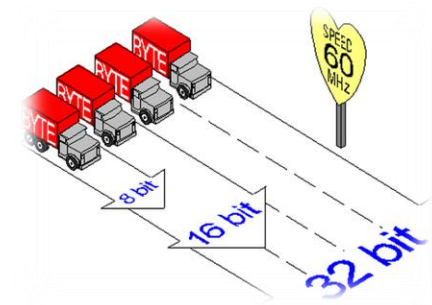
- **Word length**

For the internal memory, the unit of transfer is equal to the **number of data lines** into and out of the main memory module (word Length). The common word lengths are **8,16 and 32 bits**.

$$\text{Total memory} = \text{number of words} \times \text{word length}$$

- **Block (External Memory)**

External memory capacity is expressed in terms of blocks. For external memory, the data often transferred in much longer units than a word, and these are referred to as Blocks.

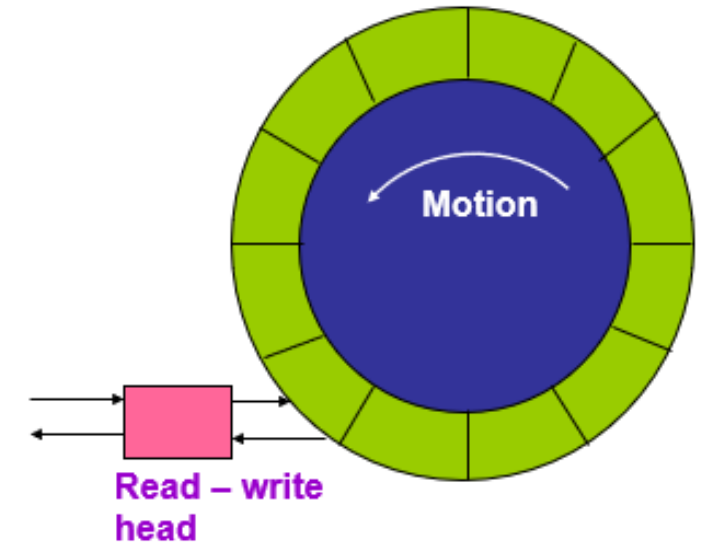
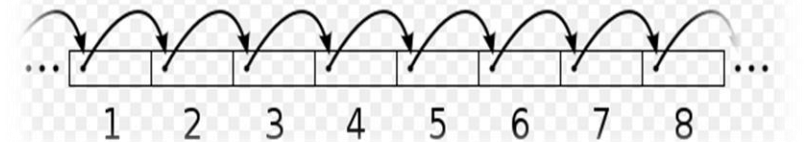


Memory Characteristics

3. Access Methods

Sequential Access

- Accesses the memory in predetermined sequence
- Shared read/write head is used, and this must be moved its current location to the desired location, passing and rejecting each intermediate record.
- Memory is organized into units of data, called Records. Access must be made in a specific linear sequence
- The time to access data in this type of method depends on the location of the data. So, the time to access an arbitrary record is highly variable
- Ex: Magnetic Tapes, data in memory array
- Slower than random access memory

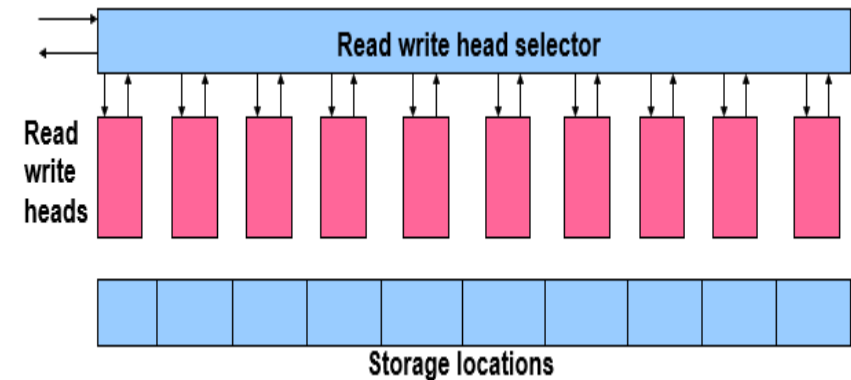
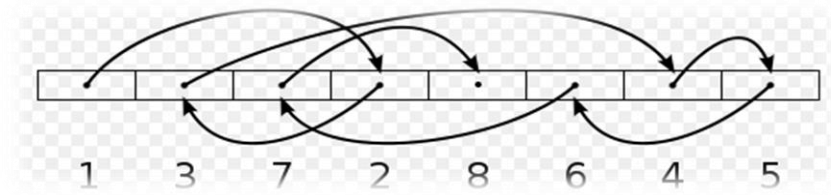


Memory Characteristics

3. Access Methods

Random access

- In random access method, data from any location of the memory can be accessed randomly.
- The access to any location is not related with its physical location and is independent of other locations.
- There is a separate access mechanism for each location.
- Main memory systems are a random access
- Storage locations can be accessed in any order
- Example of random access: Semiconductor memories like RAM, ROM use random access method.

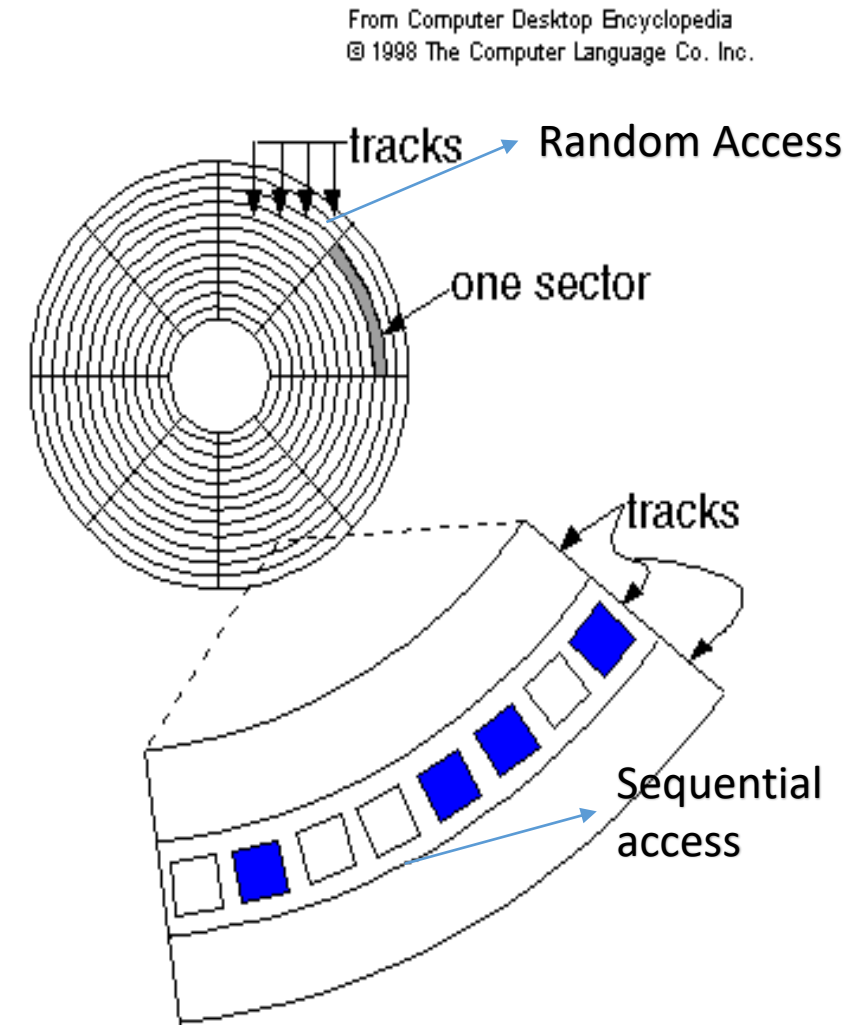


Memory Characteristics

3. Access Methods

Direct access

- Direct access method can be seen as **combination of sequential access method and random access method**. It is also referred as **semi random access** memory
- Magnetic hard disks contain many rotating storage tracks.
- Here each tracks has its own read or write head and the **tracks can be accessed randomly**. But **access within each track is sequential**.
- Access is accomplished by general access to reach a general vicinity plus sequential searching, counting, waiting to reach the final location.
- Example of direct access: Memory devices such as **magnetic hard disks**.

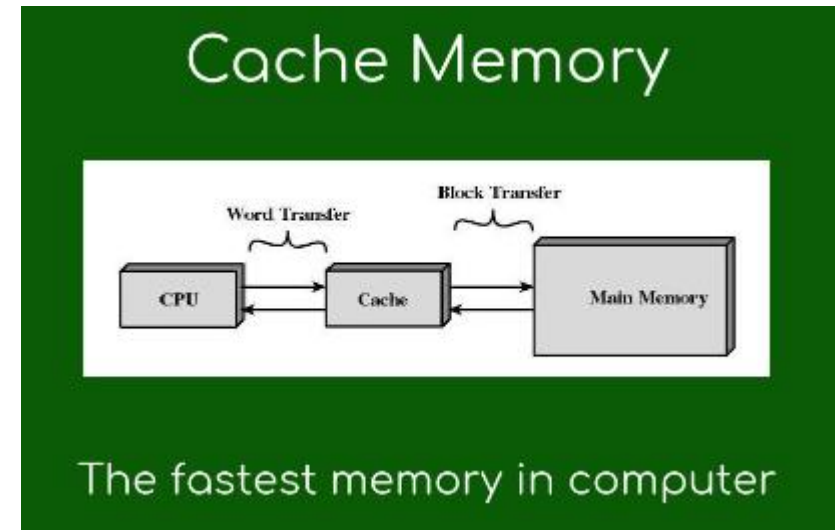


Memory Characteristics

4. Access Methods

Associate Access

- Word is retrieved based on **portion of its contents rather than its address**
- This enables one to make a **comparison of desired bit locations within a word for specific match**
- Has own addressing mechanism
- Retrieval time is constant
- Access time is independent of location or prior access patterns
- **Example: Cache memories**



Memory Characteristics

4. Performance

Access time

- The **time required to read / write the data from / into desired record**
- Depends on the amount of data to be read / write
- If the amount data is uniform for all records then the access time is same for all records.
- Time from the instant that an address is presented to the memory to the instant that data have been stored or made available for use.

Memory Cycle time

- **Access time + time required before a second access can commence**
- For Random access method ,this memory cycle time is same for all records
- The sequential access and direct access ,the memory cycle time is different

Transfer rate / Throughput

- **Rate at which the data can be transferred** into or out of a memory unit
- **Random access memory:** 1/cycle time
- **Non-Random access memory**

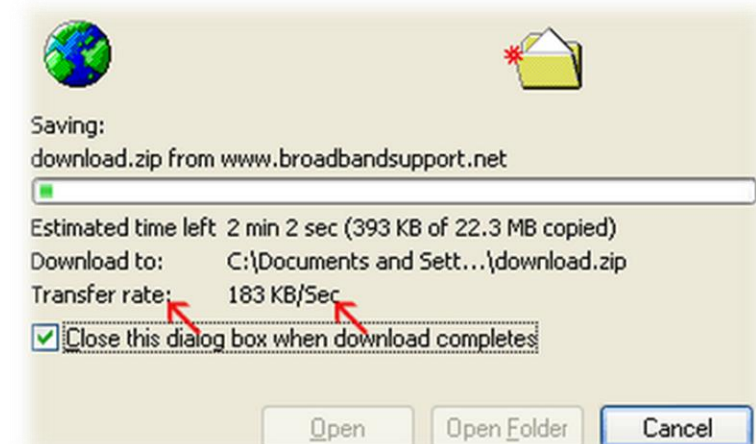
$$T_n = T_a + (N/R), \text{ where}$$

T_n – average time to read or write N bits

T_a – average access time

N – Number of bits

R – Transfer rate in bits per second (BPS)



Memory Characteristics

Memory Performance Parameters

Symbol	Definition	Units	Meaning
t_a	Access time	time	Time to access a memory word
t_c	Cycle time	time	Time from start of access to start of next access
k	Block size	words	Number of words per block
ω	Bandwidth	words/time	Word transmission rate
t_l	Latency	time	Time to access first word of a sequence of words
$t_{bl} = t_l + k/\omega$	Block access time	time	Time to access an entire block of words

Activate V
Go to Setting

(Information is often stored and moved in blocks at the cache and disk level.)

Memory Characteristics

5. Physical type

Semiconductor

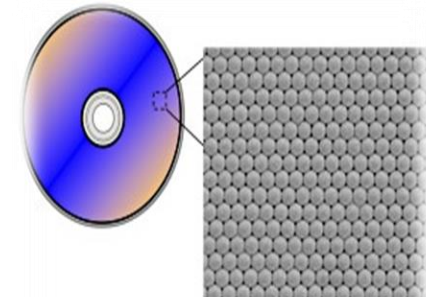
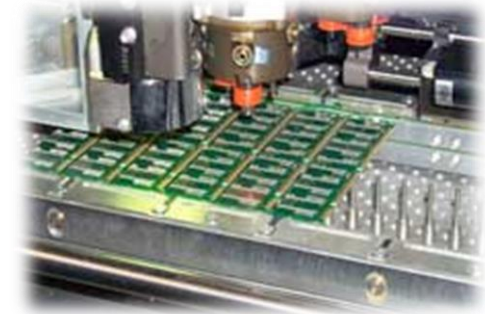
- Semiconductor memory uses **semiconductor-based integrated circuits** to store information.

Magnetic surface

- Magnetic storage uses **different patterns of magnetization on a magnetically coated surface** to store information.
- Example: Magnetic disk, Floppy disk, Hard disk drive

Optical

- The typical optical **disc**, stores information in deformities on the surface of a circular disc and reads this information by illuminating the surface with a **laser diode and observing the reflection**.



Memory Characteristics

6. Physical characteristics

Erasable/non erasable

- Erasable memory
 - Erase the stored information by writing new information
 - Ex: Magnetic storage is erasable
- Non-erasable memory
 - Cannot be altered, except by destroying the storage unit (ROM)
 - A practical non-erasable memory must also be non-volatile
 - Ex: CD-R, Flash Memories

Volatile/non volatile

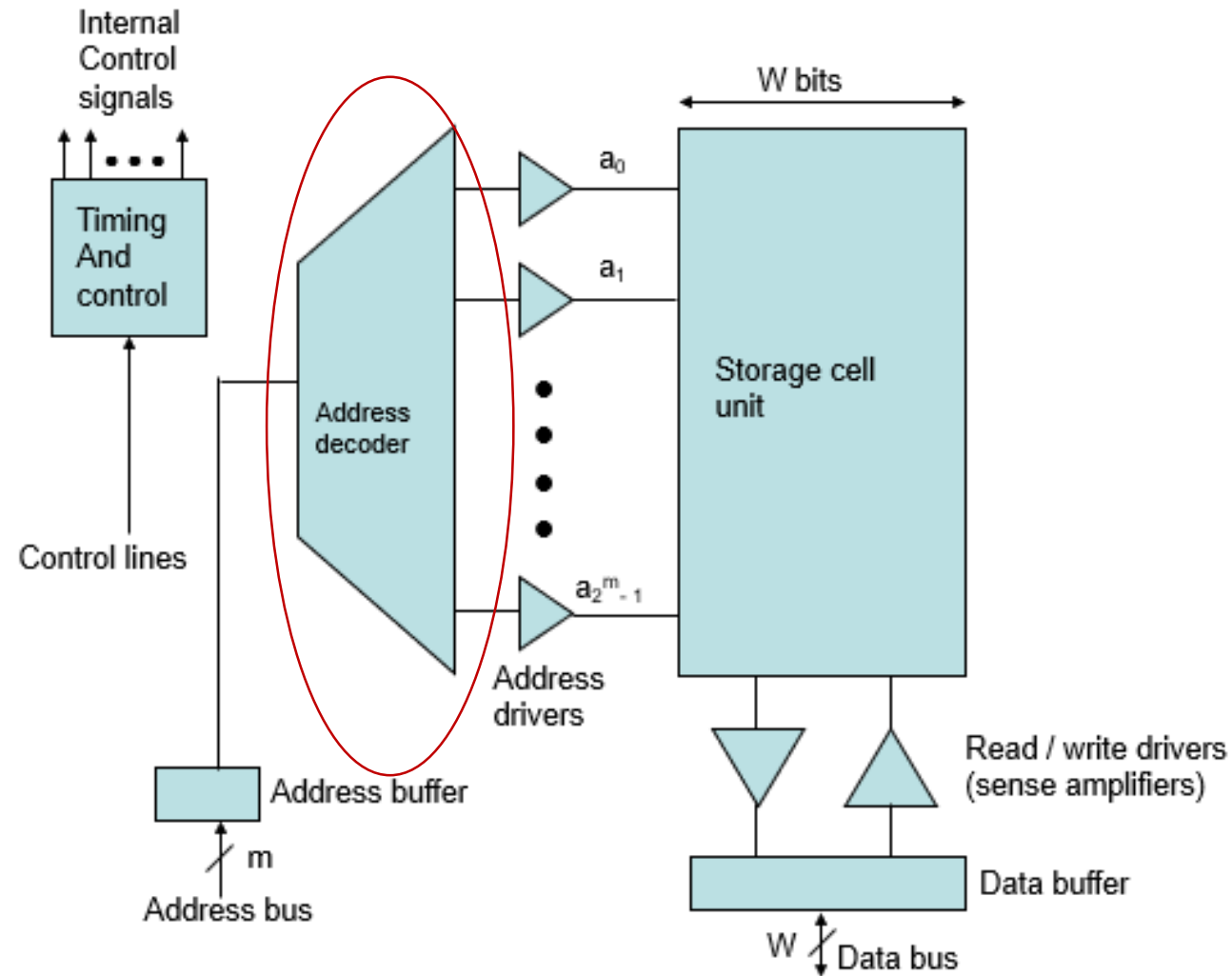
- Volatile
 - Volatile Memory Requires constant power to maintain the stored information.
 - Information decays naturally or lost when electrical power is switched off
 - A type of volatile semiconductor memory is random access memory
- Non-Volatile
 - Once recorded is retained until deliberately changed
 - No electrical power is needed to retain information
 - Magnetic storage is non-volatile.
 - A type of non-volatile semiconductor memory known as flash memory

Memory Organization

- Physical arrangement of bits to form words
- 2 types
 - 1 dimensional
 - 2 dimensional
- Basic element = memory cell
- Properties of Memory cell:
 - They exhibit **two stable states**, which can be used to represent binary 1 and 0.
 - They are capable of being written into (at least once) to set the state.
 - They are capable of being read to sense the state.

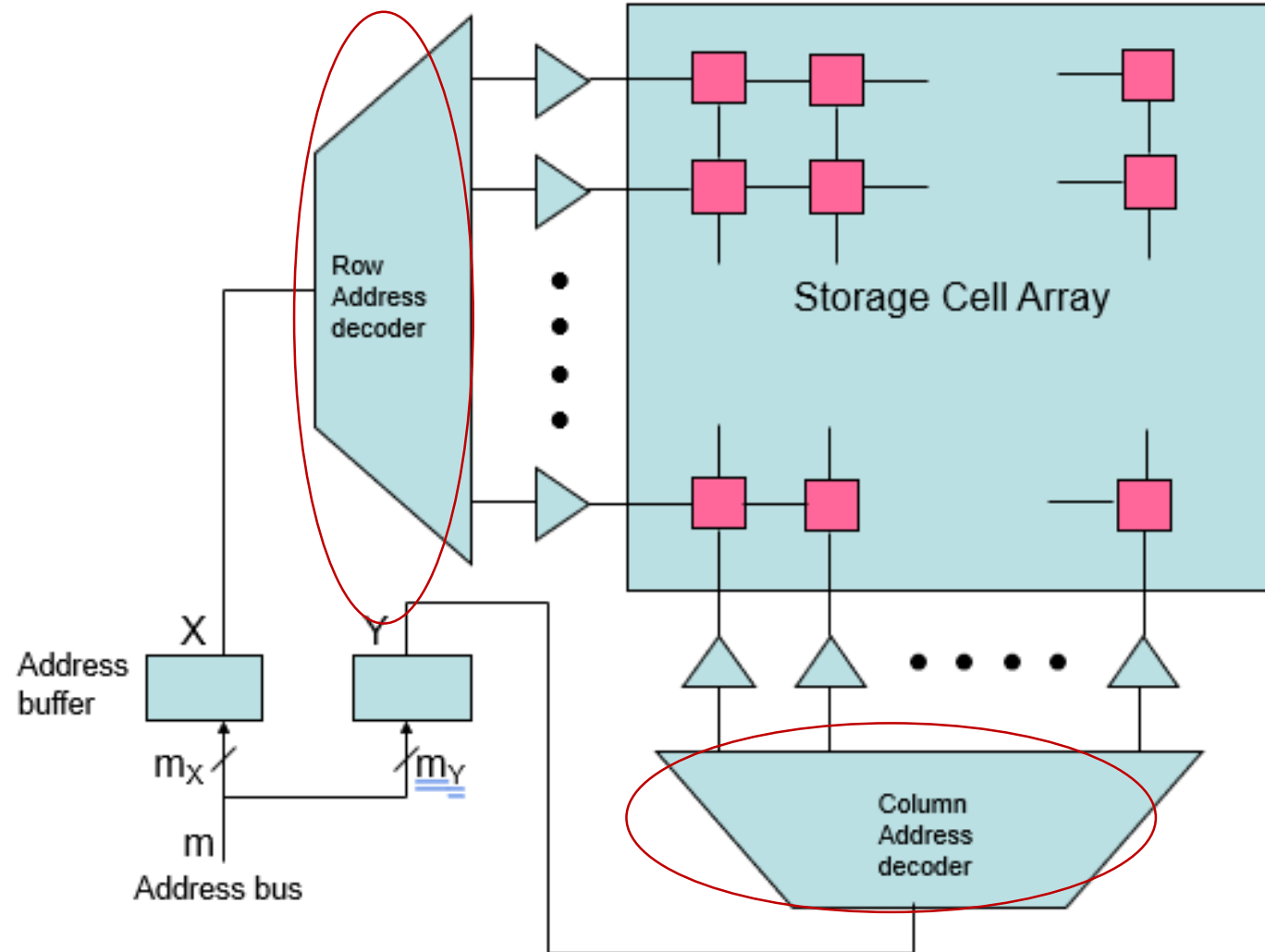
Memory Organization

1 – dimensional organization



Memory Organization

2 – dimensional organization

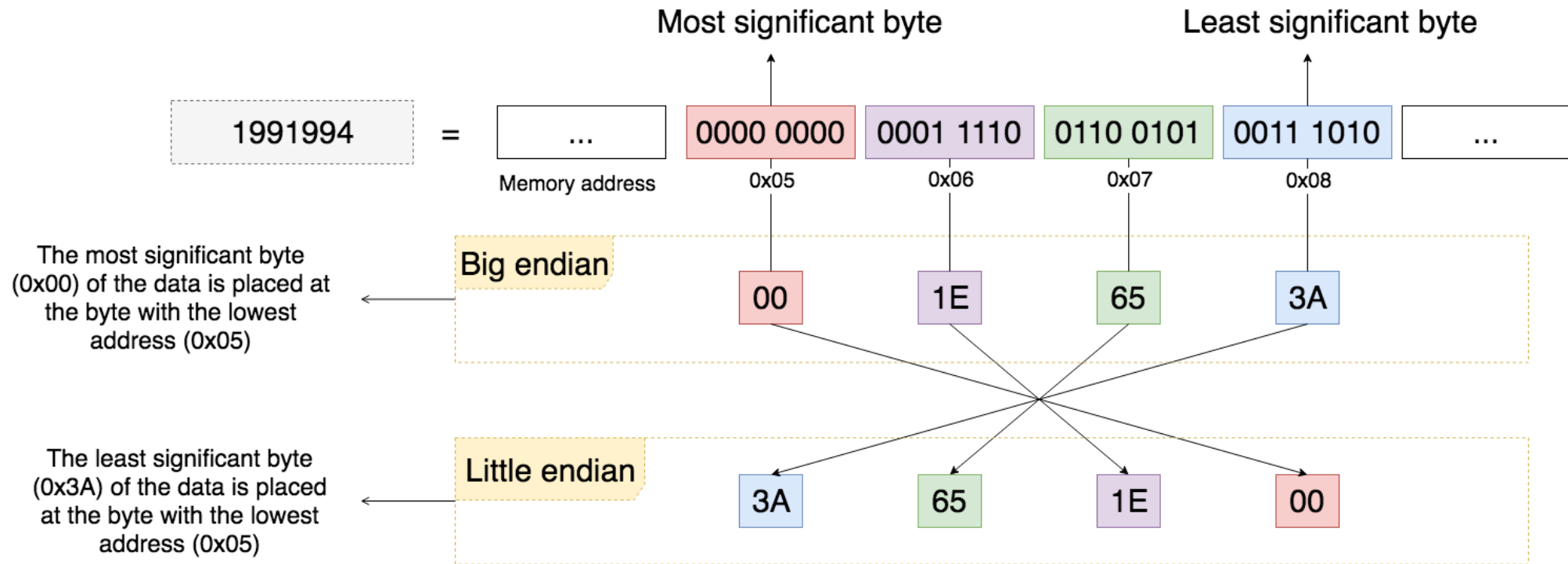


Byte Storage Methods

- Two ways to store a string of data (Bytes) in computers:
 - Big Endian
 - Little Endian
- **Least Significant Byte (LSB)** is the right-most bit in a string - because it has the least effect on the value of the binary number.
- **Most Significant Byte (MSB)** - the left-most byte that carries the greatest numerical value.
- **In Big Endian**, the MSB of the data is placed at the byte with the lowest address. (First byte stored in the memory first)
- In **Little Endian**, the LSB of the data is placed at the byte with the lowest address. (Last byte stored in the memory first)

Byte Storage Methods

- In **Big Endian**, the MSB of the data is placed at the byte with the lowest address. (First byte stored in the memory first)



- In **Little Endian**, the LSB of the data is placed at the byte with the lowest address. (Last byte stored in the memory first)

Byte Storage Methods

- **Big Endian** - the most common format in data networking (TCP, UDP, IPv4 and IPv6 are using Big endian order to transmit data)
- **Little Endian** - on microprocessors

- **Show the big endian and small endian arrangements of the bytes in 12345678_{16}**

Address →	00	01	10	11
Big Endian	12	34	56	78
Little Endian	78	56	34	12

Byte Storage Methods

- **Big-Endian**

- Assigns MSB to least address and LSB to highest address
- Ex: $0 \times \text{DEADBEEF}$

Memory Location	Value
Base Address + 0	DE
Base Address + 1	AD
Base Address + 2	BE
Base Address + 3	EF

Byte Storage Methods

- Little Endian

- Assigns MSB to highest address and LSB to least address
- Ex: $0 \times \text{DEADBEEF}$

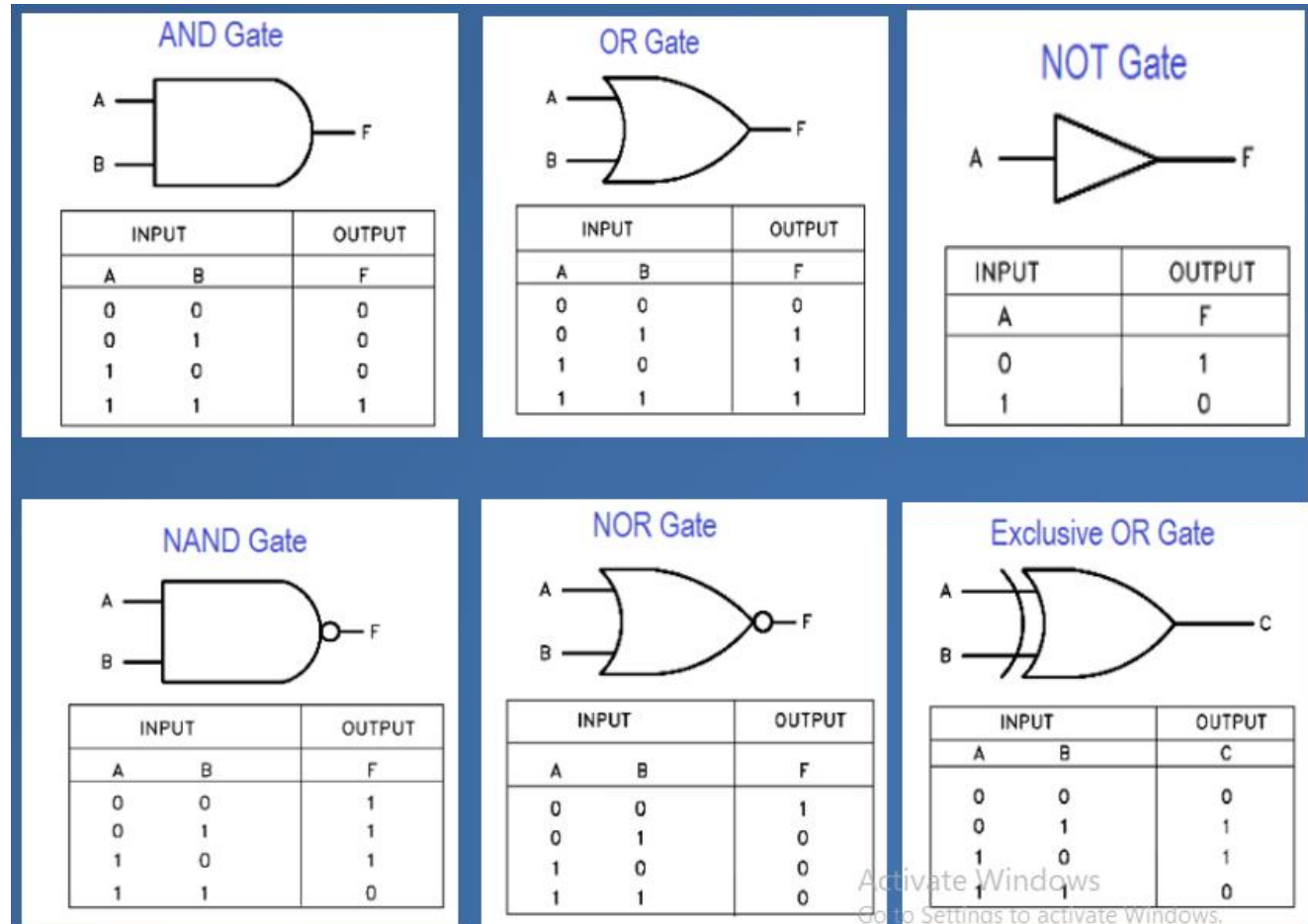
Memory Location	Value
Base Address + 0	EF
Base Address + 1	BE
Base Address + 2	AD
Base Address + 3	DE

Byte Storage Methods

- Little Endian
 - Intel × 86 family
 - Digital equipment corporation architectures (PDP – 11, VAX, Alpha)
- Big Endian
 - Sun SPARC
 - IBM 360 / 370
 - Motorola 68000
 - Motorola 88000
- Bi-Endian (The ability to switch between big endian and little endian ordering.)
 - Power PC
 - MIPS
 - Intel's 64 IA - 64

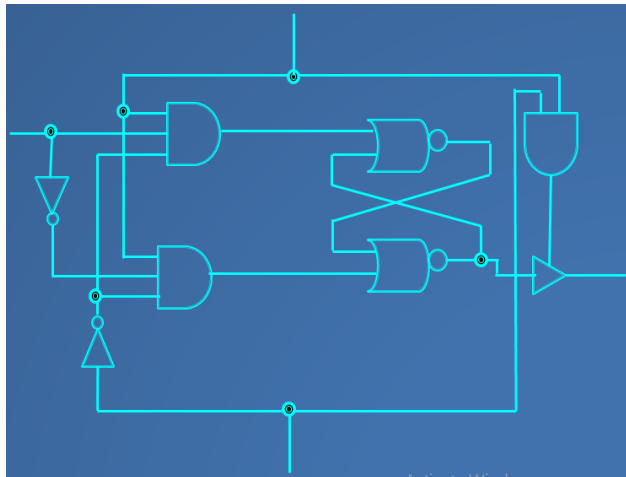
Conceptual View of Memory Design

- Revisiting Logic Gates

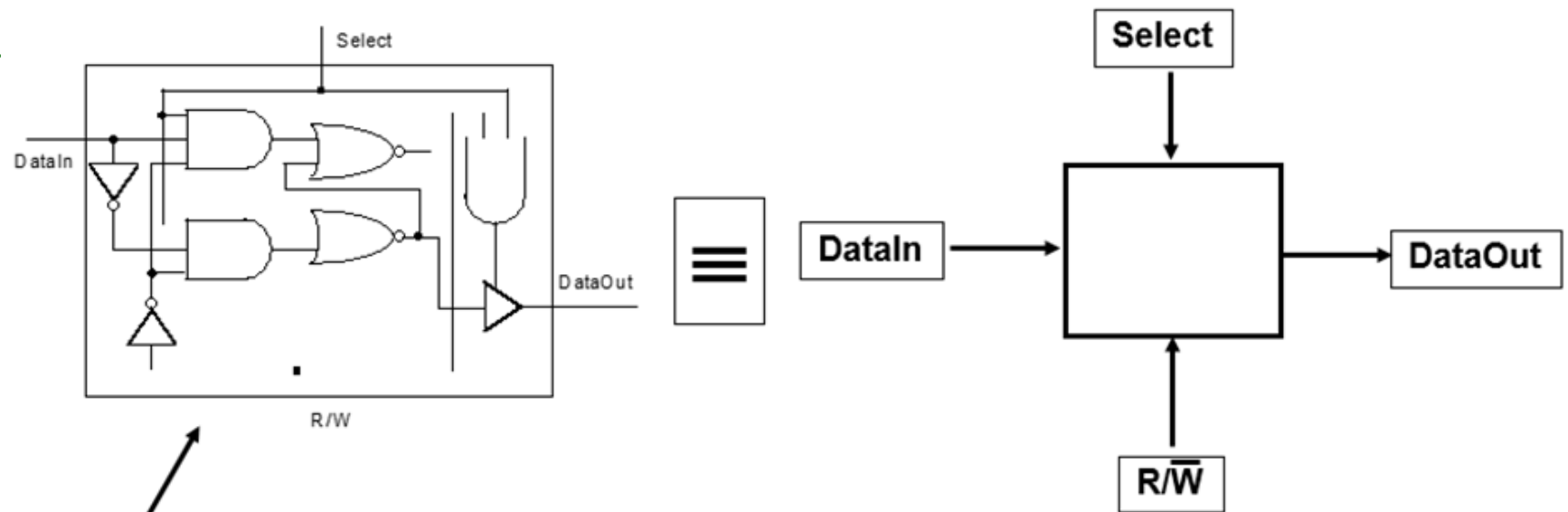


Conceptual View of Memory Design

A Single RAM Memory Cell



Regardless of the technology, all RAM memory cells must provide these four functions: Select, DataIn, DataOut, and $R/\bar{W}$.

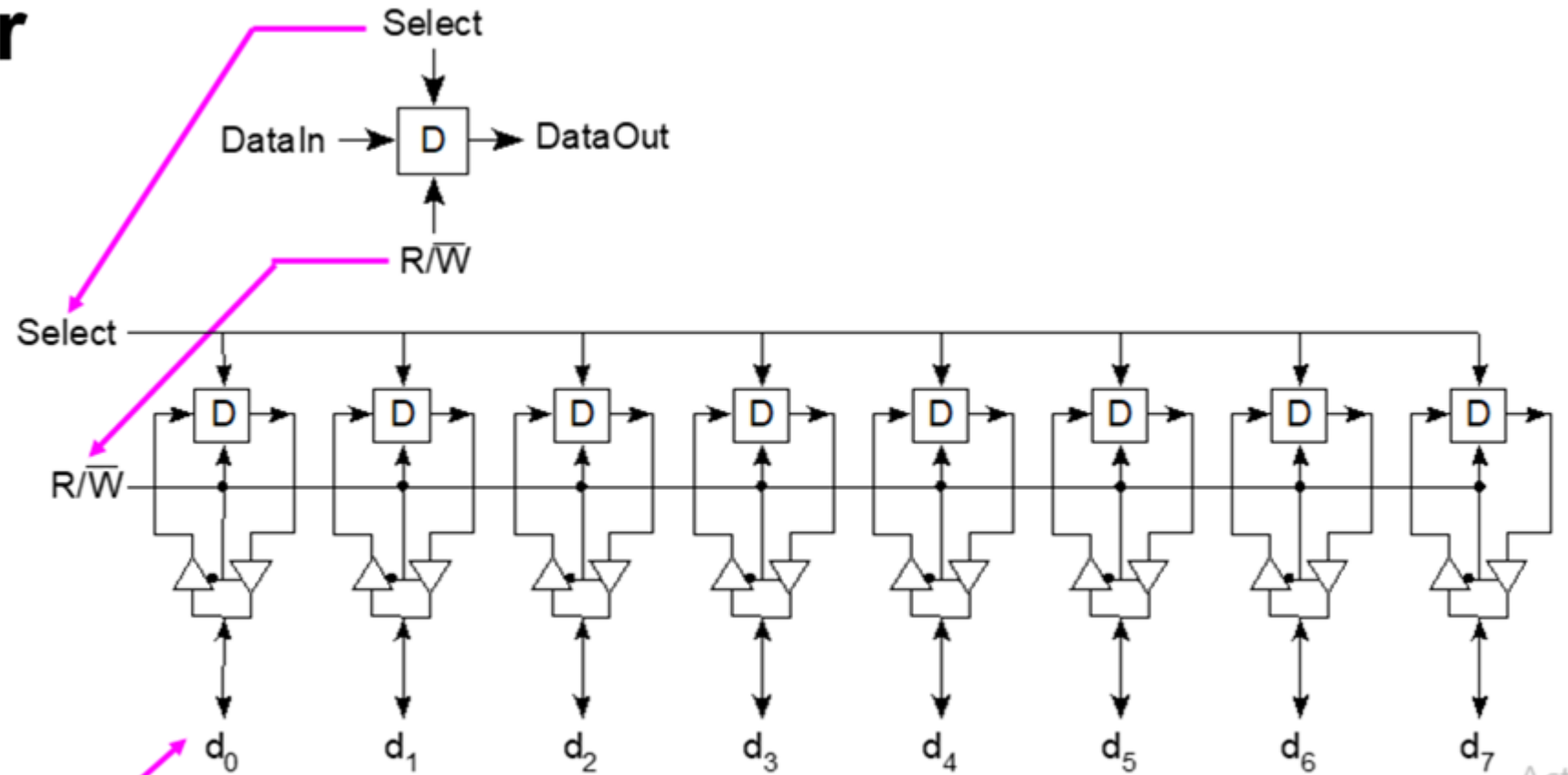


This “static” RAM cell is unrealistic in practice, but it is functionally correct.

Conceptual View of Memory Design

The entire register is selected with one select line, and uses one $R/\bar{W}$ line

An 8-Bit Register as a 1-D RAM Array

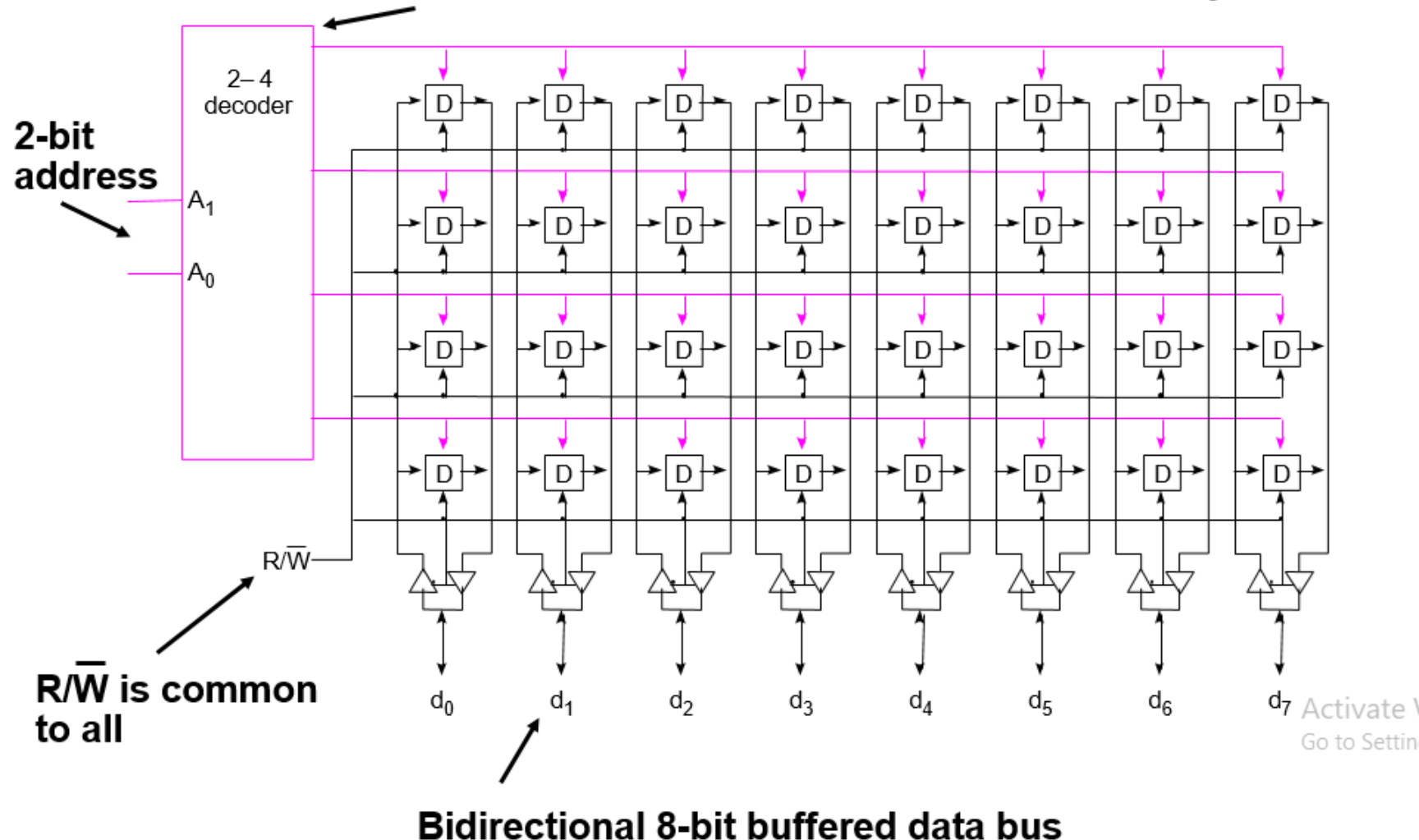


Data bus is bidirectional and buffered. (Why?)

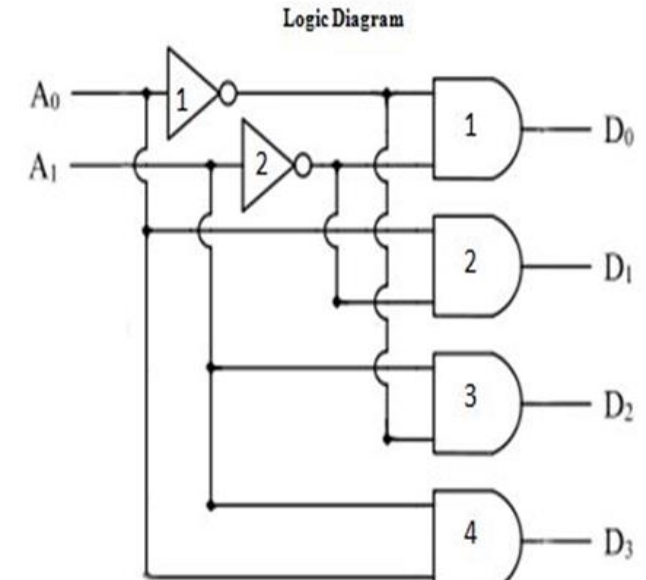
Conceptual View of Memory Design

A 4 x 8 2-D Memory Cell Array

2-4 line decoder selects one of the four 8-bit arrays



2-4 Decoder Circuit



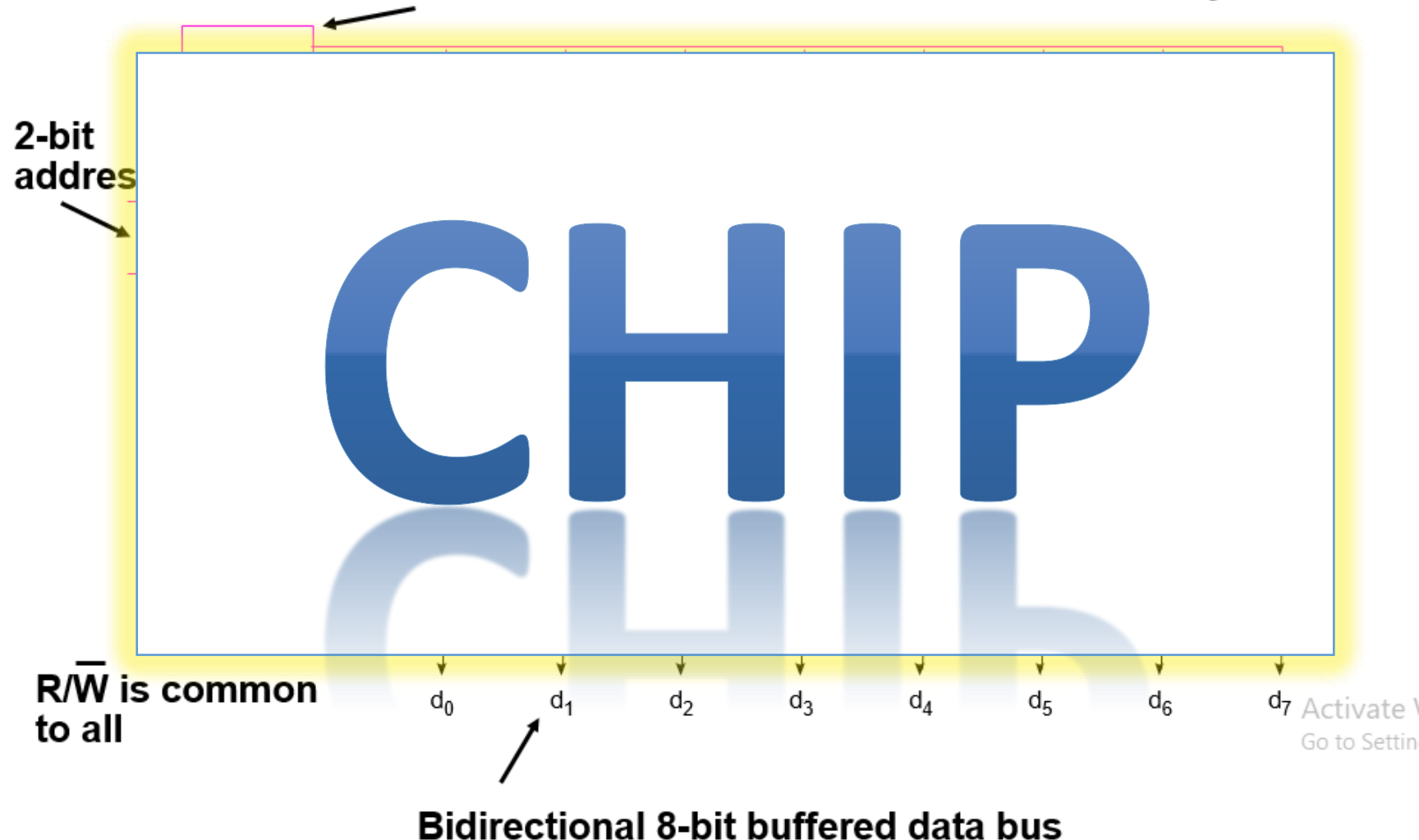
Truth Table

A_1	A_0	D_3	D_2	D_1	D_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

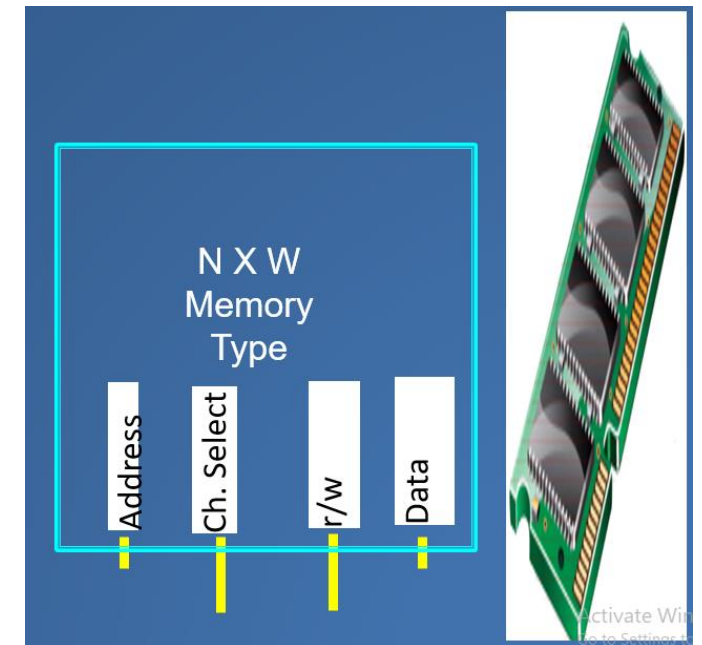
Conceptual View of Memory Design

A 4 x 8 2-D Memory Cell Array

2-4 line decoder selects one of the four 8-bit arrays



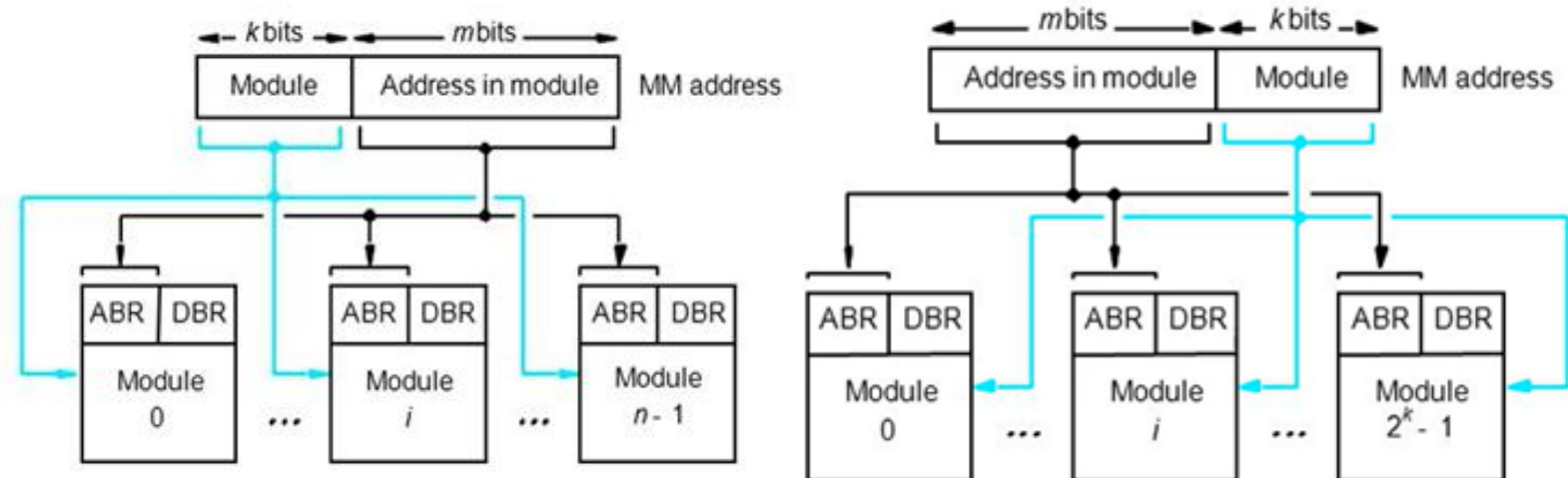
A typical chip Layout



Memory Interleaving

Main Memory Structure:

- If the Main Memory is structured as collection of physically separate modules - each with its own Address Buffer Register (ABR) and Data Buffer Register (DBR), memory access operations....
 - It may proceed in more than one module at the same time.
- Hence, aggregate rate of transmission of words to and from the memory can be increased.



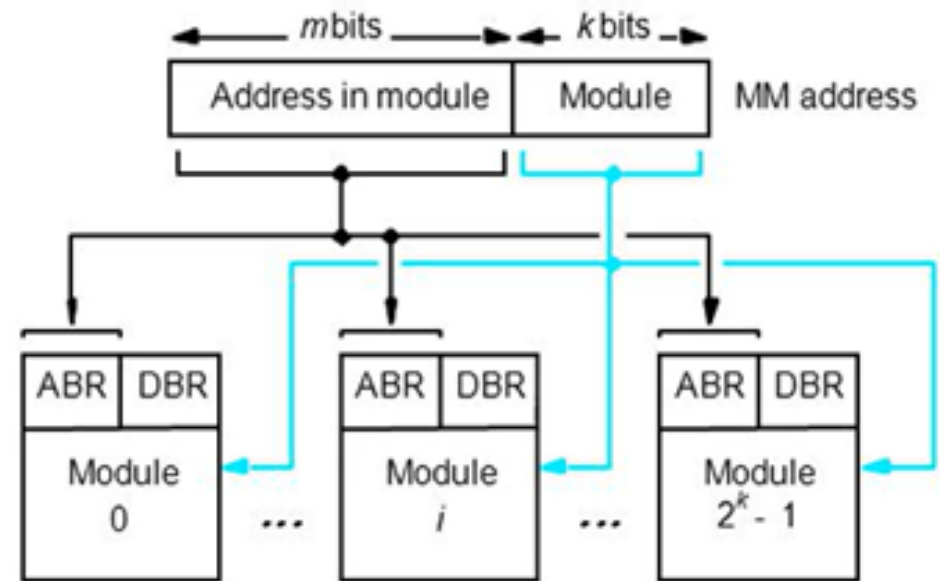
(a) Consecutive words in a module

(b) Consecutive words in consecutive modules

Memory Interleaving

Interleaving:

- Memory Interleaving is an **abstraction technique**.
- Designed to compensate for core memory by **spreading memory addresses evenly across memory banks**.
- Divides memory into a number of modules such that successive words in the address space are placed in the different module.
- To implement interleaved structure, there must be **2^k modules**.
(k =lower order k bits)



(b) Consecutive words in consecutive modules

Memory Interleaving

Usage of Memory Interleaving:

- Main memory is relatively slower than the cache.
- So to improve the access time of the main memory, interleaving is used.
- This method uses memory effectively.

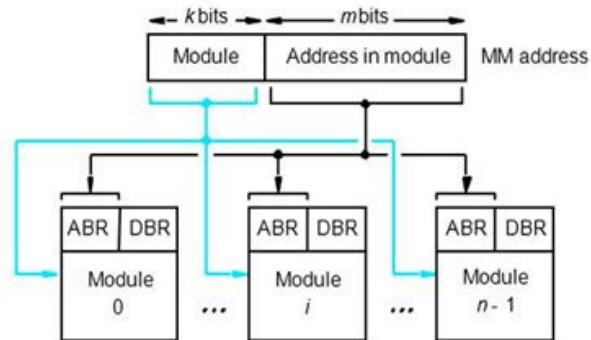
Classification of Memory Interleaving: (Two address formats for Memory Interleaving)

- High Order Interleaving
- Low Order Interleaving

Memory Interleaving

Distribution Methods in Memory system

- Consecutive words in a module (**High-Order Interleaving**)
 - When consecutive locations are accessed, as happens when a block of data is transferred to a cache, only one module is involved.



(a) Consecutive words in a module

Lower order bits	0000	10
Higher order bits	0001	20
	0010	30
	0011	40
	0100	50
	0101	60
	0110	70
	0111	80
	1000	90
	1001	100
	1010	110
	1011	120
	1100	130
	1101	140
	1110	150
	1111	160

	Module 00
00	10
01	20
10	30
11	40

	Module 01
00	50
01	60
10	70
11	80

	Module 10
00	90
01	100
10	110
11	120

	Module 11
00	130
01	140
10	150
11	160

Memory Interleaving

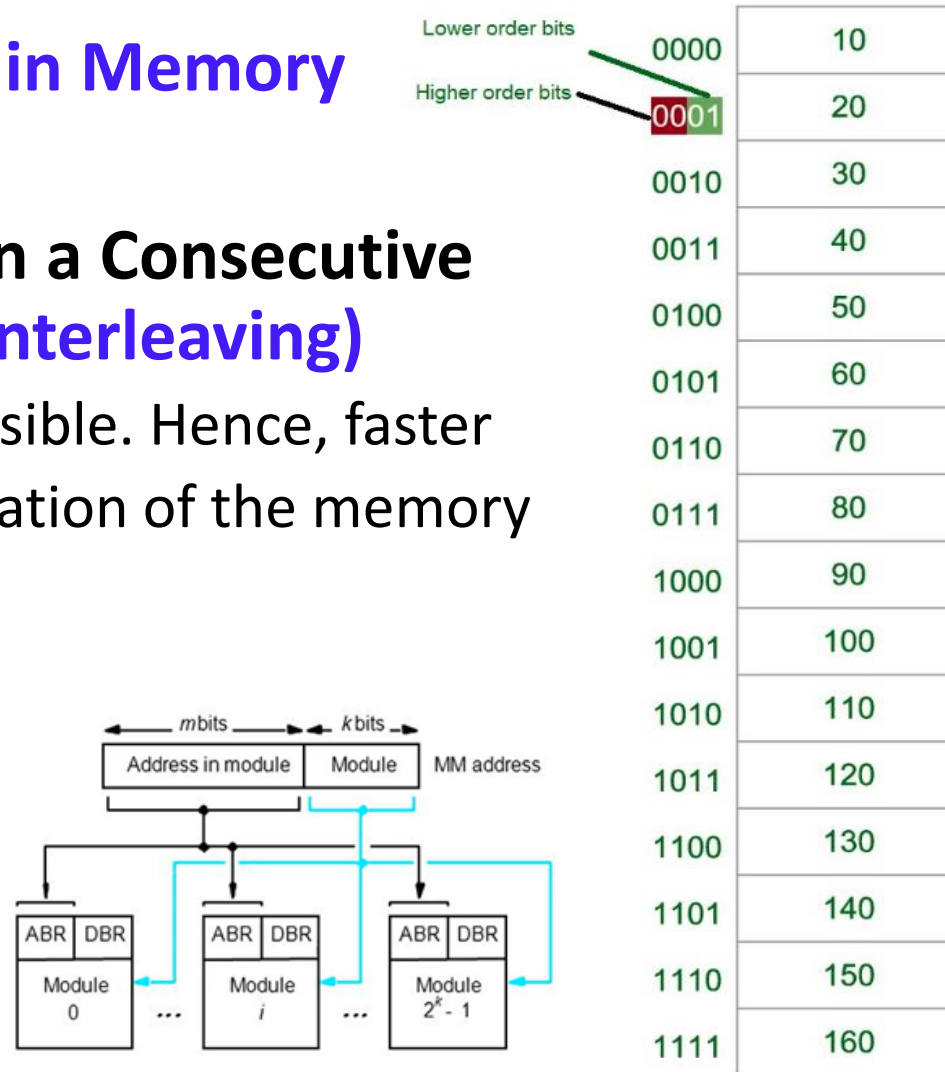
High Order Interleaving:

- In high-order interleaving, the most significant bits of the address select the memory chip. (module)
- The least significant bits are sent as addresses to each chip.
- The maximum rate of data transfer is limited by the memory cycle time.

Memory Interleaving

Distribution Methods in Memory system

- Consecutive words in a Consecutive module(**Low Order Interleaving**)
 - Parallel access is possible. Hence, faster
 - Higher average utilization of the memory system



(b) Consecutive words in consecutive modules

	Module 00	Module 01
00	10	20
01	50	60
10	90	100
11	130	140

	Module 10	Module 11
00	30	40
01	70	80
10	110	120
11	150	160

Memory Interleaving

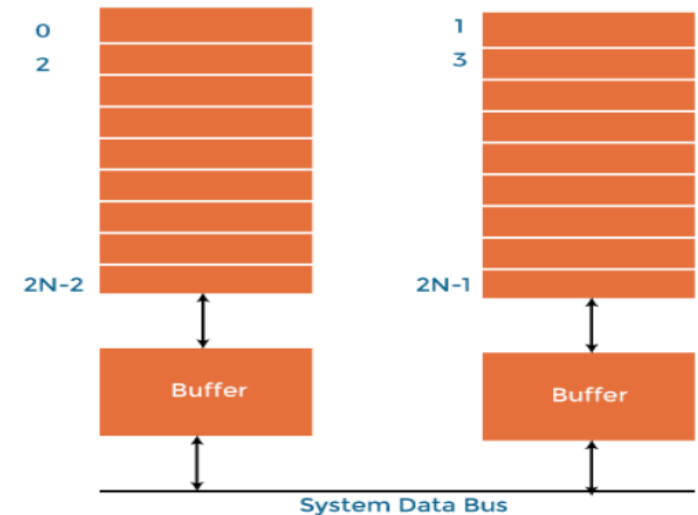
Low Order Interleaving:

- In low-order interleaving, the least significant bits select the memory bank (module).
- In this, consecutive memory addresses are in different memory modules.
- This allows memory access at much faster rates than allowed by the cycle time.

Memory Interleaving

Benefits of Memory Interleaving

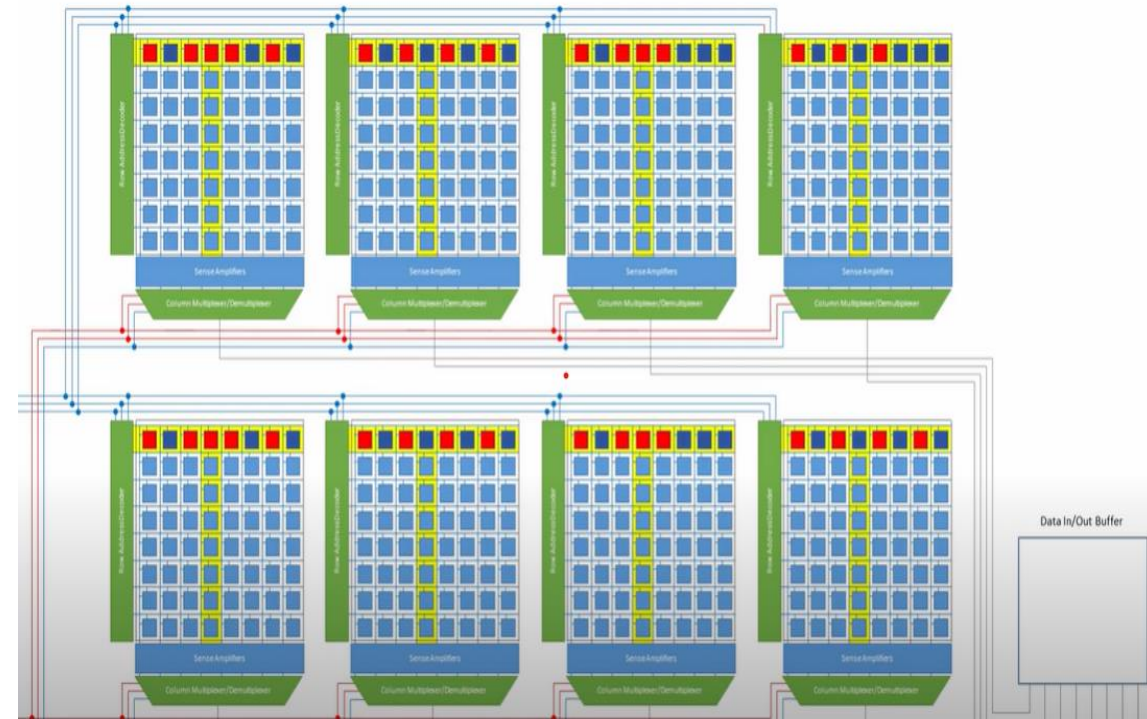
- It allows **simultaneous access to different modules** of memory.
- Interleave memory is useful in the system with **pipelining and vector processing**.
- In an interleaved memory, consecutive memory addresses are spread across different memory modules.
- **Reduce the memory access time by a factor close to the number of memory banks.**



Memory Interleaving

Interleaving DRAM

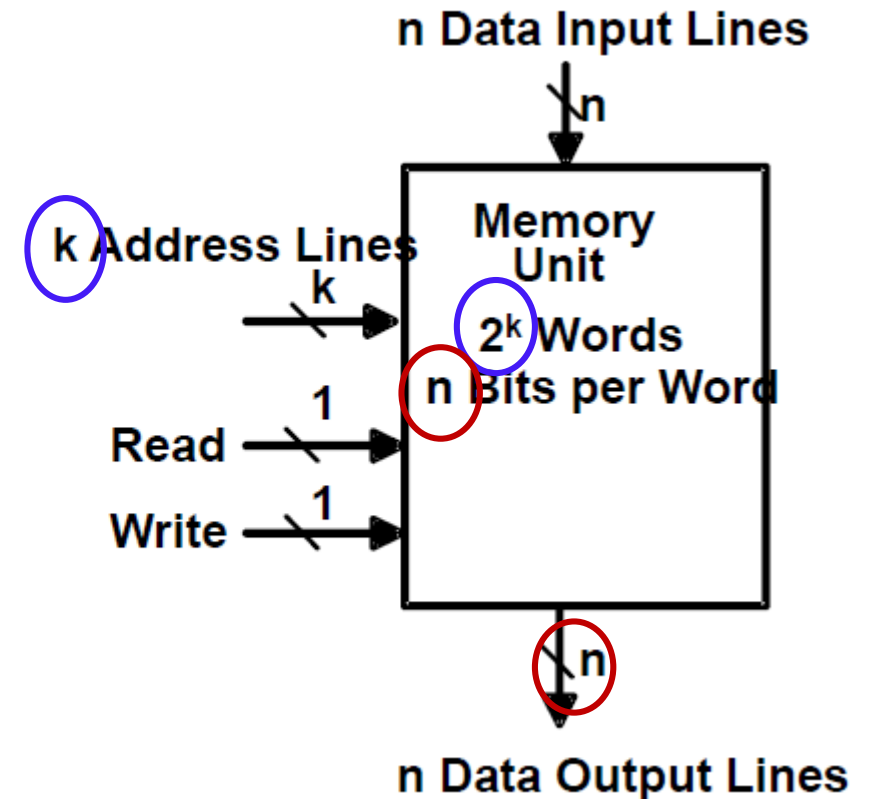
- Main memory is usually composed of a collection of DRAM (Dynamic random-access memory) memory chips grouped together to form a memory bank.
- The memory banks will be interleaved.
- Memory accesses to different banks can proceed in parallel with high throughput.
- Memory banks can be allocated a contiguous block of memory addresses, gives an equal performance and access gives far better performance in interleaved layouts.



Design of scalable memory using RAM's

Memory - Block Diagram

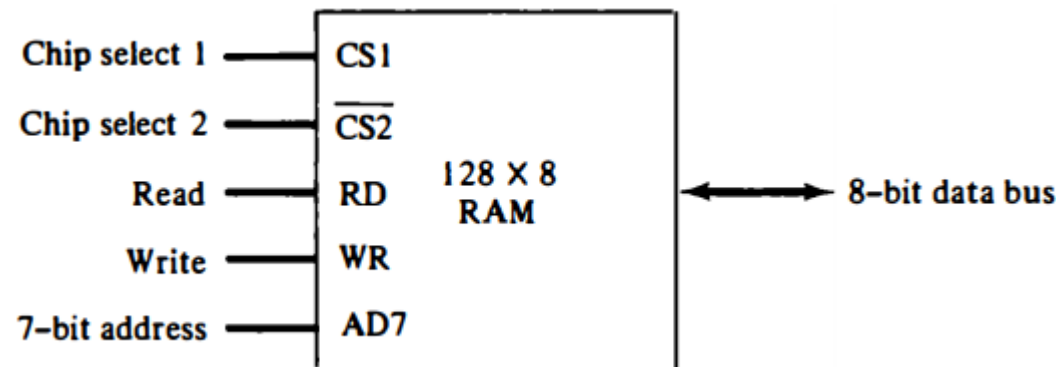
- A basic memory system is shown here:
- k address lines are decoded to address 2^k words of memory.
- Each word is n bits.
- Read and Write are single control lines defining the simplest of memory operations.



Design of scalable memory using RAM's

RAM Chips

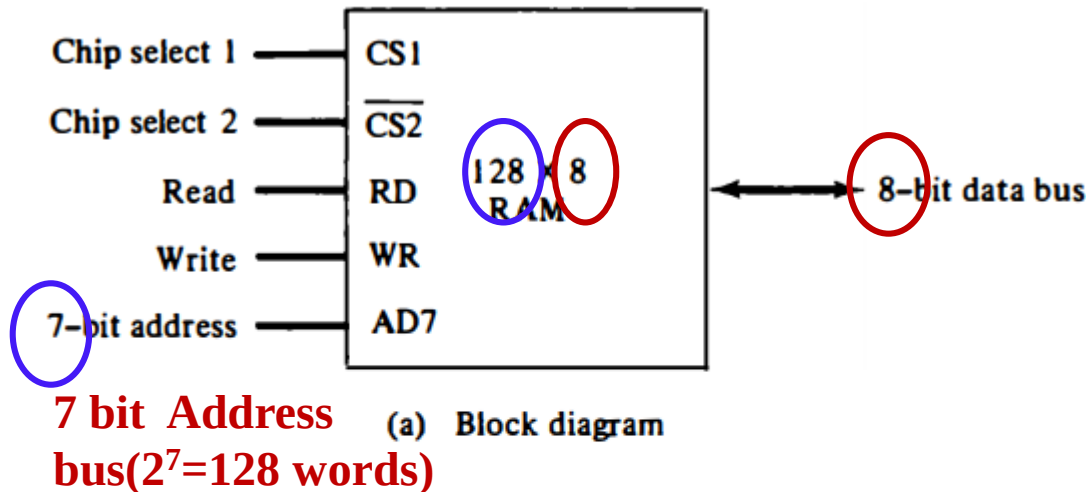
- Features:
 - RAM Chips is suited for the communication with the CPU if it has one or more control inputs that select the chip only when needed
 - Bidirectional Databus – that allows the data either from CPU to memory or memory to CPU
 - Bidirectional Data Bus can be constructed with three state Buffers
 - » Logic 0
 - » Logic 1
 - » High Impedence state
 - Capacity of the Memory is 128 words of eight bits
 - 7 bit Address and 8 –bit bidirectional data bus
 - Read & Write -Memory operation
 - Chip Select control Inputs are for enabling the chip only when it is selected by the microprocessor



Design of scalable memory using RAM's

RAM Chips

Figure 2 Typical RAM chip.



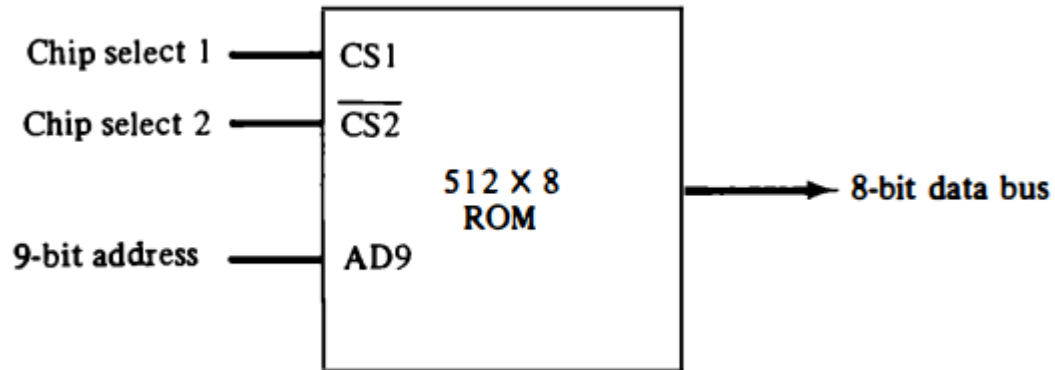
CS1	$\overline{CS2}$	RD	WR	Memory function	State of data bus
0	0	x	x	Inhibit	High-impedance
0	1	x	x	Inhibit	High-impedance
1	0	0	0	Inhibit	High-impedance
1	0	0	1	Write	Input data to RAM
1	0	1	x	Read	Output data from RAM
1	1	x	x	Inhibit	High-impedance

(b) Function table

- The logic 1 and 0 are normal digital signals.
- High impedance state behaves like an **open circuit**, which means that the **output does not carry a signal and has no logic significance**.
- The unit is in operation only when $CS1 = 1$ and $\overline{CS2} = 0$. The bar on top of the second select variable indicates that **this input is enabled when it is equal to 0**.
- If the chip select inputs are not enabled, or if they are enabled but the read or write inputs are not enabled, the **memory is inhibited (temporarily inaccessible)** and its data bus is in a **high-impedance state (the output of a buffer is disconnected from the output bus)**.
- When $CS1 = 1$ and $\overline{CS2} = 0$, the memory can be placed in a write or read mode.
- When the WR input is enabled, the memory stores a byte from the data bus into a location specified by the address input lines.
- When the RD input is enabled, the content of the selected byte is placed into the data bus. The RD and WR signals control the memory operation as well as the bus buffers associated with the bidirectional data bus.

Read Only Memory (ROM)

ROM Chips

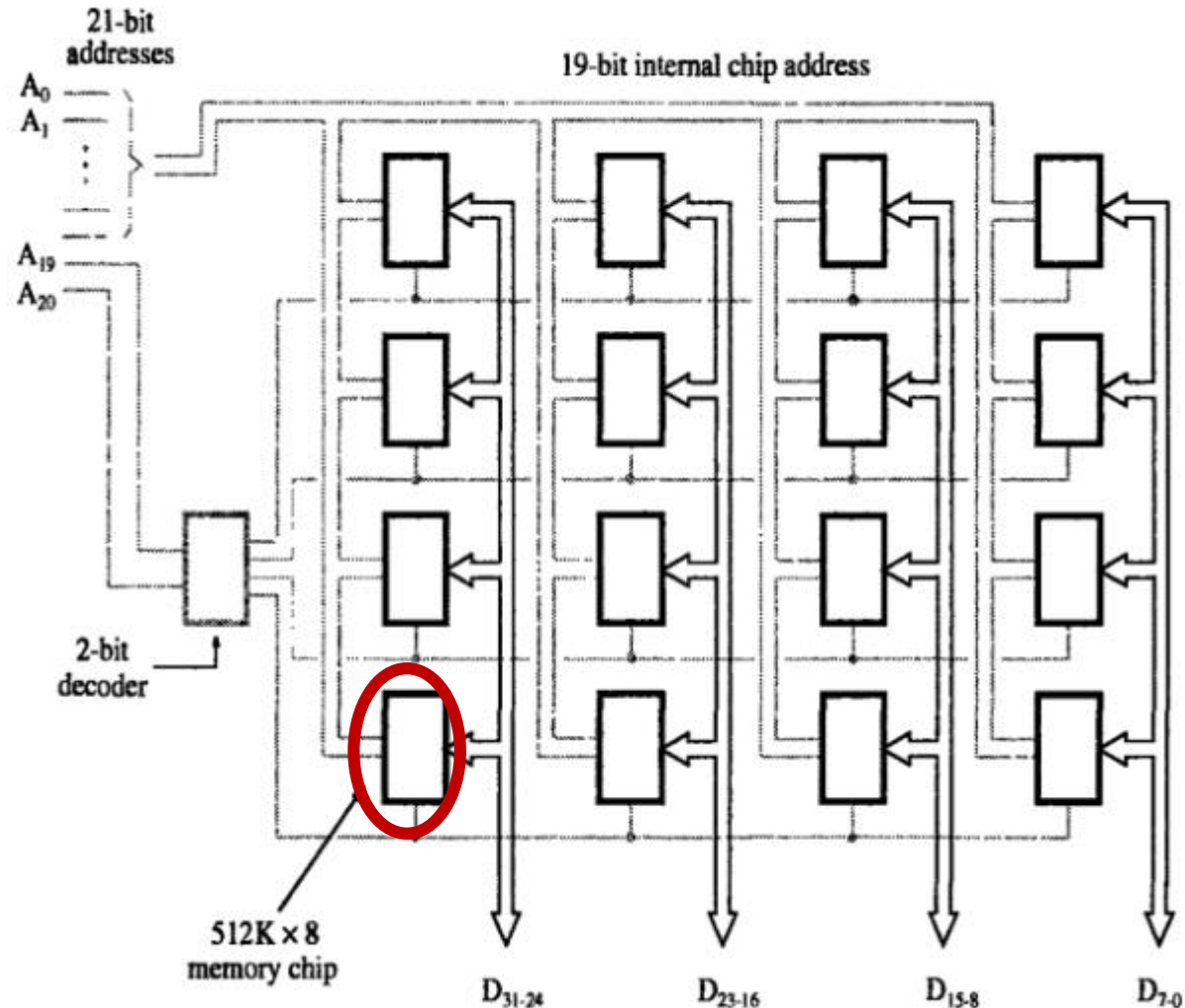


- No R/W signal – Default Read only.
- Data Bus - unidirectional

- A ROM chip is organized externally in a similar manner. However, since a ROM can only read, the data bus can only be in an output mode
- For the same-size chip, it is possible to have more bits of ROM than of RAM, because the internal binary cells in ROM occupy less space than in RAM.
- For this reason, the diagram specifies a 512-byte ROM, while the RAM has only 128 bytes.
- The nine address lines in the ROM chip specify any one of the 512 bytes stored in it.
- The two chip select inputs must be CS1 = 1 and $\overline{\text{CS2}} = 0$ for the unit to operate. Otherwise, the data bus is in a high-impedance state.
- There is no need for a read or write control because the unit can only read. Thus when the chip is enabled by the two select inputs, the byte selected by the address lines appears on the data bus.

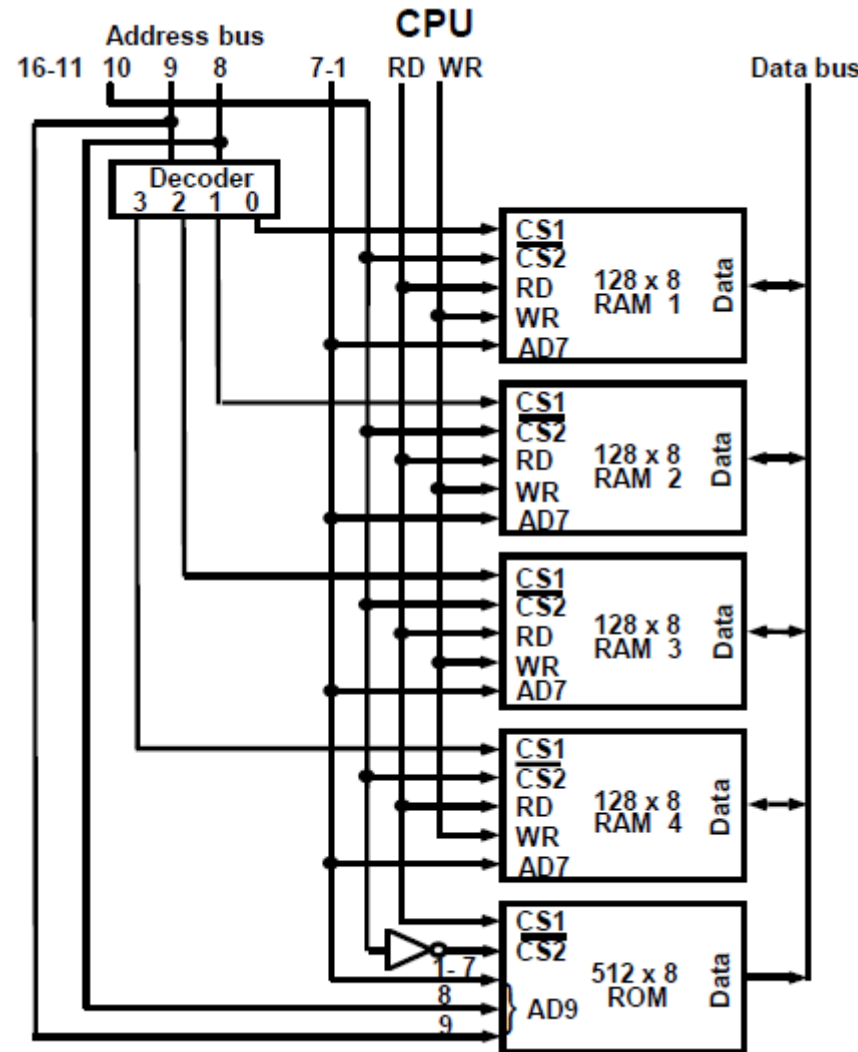
Construction of Larger Size memory

- Mix of RAM and ROM
- Decoder Circuit
- Address Lines
- Data Lines
- SELECT signal



Memory Connection to CPU

Address Lines – specifies the No. Rows
Data Lines – Specifies the No. of Columns



- RAM and ROM chips are connected to a CPU through the data and address buses
- The low-order lines in the address bus select the byte within the chips and other lines in the address bus select a particular chip through its chip select inputs

Memory Design - Memory Interface

Address Map

- **The designer** of a computer system must calculate the amount of memory required for the particular application and assign it to either RAM or ROM.
- **The interconnection** between memory and processor is then established from knowledge of the size of memory needed and the type of RAM and ROM chips available.
- **The addressing** of memory can be established by means of a table that specifies the memory address assigned to each chip.
- **The table**, called a memory address map, is a pictorial representation of assigned address space for each chip in the system.

To demonstrate with a particular example, assume that a computer system needs 512 bytes of RAM and 512 bytes of ROM.

The memory address map for this configuration is shown in Table 1.

TABLE 1 Memory Address Map for Microprocomputer

[illegible]

Memory Design - Memory Interface

Address Map

- **The component column** specifies whether a RAM or a ROM chip is used. The hexadecimal address column assigns a range of hexadecimal equivalent addresses for each chip.
- **The address** bus lines are listed in the third column. Although there are 16 lines in the address bus, the table shows only 10 lines because the other 6 are not used in this example and are assumed to be zero.
- **The small x's** under the address bus lines designate those lines that must be connected to the address inputs in each chip. The RAM chips have 128 bytes and need seven address lines.
- **The ROM chip has 512** bytes and needs 9 address lines. The x's are always assigned to the low-order bus lines: lines 1 through 7 for the RAM and lines 1 through 9 for the ROM.

TABLE 1 Memory Address Map for Microprocomputer

[illegible]

Memory Design - Memory Interface

Address Map

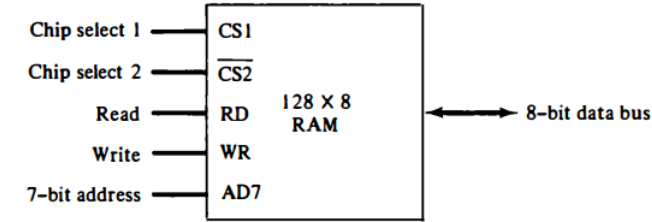
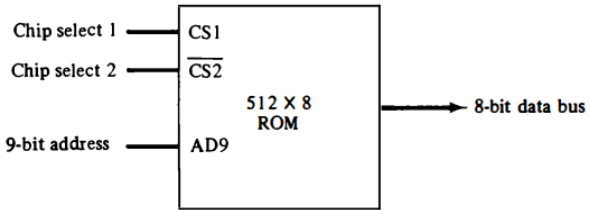
- **It is now necessary** to distinguish between four RAM chips by assigning to each a different address. For this particular example we choose bus lines 8 and 9 to represent four distinct binary combinations.
 - **Note that any other** pair of unused bus lines can be chosen for this purpose. The table clearly shows that the nine low-order bus lines constitute a memory space for RAM equal to $2^9 = 512$ bytes.
 - **The distinction** between a RAM and ROM address is done with another bus line. Here we choose line 10 for this purpose. When line 10 is 0, the CPU selects a RAM, and when this line is equal to 1, it selects the ROM.
 - **The equivalent** hexadecimal address for each chip is obtained from the information under the address bus assignment. The address bus lines are subdivided into groups of four bits each so that each group can be represented with a hexadecimal digit.
 - **The first hexadecimal** digit represents lines 13 to 16 and is always 0. The next hexadecimal digit represents lines 9 to 12, but lines 11 and 12 are always 0.
 - **The range of hexadecimal** addresses for each component is determined from the x's associated with it. These x's represent a binary number that can range from an all-0's to an all-1's value.
- TABLE 1** Memory Address Map for Microcomputer

TABLE 1 Memory Address Map for Microprocomputer

[illegible]

Memory Design - Memory Interface

Address Map



Memory Address Map

Component	Hex Address	Address bus															
		16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
RAM 1	0000–007F	0	0	0	0	0	0	0	0	0	x	x	x	x	x	x	x
RAM 2	0080–00FF	0	0	0	0	0	0	0	0	1	x	x	x	x	x	x	x
RAM 3	0100–017F	0	0	0	0	0	0	0	1	0	x	x	x	x	x	x	x
RAM 4	0180–01FF	0	0	0	0	0	0	0	1	1	x	x	x	x	x	x	x
ROM 1	0200–03FF	0	0	0	0	0	0	1	x	x	x	x	x	x	x	x	x

Address Calculation
(From – To)
-calculated from 16 bit address
- Hexadecimal format

Bus line 10 → select RAM or ROM

Bus lines 8 and 9 are used to select one RAM out of 4 RAMs

No. of Address line bits (x) is given by $N=2^x$
No. of data bus = No. of columns (Word size)

Assumptions

RAM size: 128 * 8 ----- 7 bit address bits needed, No. of Data Lines --- 8

ROM size: 512 * 8 ----- 9 bit address bits required, No. of Data Lines --- 8

Memory Design - Memory Interface

Address Map

Memory Address Map																	
Component	Hex Address	Address bus															
		16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
RAM 1	0000–007F	0	0	0	0	0	0	0	0	0	x	x	x	x	x	x	x
RAM 2	0080–00FF	0	0	0	0	0	0	0	0	1	x	x	x	x	x	x	x
RAM 3	0100–017F	0	0	0	0	0	0	0	1	0	x	x	x	x	x	x	x
RAM 4	0180–01FF	0	0	0	0	0	0	0	1	1	x	x	x	x	x	x	x
ROM 1	0200–03FF	0	0	0	0	0	0	1	x	x	x	x	x	x	x	x	x

Address Calculation
(From Address)

-calculated from 16 bit address

- Hexadecimal format

Substitute x=0 to get 'From' address

If x=0 → 'From' address is

0 0 0 0

0 0 1 0

0 0 0 0

0 0 0 0

Convert to hexadecimal value

0

2

0

0

So, from address for ROM 1 is 0200

Memory Design - Memory Interface

Address Map

Memory Address Map																	
Component	Hex Address	Address bus															
		16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
RAM 1	0000–007F	0	0	0	0	0	0	0	0	0	x	x	x	x	x	x	x
RAM 2	0080–00FF	0	0	0	0	0	0	0	0	1	x	x	x	x	x	x	x
RAM 3	0100–017F	0	0	0	0	0	0	0	1	0	x	x	x	x	x	x	x
RAM 4	0180–01FF	0	0	0	0	0	0	0	1	1	x	x	x	x	x	x	x
ROM 1	0200–03FF	0	0	0	0	0	0	1	x	x	x	x	x	x	x	x	x

Address Calculation
(To Address)

-calculated from 16 bit address

- Hexadecimal format

Substitute x=1 to get 'To' address

If x=1 → 'To' address is

0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1

Convert to hexadecimal value

0

3

F

F

So, To address for ROM 1 is 03FF

Memory Design - Memory Interface Address Map

- Hence,
 - Available Memory chip Size: $N \times W$ (N – represents ROWS – No. of words) (W – represents COLUMNS – word size)
 - Required Memory chip size: $N' \times W'$,
where $N' \geq N$ and $W' \geq W$ (Required > Available)

- Required number of chips = $p \times q$

where $p = N' / N$ (No. of rows reqd.)

and $q = W' / W$ (no. of Cols reqd.)

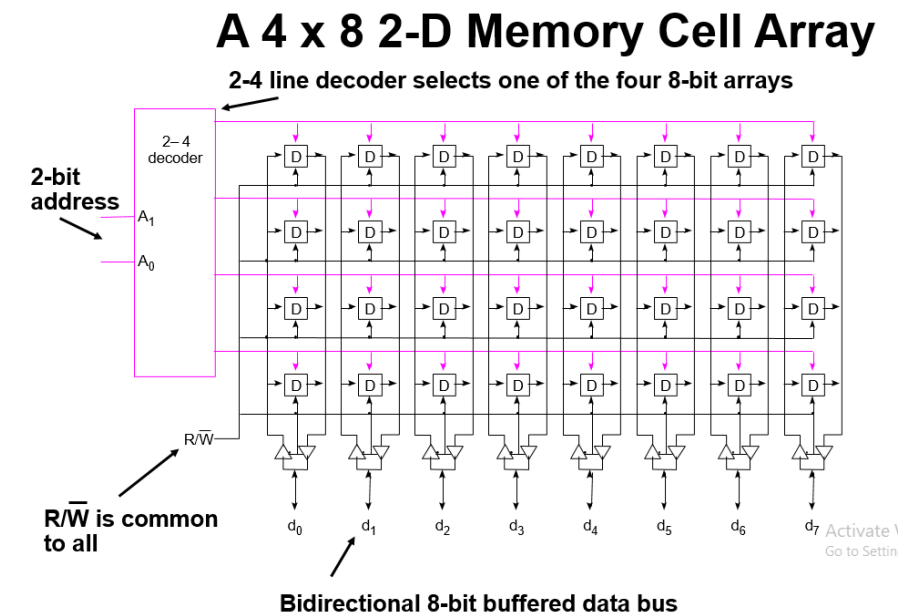
Address bits - required to map to the row

- Decoder used

No. of Address line bits (x) is given by

$$N = 2^x$$

No. of data bus = No. of columns (Word size)



Memory design - Memory Interface

Address Map

There are 3 types of organizations of $N' \times W'$ that can be formed using $N \times W$

- $N' > N$ and $W' = W \Rightarrow$ increasing the number of words (Rows - N) in the memory
- $N' = N$ and $W' > W \Rightarrow$ increasing the word size (Columns - W) of the chip
- $N' > N$ and $W' > W \Rightarrow$ increasing both N & W (Rows & Columns) the number of words and number of bits in each word.

There are different types of organization of $N^1 \times W^1$ –memory using $N \times W$ –bit chips

How many 1024x 8 RAM chips are needed to provide a memory capacity of 2048 x 8?

Case 1: If $N' > N$ & $W' = W$

Increase number of words by the factor of $p = \left\lceil \frac{N'}{N} \right\rceil$

How many 1024x 4 RAM chips are needed to provide a memory capacity of 1024 x 8?

Case 2: If $N' = N$ & $W' > W$

Increase the word size of a Memory by a factor of $q = \left\lceil \frac{W'}{W} \right\rceil$

How many 1024x 4 RAM chips are needed to provide a memory capacity of 2048 x 8?

Case 3: If $N' > N$ & $W' > W$

Increase number of words by the factor of p &
Increase the word size of a Memory by a factor of q

Memory Design - Memory Interface Address Map

Problem – 1 (CASE 1) – Increasing the word size (Columns)

- Design **128 × 16 (N' × W')**- bit RAM using **128 × 4(N × W)** - bit RAM
- Solution: $p = 128 / 128 = 1$;

$q = 16 / 4 = 4$

$p = \left\lceil \frac{N'}{N} \right\rceil \quad q = \left\lceil \frac{W'}{W} \right\rceil$

- Therefore, No. of chips required is calculated by,

$p \times q = 1 \times 4 = 4$ **(i.e. 4 memory chips of size 128 × 4 are required to construct 128 × 16 bit RAM)**

x – Number of bits required to represent the address lines

$128 \times 4 = 2^7 \times 4$ **(by $N=2^x$)**
x=7 bit address is required

Y- Number of bits required for Selecting the specific RAM. **($p = 2^y$)**

1 = 2⁰
y=0 (since only one RAM)

Z – Number of bits to select the RAM or ROM or Interface...

No. of Types (T) = 1
Z=0 ((by $T=2^Z$))

S.No	Memory Type	N × W	N' × W'	p	q	p * q	x	y	z	Total
1	RAM	128 × 4	128 × 16	1	4	4	7	0	0	7

X=7,Y=0,Z=0

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1.1	0000	007F										X	X	X	X	X	X	X
RAM 1.2	0000	007F										X	X	X	X	X	X	X
RAM 1.3	0000	007F										X	X	X	X	X	X	X
RAM 1.4	0000	007F										X	X	X	X	X	X	X

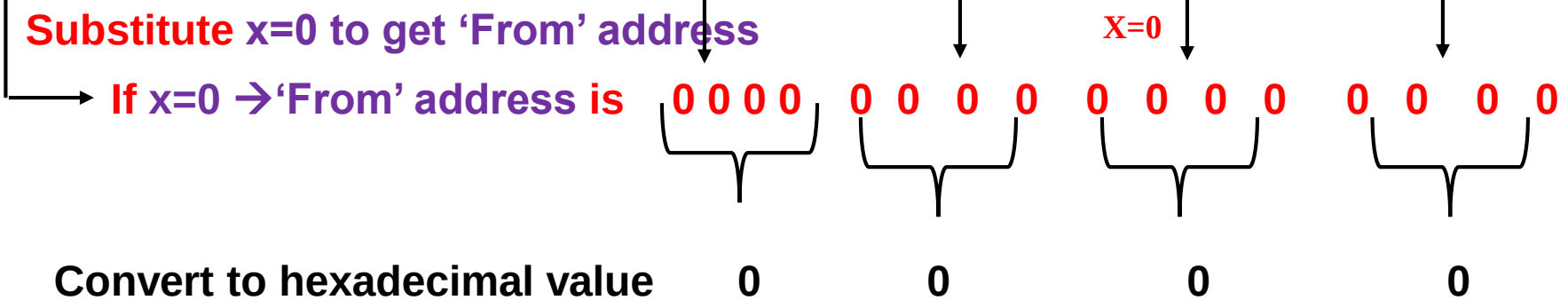
Z – Number of bits to select the RAM or ROM or Interface...
Z=0 → address line 9 is empty

Y- Number of bits required for Selecting the specific RAM
Y=0 → Only one row for the RAM so address lines 7 and 8 are empty

X=7 → 7 bit address lines(from 0 to 6)

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1.1	0000	007F										x	x	x	x	x	x	x
RAM 1.2	0000	007F										x	x	x	x	x	x	x
RAM 1.3	0000	007F										x	x	x	x	x	x	x
RAM 1.4	0000	007F										x	x	x	x	x	x	x



So, from address of RAM is 0000

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1.1	0000	007F										x	x	x	x	x	x	x
RAM 1.2	0000	007F										x	x	x	x	x	x	x
RAM 1.3	0000	007F										x	x	x	x	x	x	x
RAM 1.4	0000	007F										x	x	x	x	x	x	x

Substitute x=1 to get 'to' address

If x=1 → 'to' address is

0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1

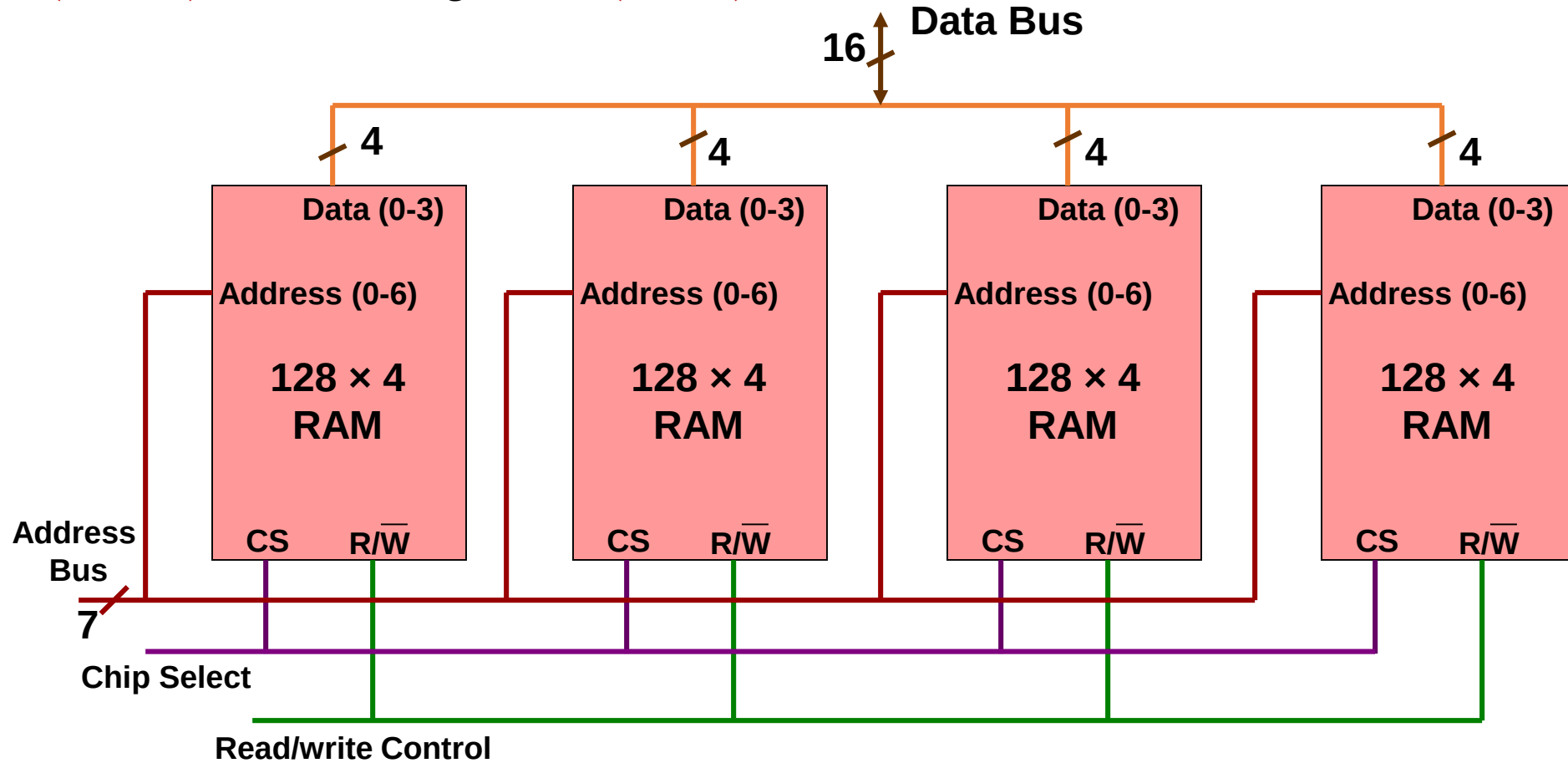
Convert to hexadecimal value 0 0 7 F

So, to address of RAM is 007F

Memory Design - Memory Interface Address Map

Memory design – Increasing the word size

Design 128×16 ($N' \times W'$)- bit RAM using 128×4 ($N \times W$) - bit RAM



Since W is increased ---- the 4 chips should be arranged horizontally.

[illegible]

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1	0000	00FF							0	0	x	x	x	x	x	x	x	x
RAM 2	0100	01FF							0	1	x	x	x	x	x	x	x	x
RAM 3	0200	02FF							1	0	x	x	x	x	x	x	x	x
RAM 4	0300	03FF							1	1	x	x	x	x	x	x	x	x

X=8→ 8 bit address lines(from 0 to 7)

Y=2→ address lines 8 and 9 will select one RAM among 4 RAM

Z=0→ address line 10 is empty

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1	0000	00FF							0	0	x	x	x	x	x	x	x	x
RAM 2	0100	01FF							0	1	x	x	x	x	x	x	x	x
RAM 3	0200	02FF							1	0	x	x	x	x	x	x	x	x
RAM 4	0300	03FF							1	1	x	x	x	x	x	x	x	x

Substitute x=0 to get 'From' address

If x=0 → 'From' address is

0000

0011

0000

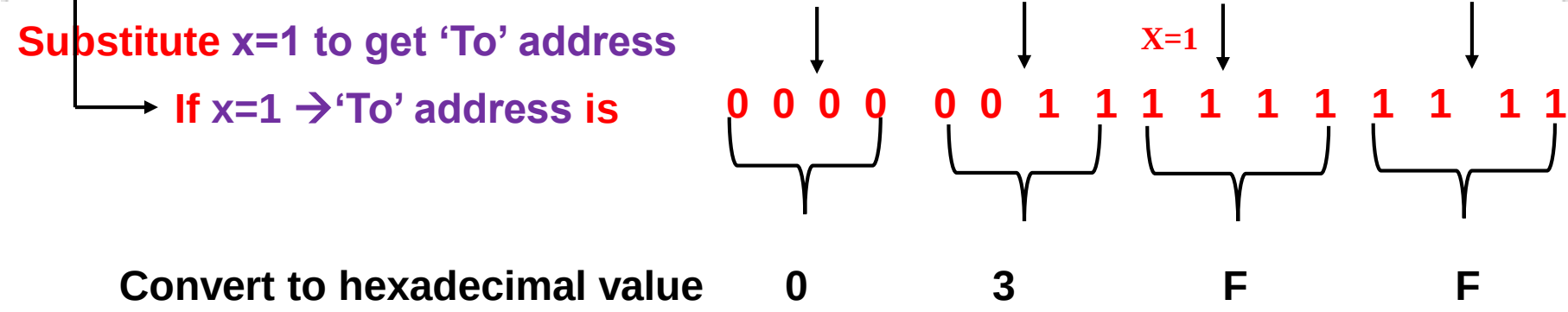
0000

Convert to hexadecimal value 0 3 0 0

So, from address of RAM is 0300

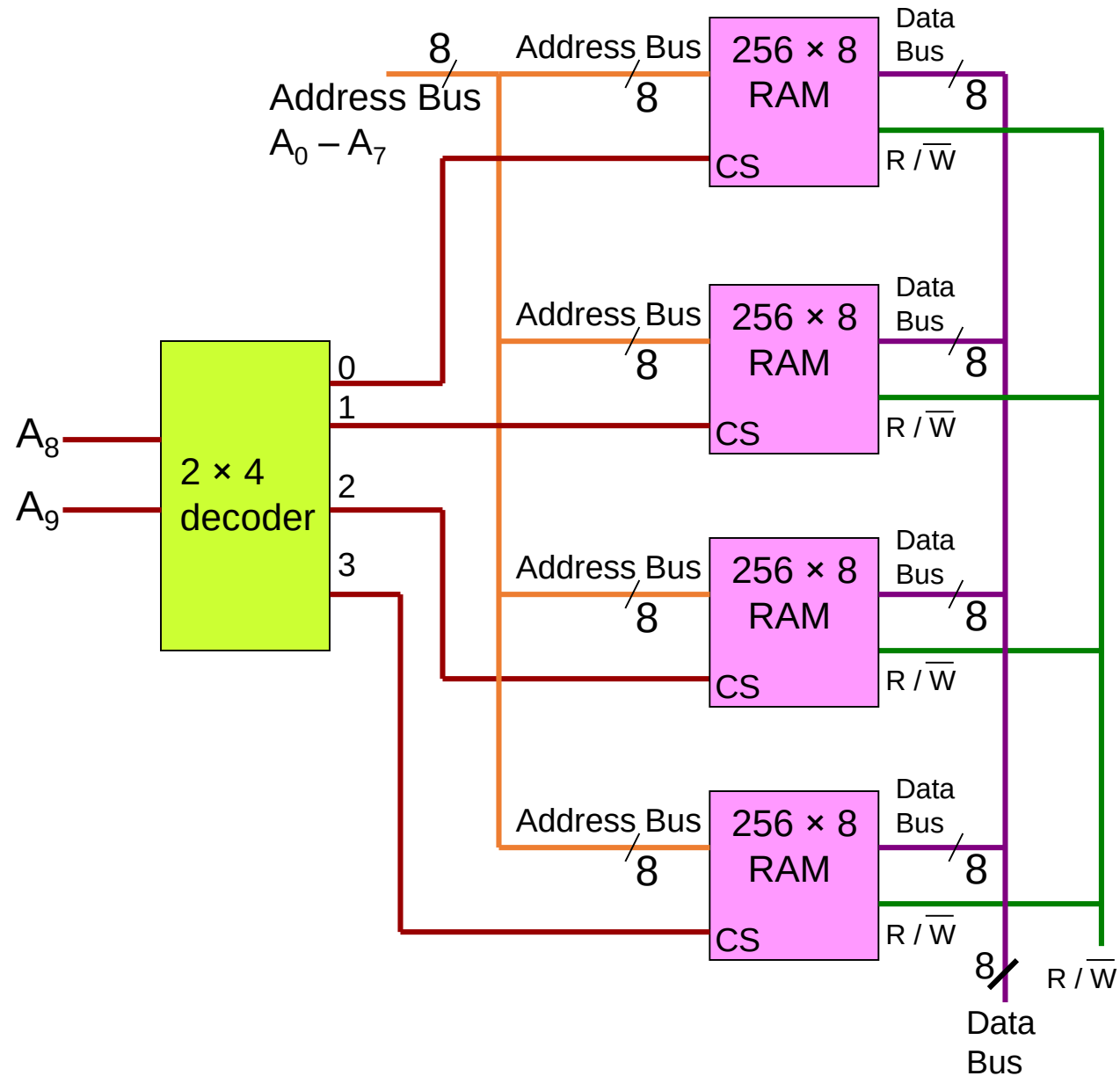
Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1	0000	00FF							0	0	x	x	x	x	x	x	x	x
RAM 2	0100	01FF							0	1	x	x	x	x	x	x	x	x
RAM 3	0200	02FF							1	0	x	x	x	x	x	x	x	x
RAM 4	0300	03FF							1	1	x	x	x	x	x	x	x	x



So, from address of RAM is 03FF

Memory Design - Memory Interface Address Map



Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1.1	0000	007F									0	x	x	x	x	x	x	x
RAM 1.2	0000	007F									0	x	x	x	x	x	x	x
RAM 2.1	0080	00FF									1	x	x	x	x	x	x	x
RAM 2.2	0080	00FF									1	x	x	x	x	x	x	x

Substitute x=0 to get 'From' address

If x=0 → 'From' address is

0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0

Convert to hexadecimal value

0

0

8

0

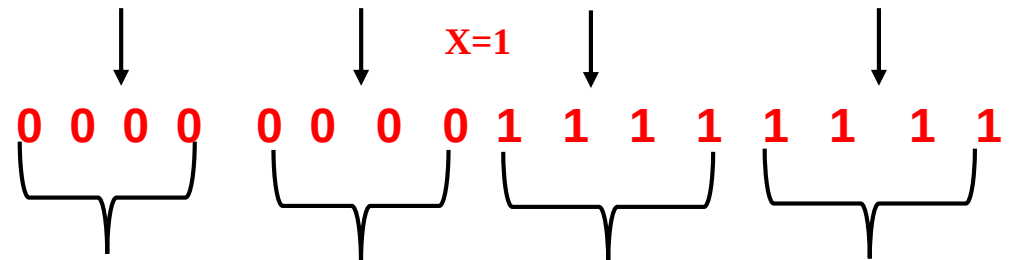
So, from address of RAM is 0080

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1.1	0000	007F									0	x	x	x	x	x	x	x
RAM 1.2	0000	007F									0	x	x	x	x	x	x	x
RAM 2.1	0080	00FF									1	x	x	x	x	x	x	x
RAM 2.2	0080	00FF									1	x	x	x	x	x	x	x

Substitute x=1 to get 'To' address

If x=1 → 'To' address is



Convert to hexadecimal value

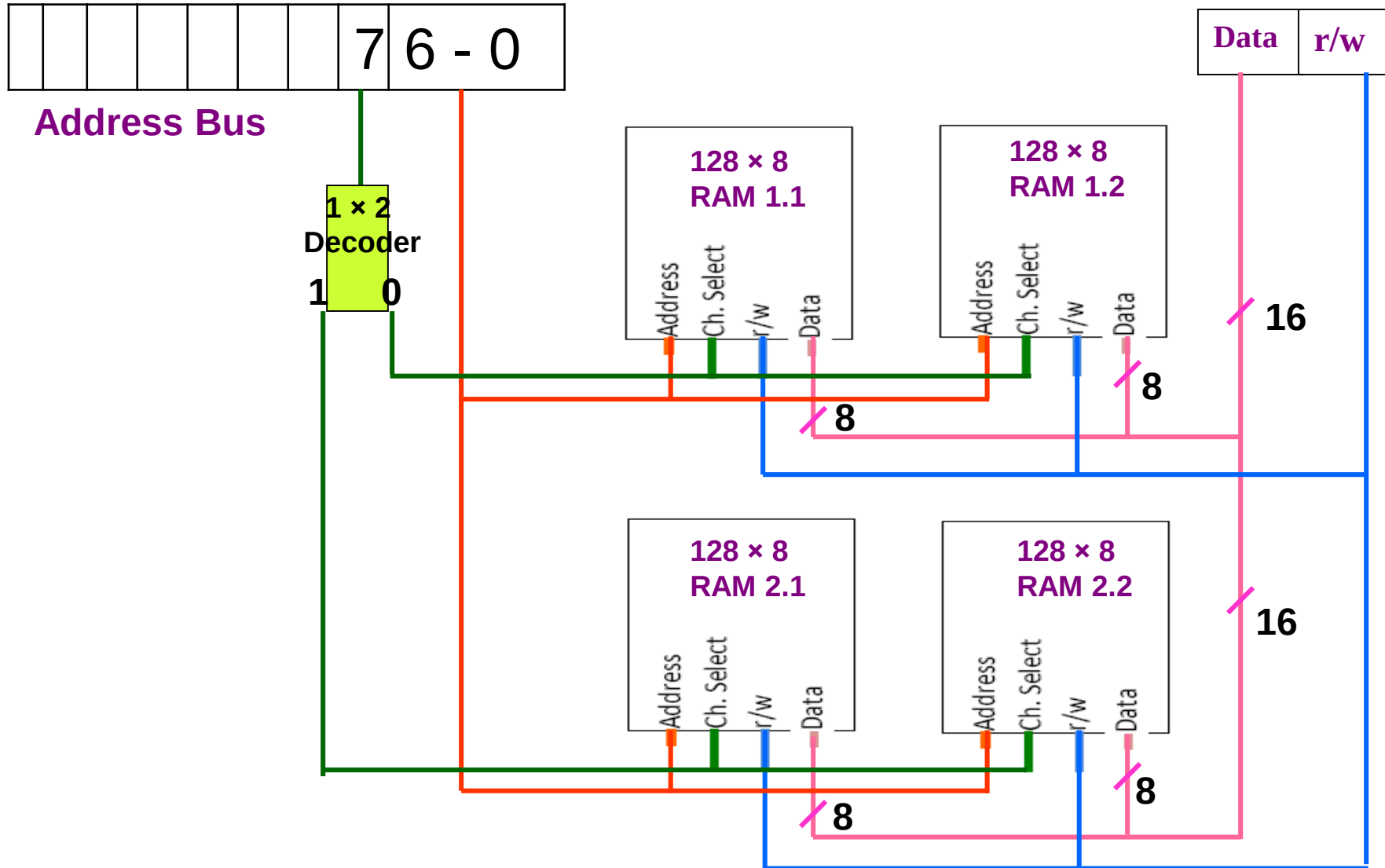
0

0

F

F

So, from address of RAM is 00FF



Memory Design - Memory Interface Address Map

Problem – 4: Design 256×16 – bit RAM using 256×8 – bit RAM chips and 256×8 – bit ROM using 128×8 – bit ROM chips.

RAM Chip calculation

Solution: $p = 256 / 256 = 1$; $q = 16 / 8 = 2$

$p \times q = 1 \times 2 = 2$ RAM $\rightarrow$ 2 chips are required $\rightarrow$ 1 row $\rightarrow$ 2 chips (cols)

x – Number of bits required to represent the address lines
 $256 \times 8 = 2^8 \times 8$
x=8 bit address is required

**Y- Number of bits for
Selecting the specific RAM.
($p = 2^y$) $\rightarrow$ ($p=1 \rightarrow 2^{0 \rightarrow y}$) $y=0$**

Z – Number of 1
(select the RAM or ROM)

[illegible]

Memory Design - Memory Interface Address Map

Problem – 4: Design 256×16 – bit RAM using 256×8 – bit RAM chips and 256×8 – bit ROM using 128×8 – bit ROM chips.

ROM Chip calculation

Solution: $p = 256 / 128 = 2$; $q = 8 / 8 = 1$

$p \times q = 2 \times 1 = 2$ ROM $\rightarrow$ 2 chips are required $\rightarrow$ 2 chips(rows) , 1 column

x – Number of bits required to represent the address lines
 $128 \times 8 = 2^7 \times 8$
x=7 bit address is required

**Y- Number of bits for
Selecting the specific RAM.
($p = 2^y$) $\rightarrow$ ($p=2 \rightarrow 2^{1 \rightarrow y}$) $y=1$**

Z – Number of 1
(select the RAM or ROM)

[illegible]

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1.1	0000	00FF								0	x	x	x	x	x	x	x	x
RAM 1.2	0000	00FF								0	x	x	x	x	x	x	x	x
ROM 1	0100	017F								1	0	x	x	x	x	x	x	x
ROM 2	0180	01FF								1	1	x	x	x	x	x	x	x

Z = 1 bit

For RAM – No bits required – (No address line used)
Y = 1 bit --- For ROM (7 th address line is used)

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1.1	0000	00FF								0	x	x	x	x	x	x	x	x
RAM 1.2	0000	00FF								0	x	x	x	x	x	x	x	x
ROM 1	0100	017F								1	0	x	x	x	x	x	x	x
ROM 2	0180	01FF								1	1	x	x	x	x	x	x	x

Substitute x=0 to get 'From' address
If x=0 → 'From' address is

0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0

Convert to hexadecimal value 0 1 8 0

So, from address of RAM is 0180

Memory Design - Memory Interface Address Map

Component	Hexadecimal address		Address Bus															
	From	To	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RAM 1.1	0000	00FF								0	x	x	x	x	x	x	x	x
RAM 1.2	0000	00FF								0	x	x	x	x	x	x	x	x
ROM 1	0100	017F								1	0	x	x	x	x	x	x	x
ROM 2	0180	01FF								1	1	x	x	x	x	x	x	x

Substitute x=1 to get 'To' address

If x=1 → 'To' address is

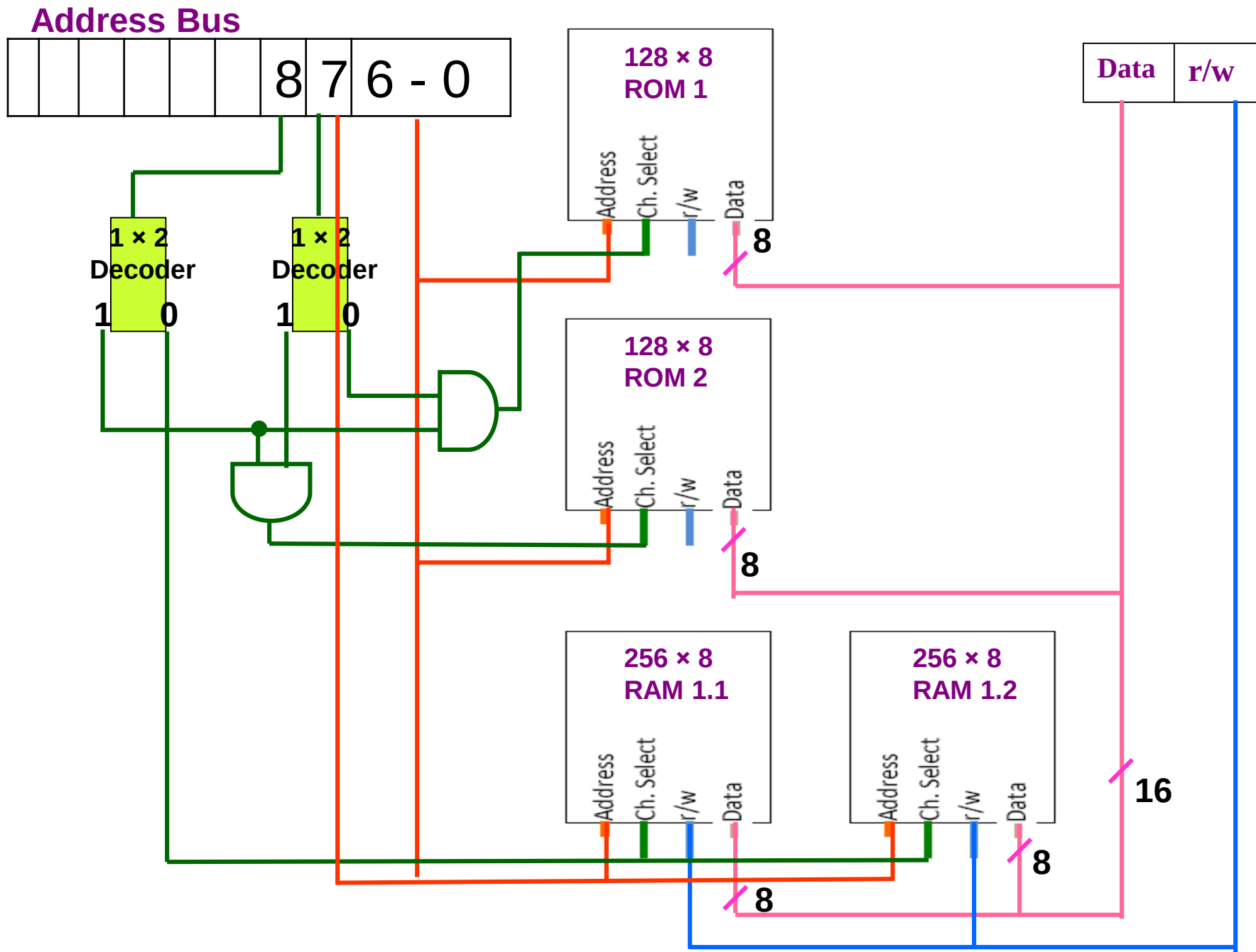
0000

0001111111

01FF

Convert to hexadecimal value

So, from address of RAM is 01FF



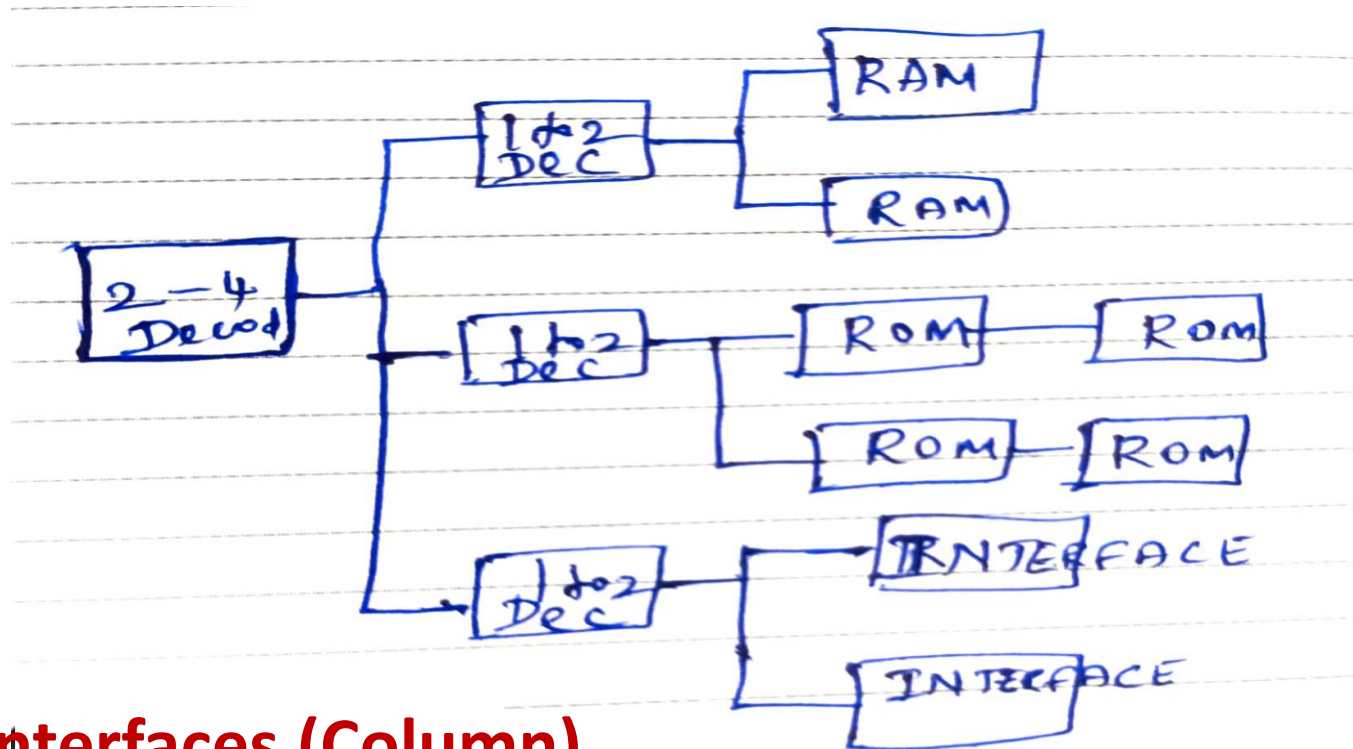
Memory Design - Memory Interface Address Map

Problem – 5

- A computer employs **RAM** chips of 128×8 and **ROM** chips of 512×8 . The computer system needs 256×8 of RAM, 1024×16 of ROM, and two **interface** units with 256 registers each. A memory mapped I/O configuration is used. The two higher -order bits of the address bus are assigned 00 for RAM, 01 for ROM, and 10 for interface registers.
- a. Compute total number of decoders are needed for the above system?
- b. Design a memory-address map for the above system
- c. Show the chip layout for the above design

Memory Design - Memory Interface Address Map

- A computer employs RAM chips of 128 x 8 and ROM chips of 512 x 8. The computer system needs 256 x 8 of RAM, 1024 x 16 of ROM, and two interface units with 256 registers each. A memory mapped I/O configuration is used. The two higher-order bits of the address bus are assigned 00 for RAM, 01 for ROM, and 10 for interface registers.



q = always 1 for Interfaces (Column)

Memory Design - Memory Interface Address Map

Solution: $p = 256 / 128 = 2$; $q = 8 / 8 = 1$

1.RAM

$$p \times q = 2 \times 1 = 2$$

RAM→2 chips are required→ 2chips(row)→ 1(col)

x – Number of bits required to represent the address lines
 $128 \times 8 = 2^7 \times 8$
x=7 bit address is required

**Y- Number of bits for
Selecting the specific RAM.
($p = 2^y$) $\rightarrow$ ($p=2 \rightarrow 2^{1 \rightarrow y}$) $y=1$**

Z – Number of 2
(select the RAM or ROM or interface)

[illegible]

Memory Design - Memory Interface Address Map

Solution: $p = 1024 / 512 = 2$; $q = 16 / 8 = 2$

2.ROM

$p \times q = 2 \times 2 = 4$

RAM→4chips are required→ 2chips(rows), 2chips(cols)

x – Number of bits required to represent the address lines
 $512 \times 8 = 2^9 \times 8$
x=9 bit address is required

**Y- Number of bits for
Selecting the specific RAM.
($p = 2^y$) $\rightarrow$ ($p=2 \rightarrow 2^{1 \rightarrow y}$) $y=1$**

Z – Number of 2
(select the RAM or ROM or interface)

[illegible]

Memory Design - Memory Interface Address Map

3.Interface

p = number of interfaces=2(input is given)
q is 1 always for interfaces.

$p * q = 2 * 1 = 2$

$2^x = 256$ (Number of registers)
 $x = 8$

$$2^y = p=2$$
$$y= 1$$

Z – Number of 2 (select the RAM or ROM or interface)

2 interfaces are required → 2 interfaces(rows), 1 column

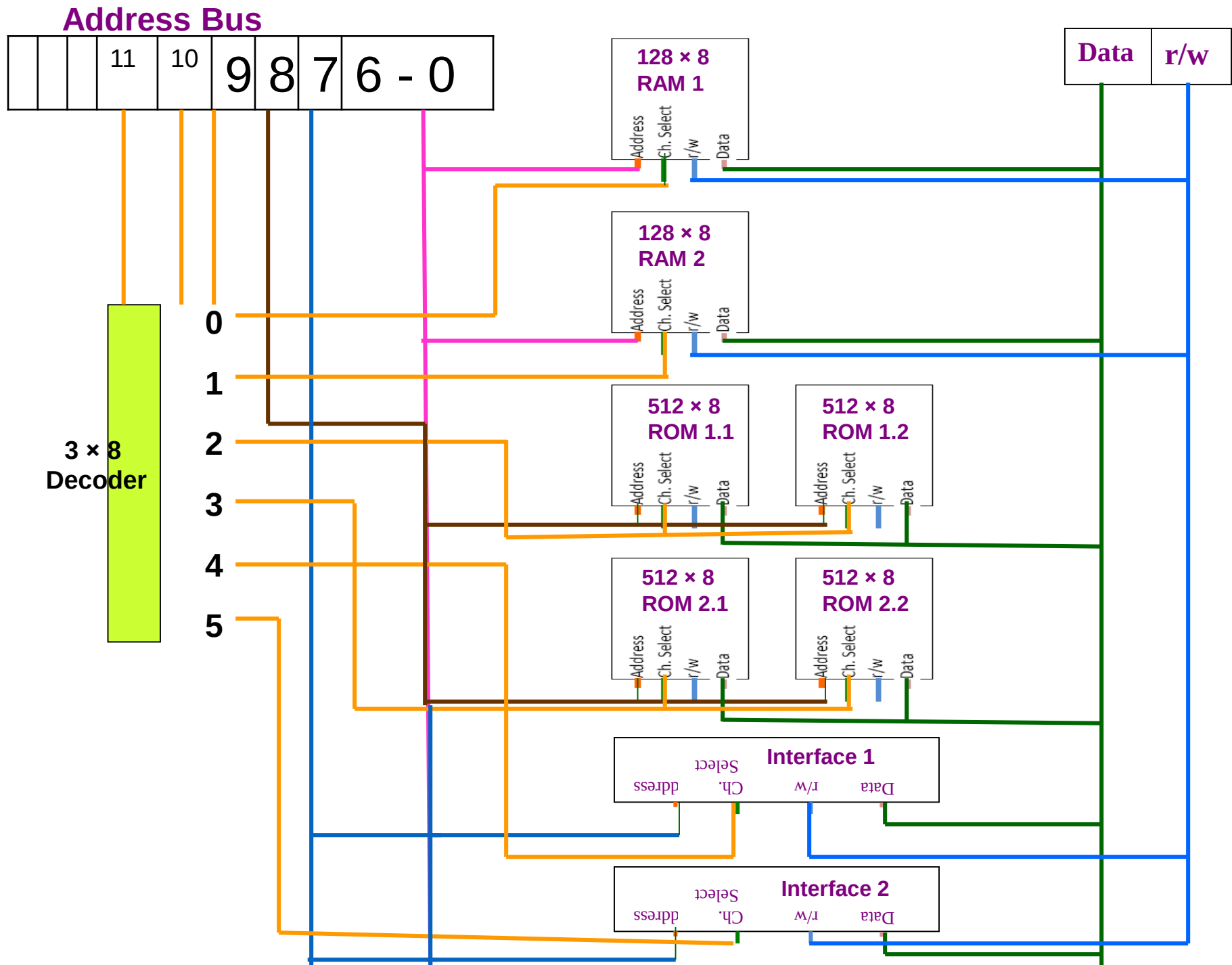
[illegible]

Memory Design - Memory Interface Address Map

Component	Hexadecimal Address		Address Bus												
	From	To	15 - 12	11	10	9	8	7	6	5	4	3	2	1	0
RAM1	0000	007F		0	0	0			x	x	x	x	x	x	x
RAM2	0200	027F		0	0	1			x	x	x	x	x	x	x
ROM1.1	0400	05FF		0	1	0	x	x	x	x	x	x	x	x	x
ROM1.2	0400	05FF		0	1	0	x	x	x	x	x	x	x	x	x
ROM2.1	0600	07FF		0	1	1	x	x	x	x	x	x	x	x	x
ROM2.2	0600	07FF		0	1	1	x	x	x	x	x	x	x	x	x
Interface1	0800	08FF		1	0	0		x	x	x	x	x	x	x	x
Interface2	0A00	0AFF		1	0	1		x	x	x	x	x	x	x	x

X=address lines(RAM→7,ROM→9,interface→8)

Y=1 Z=2



Memory Design - Memory Interface Address Map

Problem – 6

- Suppose that a $2\text{M} \times 16$ RAM memory is built using $256\text{K} \times 8$ RAM chips and $1\text{K} \times 8$ ROM memory is built using 256×8 ROM chips with the word addressable memory, find the following:
 - a) How many RAM chips and ROM chips are necessary?
 - b) If we were accessing one full word in RAM, how many chips would be involved?
 - c) How many address bits are needed for each RAM chip?
 - d) How many memory banks are required? *Hint: number of banks are addressable units in main memory and RAM chips*
 - e) If high-order interleaving is used, where would address $(1\text{B})_{16}$ be located? Also, for a low-order interleaving?

Memory Design - Memory Interface Address Map

Problem – 6 - Solution

- a) 16 RAMS , 4 ROMS
- b) Each RAM chip is $256K \times 8$, so to access one full word (2 bytes), 2 RAM chips would be involved.
- c) The $256K \times 8$ RAM chip has 18 address bits ($2^{18} = 256K$), so each RAM chip would need 18 address bits.
- d) 8
- e) module 0 – word 27
module 3 - word 3

Cache Memory

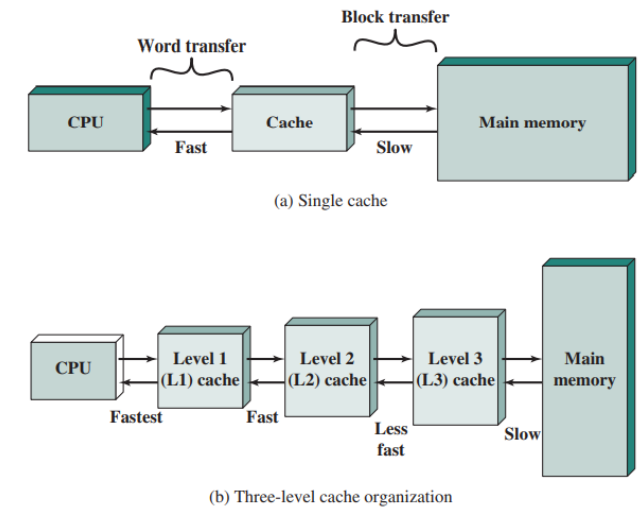
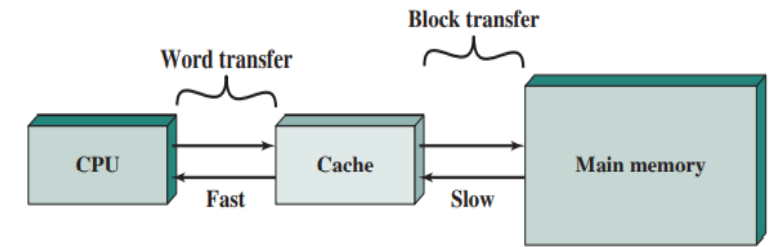


Figure 4.3 Cache and Main Memory

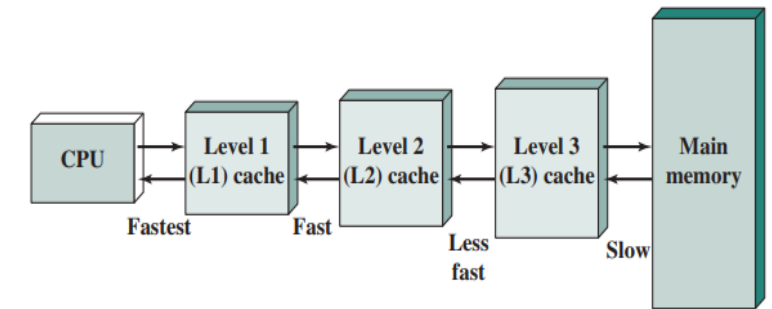
- Special **very high-speed memory**
- used to **speed up and synchronize with high-speed CPU**
- Cache memory is **costlier** than main memory or disk memory but more economical than CPU registers
- extremely fast memory type that **acts as a buffer** between RAM and the CPU
- It holds **frequently requested data and instructions** so that they are immediately available to the CPU when needed
- used to **reduce the average time to access data** from the Main memory
- The cache is a smaller and faster memory that **stores copies of the data** from frequently used main memory locations

Cache Memory: Principles

- The intent of cache memory is to provide the **fastest access** to resources without **compromising on size and price** of the memory.
- The processor attempting to read a byte of data, **first looks at the cache memory**.
- If the byte **does not exist in cache memory**, it searches for the byte in the **main memory**.



(a) Single cache



(b) Three-level cache organization

Figure 4.3 Cache and Main Memory

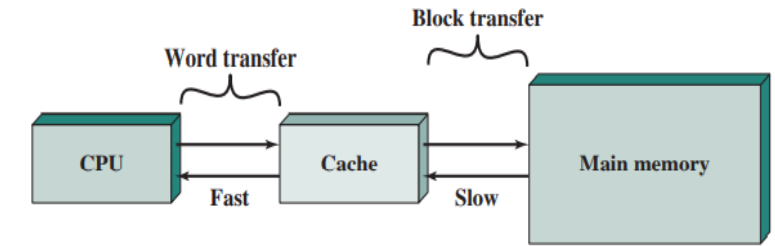
Types of Cache Memory

- **Primary Cache:**

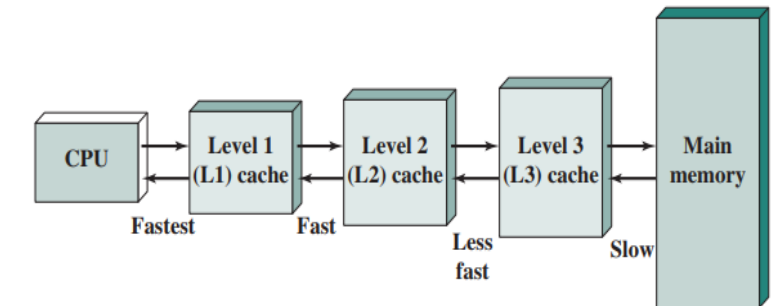
- very fast and its access time is similar to the processor registers
- it is built onto the processor chip
- its size is quite small
- also known as a level 1 cache and is built using static RAM (SRAM)

- **Secondary Cache:**

- The secondary cache or external cache is cache memory that is external to the primary cache
- It is located between the primary cache and the main memory
- It is also known as a level 2 cache



(a) Single cache

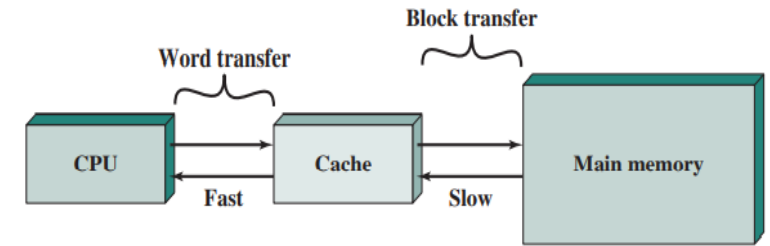


(b) Three-level cache organization

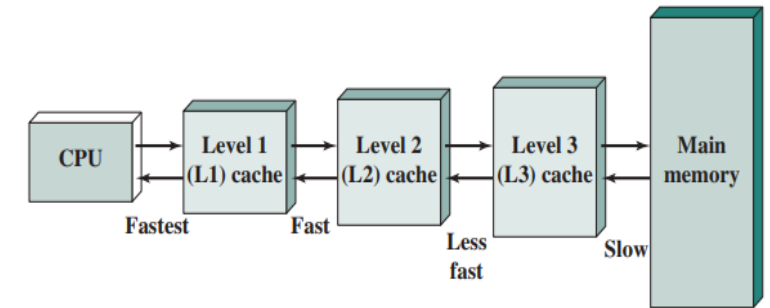
Figure 4.3 Cache and Main Memory

Cache Memory: Advantages & Disadvantages

- Advantages of Cache Memory
 - Faster
 - Less access time
- Disadvantages of Cache Memory
 - Expensive
 - Limited capacity



(a) Single cache



(b) Three-level cache organization

Figure 4.3 Cache and Main Memory

Locality of Reference

- An *implication of locality* is that we can predict with reasonable accuracy what instructions and data a program will use in the near future based on its accesses in the recent past.

- ▼ **Temporal Locality (Locality in Time):**

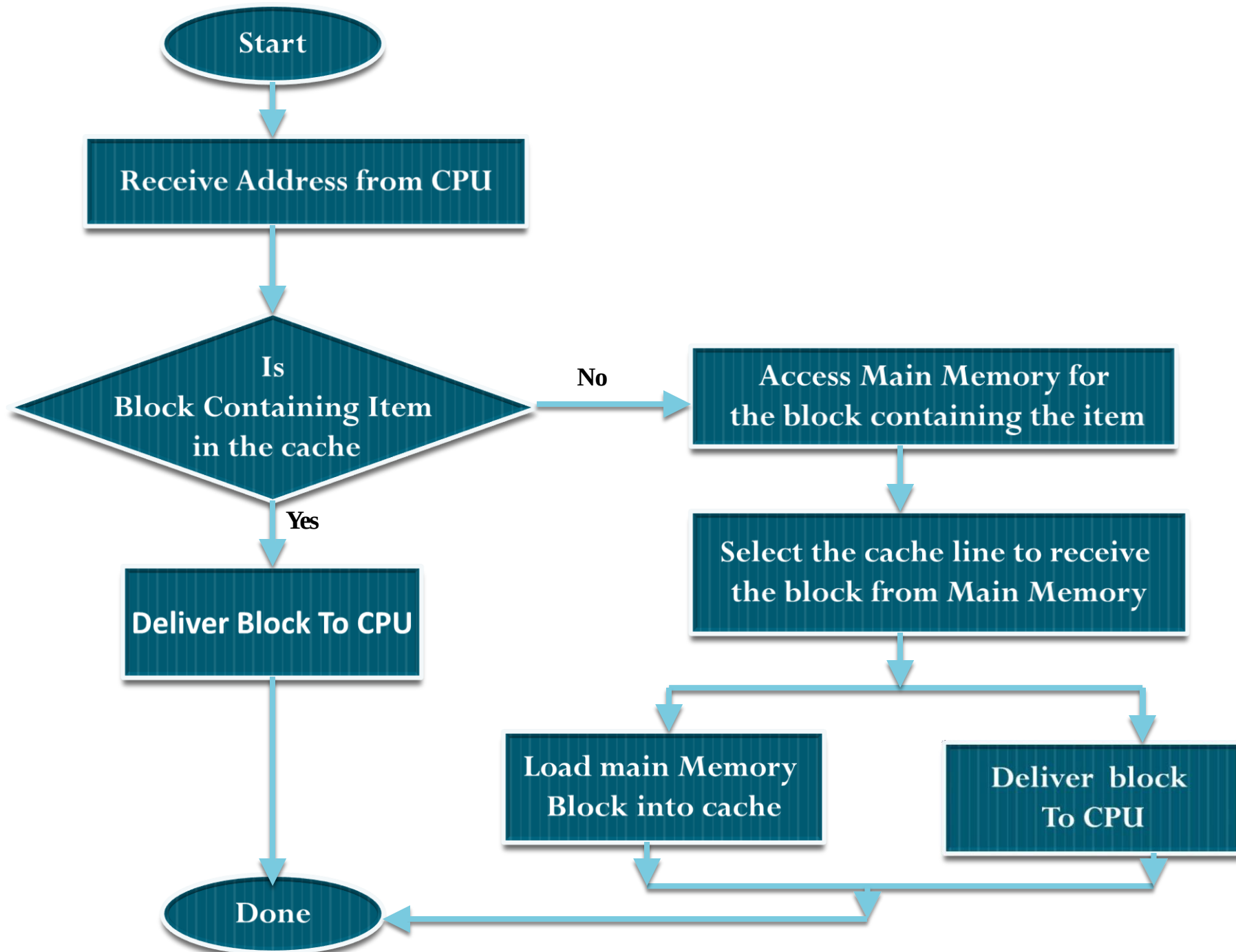
- ◆ If an item is referenced, it will tend to be referenced again soon
- ◆ Example: code and variables in loops

📌 Keep recently accessed data closer to the processor

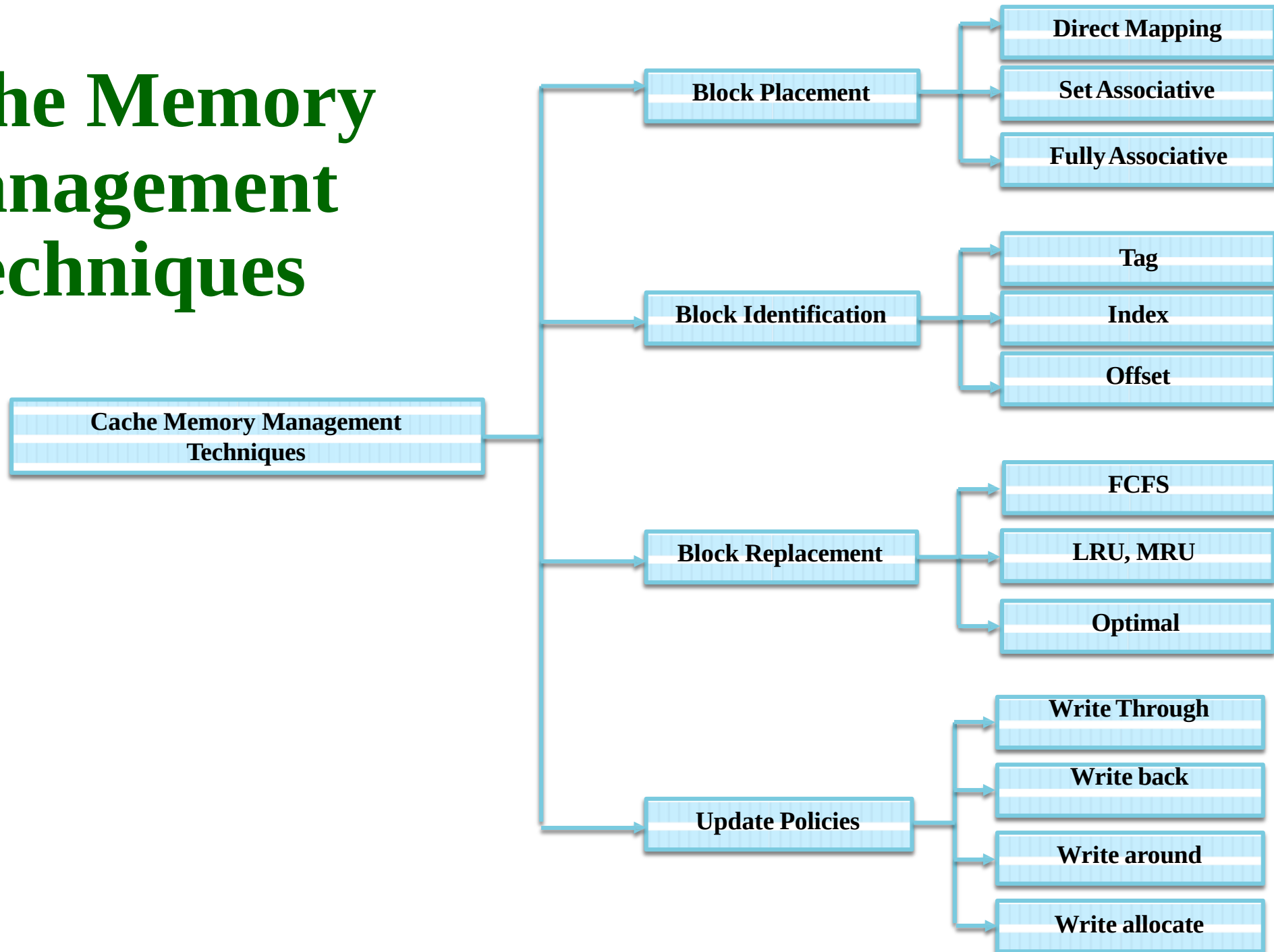
- ▼ **Spatial Locality (Locality in Space):**

- ◆ If an item is referenced, nearby items tend to be referenced soon
- ◆ Example: scanning an array

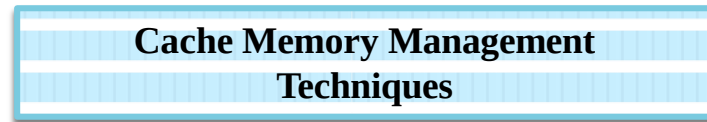
📌 Move contiguous blocks closer to the processor



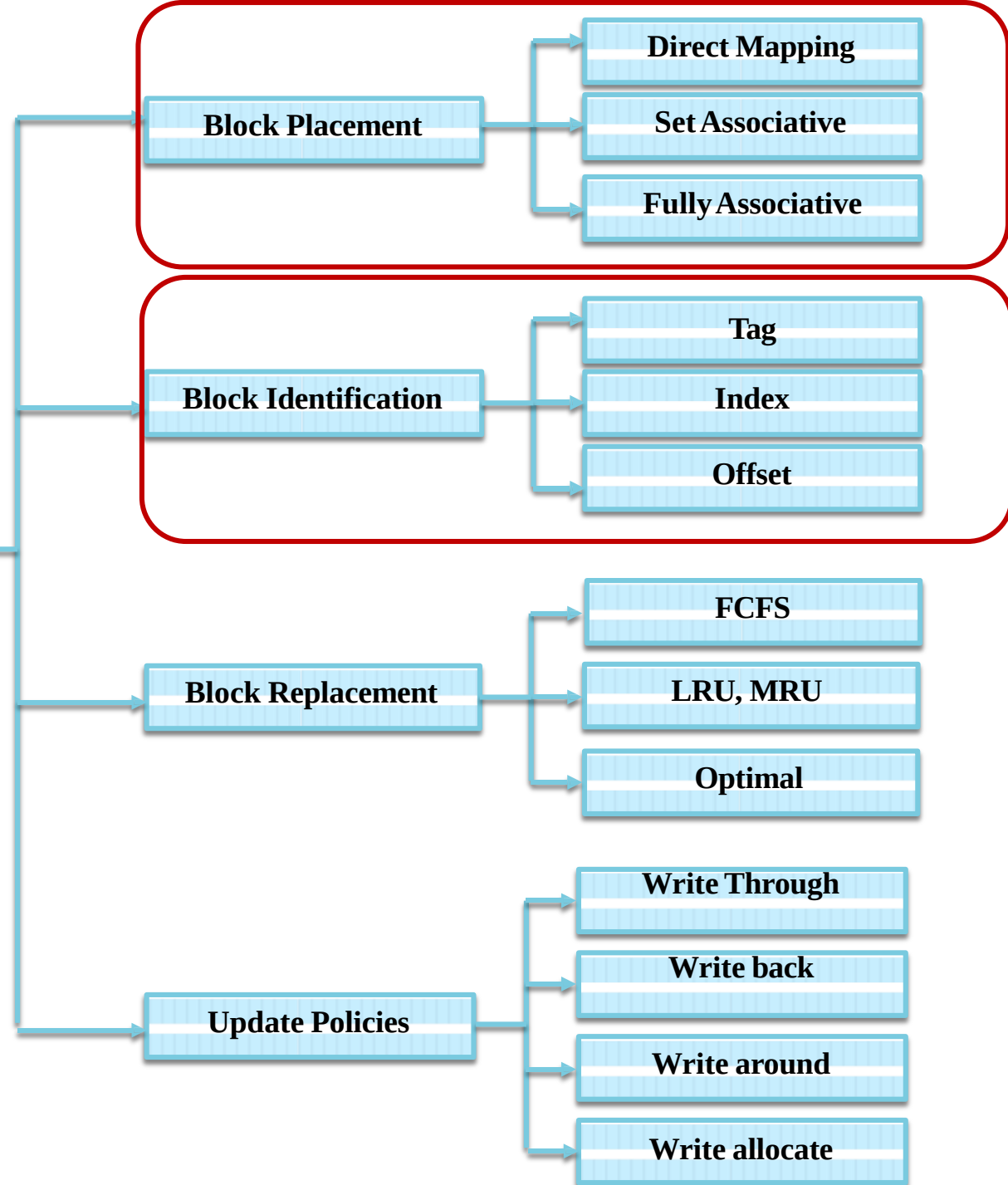
Cache Memory Management Techniques



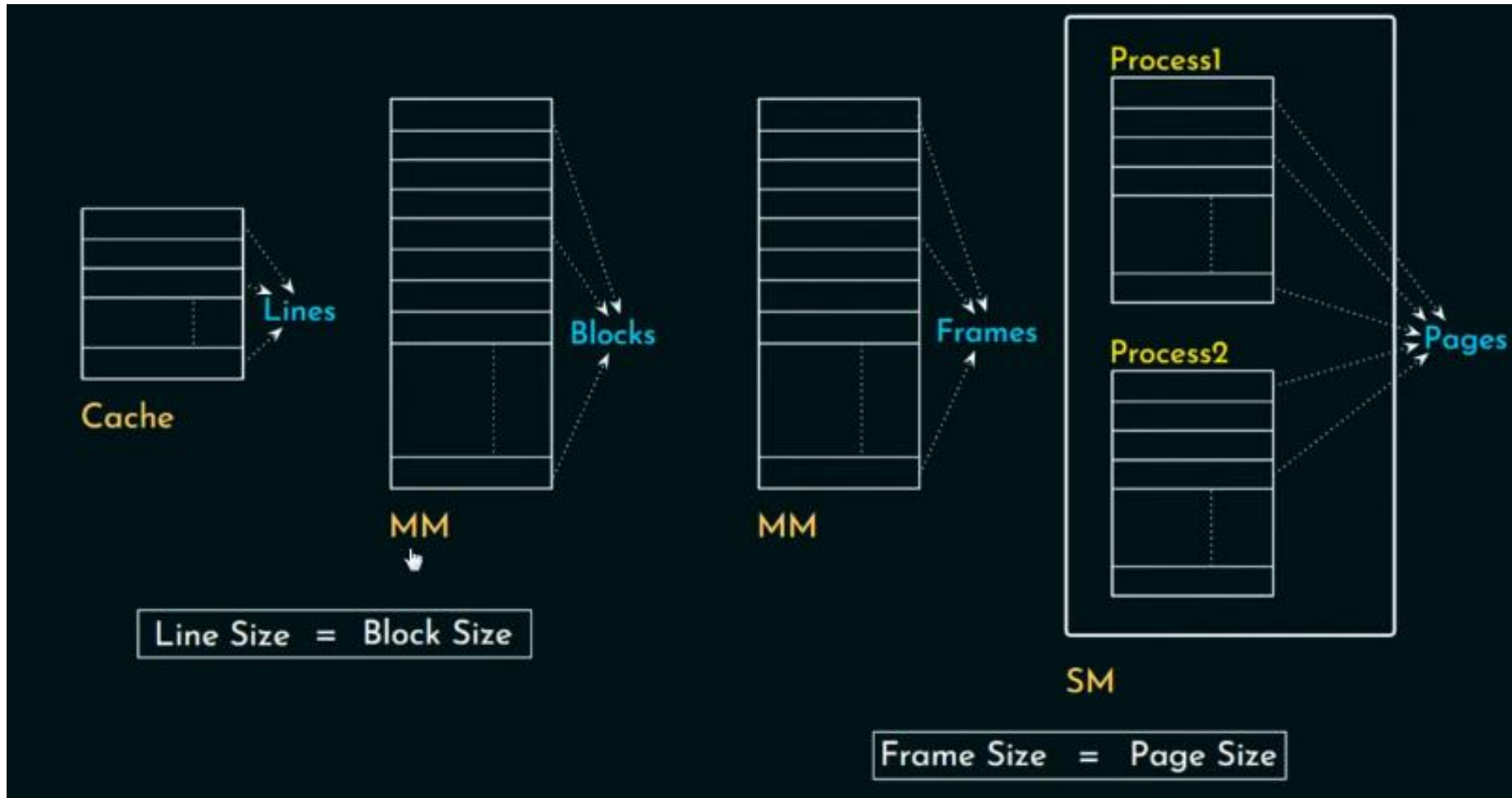
Cache Memory Management Techniques



Block Placement Strategies – Mapping Techniques



Mapping - Basics



In **SM to MM** – The terms **Pages** and **Frames** are used.

In **MM to CM** – The terms **Blocks** and **Lines** are used.

Pages, Frames, Blocks, Lines
– Same

Blocks – MM Blocks
Cache Lines - Cache Blocks

Mapping - Basics

Mapping 64 words of Main memory on to 16 words of Cache

- Main Memory Address

Word: Smallest Addressable Unit of Memory.

- Byte-Addressable Memory: 1 word = 1 Byte

Main Memory Size : 64 words

Block Size : 4 words

No. of Blocks in MM : $64 / 4 = 16$ i.e. 0, 1, 2, 3, ..., 15

0				
1				
2				
3				
4				
15				

16 Blocks

4 words per Block

0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15
4	16	17	18	19
5	20	21	22	23
6	24	25	26	27
7	28	29	30	31
8	32	33	34	35
9	36	37	38	39
10	40	41	42	43
11	44	45	46	47
12	48	49	50	51
13	52	53	54	55
14	56	57	58	59
15	60	61	62	63

Mapping - Basics

Mapping 64 words of Main memory on to 16 words of Cache

- Main Memory Address

Each word (0 to 63) – has data – (needs to be transferred)

Hence, Main Memory Address(Physical Address)

– should point to the individual word in a block.

No. of bits needed to store 64 words

$$\begin{aligned} &64 \text{ words} \\ &\log_2 64 = \log_2 2^6 \\ &= 6 \text{ bits} \end{aligned}$$

16 Blocks

Main Memory Address bits (Physical Address Bits – PA Bits): 6 bits



4 words per Block

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15
4	16	17	18	19
5	20	21	22	23
6	24	25	26	27
7	28	29	30	31
8	32	33	34	35
9	36	37	38	39
10	40	41	42	43
11	44	45	46	47
12	48	49	50	51
13	52	53	54	55
14	56	57	58	59
15	60	61	62	63

Mapping - Basics

Mapping 64 words of Main memory on to 16 words of Cache

- Main Memory Address

No. of bits needed to store 64 words = 6 bits

No. of words in the memory: 64 words

$$\begin{aligned} 64 \text{ words} \\ \log_2 64 &= \log_2 2^6 \\ &= 6 \text{ bits} \end{aligned}$$

No. of bits needed to identify the block (total 16 blocks)

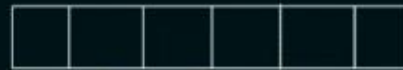
$$\begin{aligned} 16 \text{ Blocks} \\ \log_2 16 &= \log_2 2^4 \\ &= 4 \text{ bits} \end{aligned}$$

0	1	2	3	0
4	5	6	7	1
8	9	10	11	2
12	13	14	15	3
16	17	18	19	4
20	21	22	23	5
24	25	26	27	6
28	29	30	31	7
32	33	34	35	8
36	37	38	39	9
40	41	42	43	10
44	45	46	47	11
48	49	50	51	12
52	53	54	55	13
56	57	58	59	14
60	61	62	63	15

Physical Address Space

$$\log_2 64 = \log_2 2^6 = 6 \text{ bits}$$

P. A. bits :
(Physical Address bits)



0	0	→	0 th
0	1	→	1 st
1	0	→	2 nd
1	1	→	3 rd

16 Blocks

$$\begin{aligned} \log_2 16 &= \log_2 2^4 \\ &= 4 \text{ bits} \end{aligned}$$



No. of bits to identify a block
in 16 blocks = 4 bits

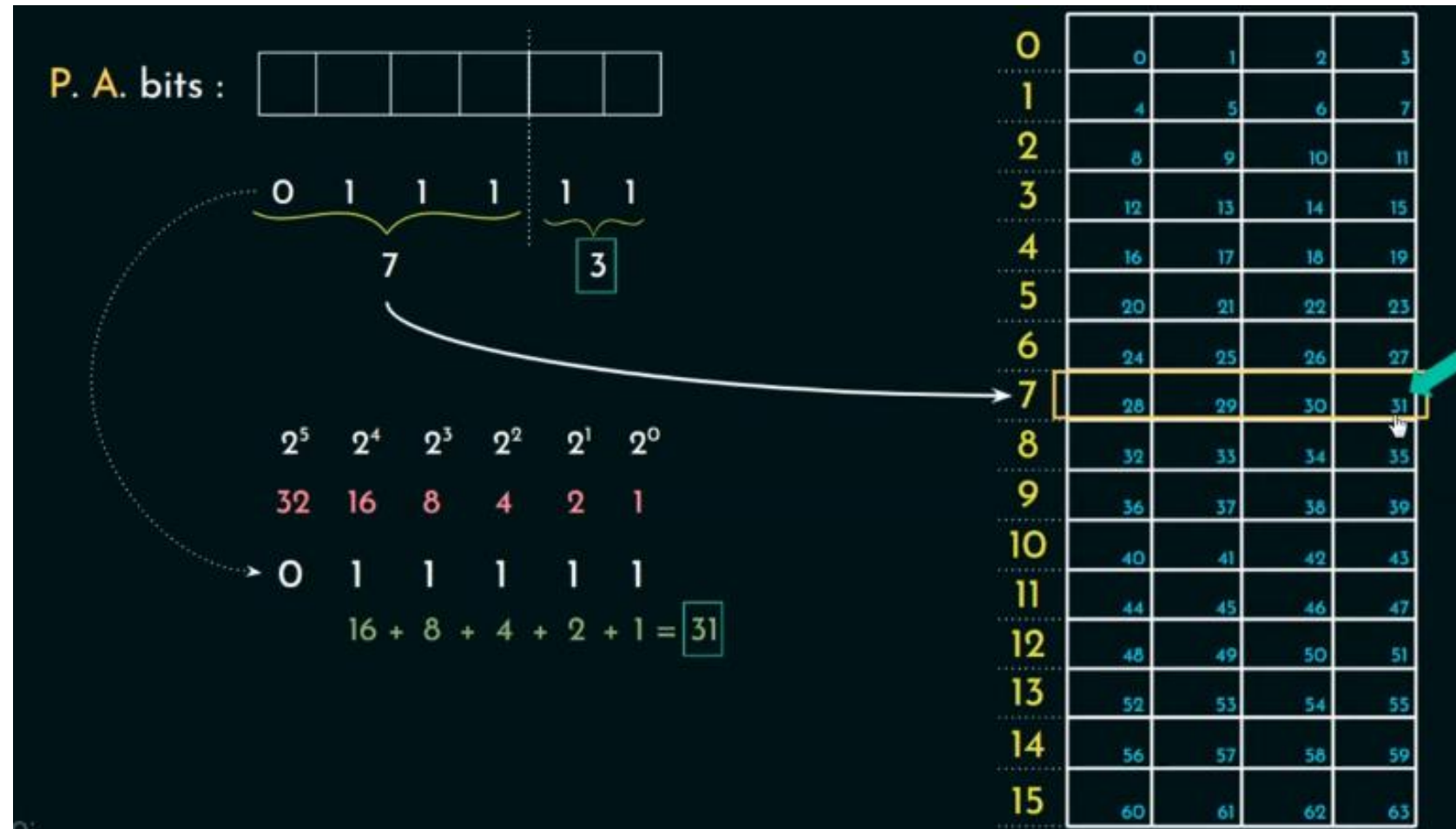
No. of bits to identify a single word
in the identified block = 2 bits

Mapping - Basics

Mapping 64 words of Main memory on to 16 words of Cache

- Main Memory Address

If the PA is 011111,
Then,
the corresponding block and
the word in that block can be
identified with the first 4 bits and
last 2 bits respectively.



Mapping - Basics

Mapping 64 words of Main memory on to 16 words of Cache

- Cache Memory Organization

Cache Size : 16 words

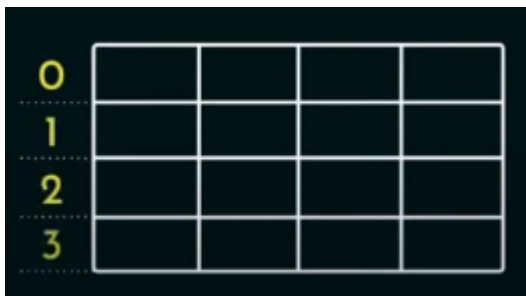
Line Size : 4 words

In Main memory,

Block Size : 4 words

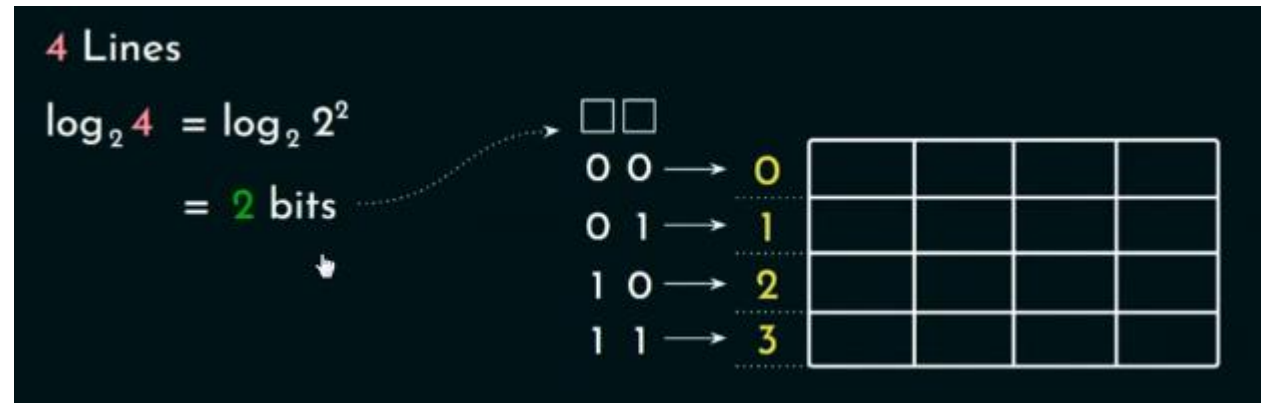
Block Size = Line Size

No. of Lines in Cache : $16 / 4 = 4$ i.e. 0, 1, 2, 3



No. of Cache Blocks (Lines) = 4

No. of bits needed to store 4 lines (Cache Blocks) = 2 bits



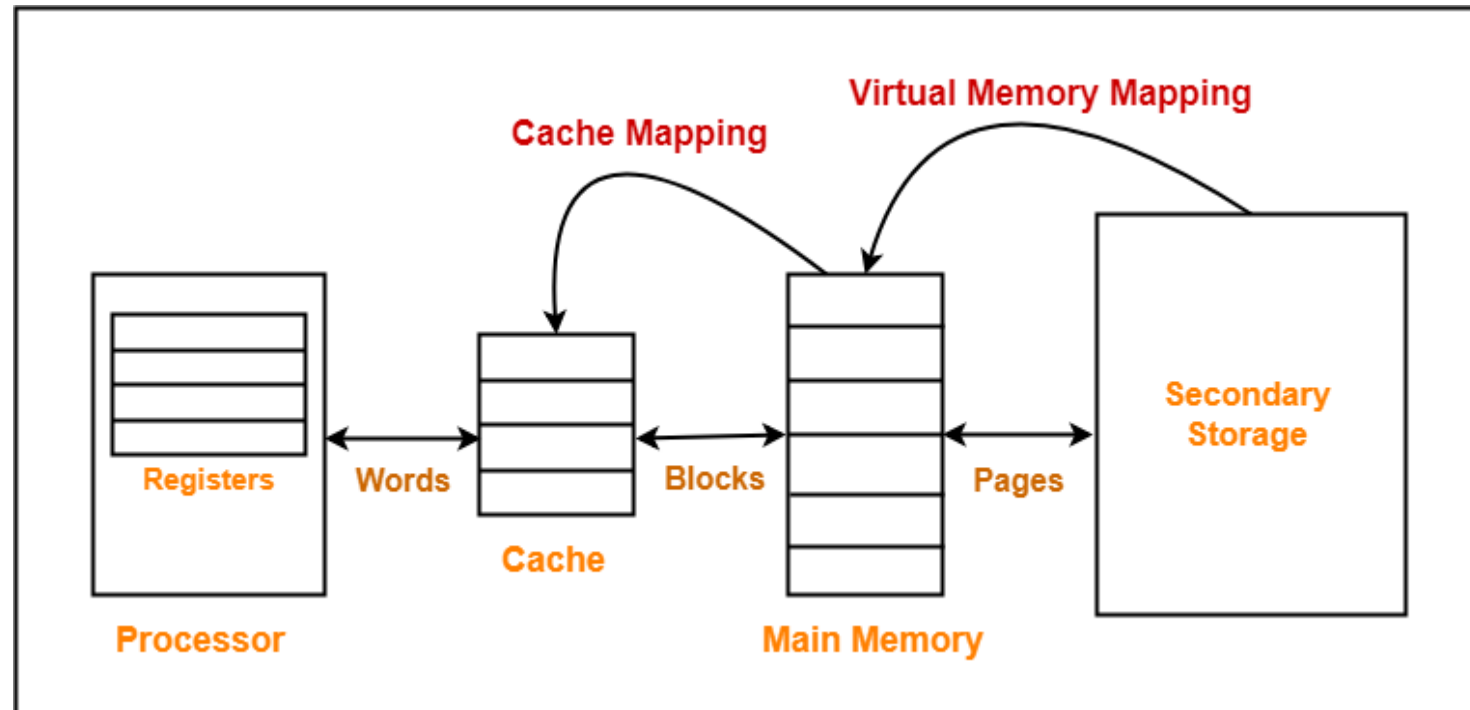
MM Size – 16 blocks

Cache Size – 4 blocks(Lines)

Hence, Mapping is Required

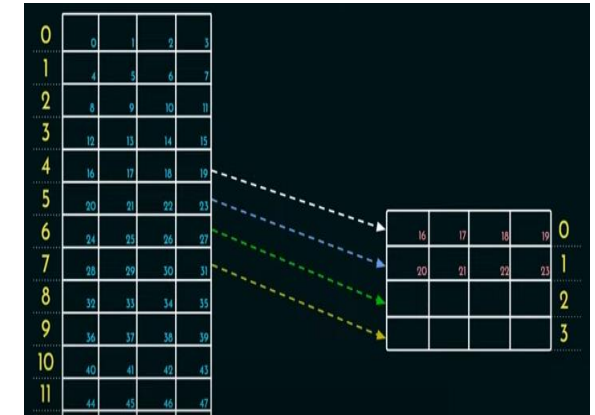
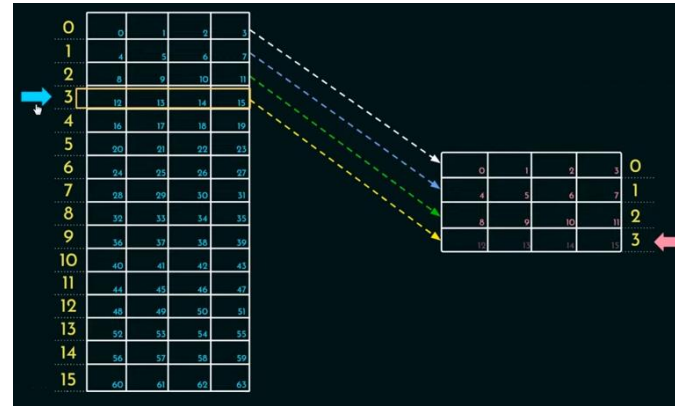
Cache - Memory Mapping Techniques

- Cache mapping defines how a block from the main memory is mapped to the cache memory in case of a cache miss
- Cache mapping is a technique by which the contents of main memory are brought into the cache memory.
- Direct mapping
- Associative mapping
- Set-Associative mapping

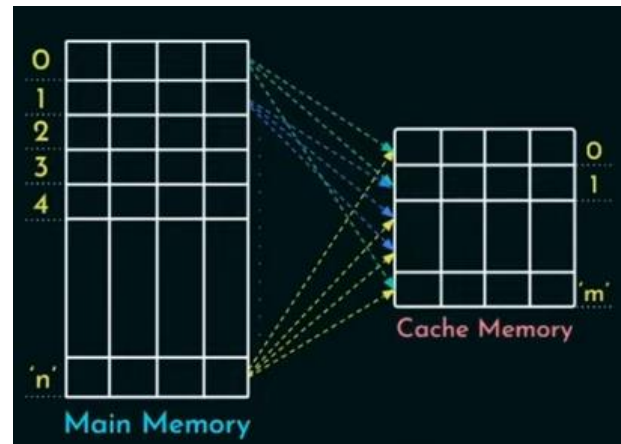


Cache - Memory Mapping Techniques

- Direct Mapping
 - Many to One Mapping
 - Map to a fixed Cache Line



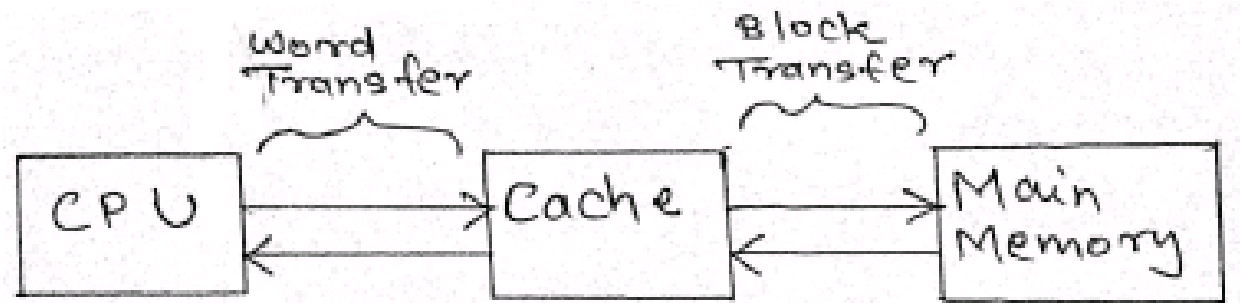
- Associative Mapping
 - Many to Many Mapping
 - Map to any Cache Line
- Set Associative Mapping



Mapping Techniques - Direct Mapping

- A particular block of main memory can map only to a particular line of the cache
- Assign each memory block to a specific line in the cache
- If a line is previously taken up by a memory block when a new block needs to be loaded, the old block is trashed

Cache line number = (MM Block Address) % (Number of CM lines)



Mapping Techniques - Direct Mapping

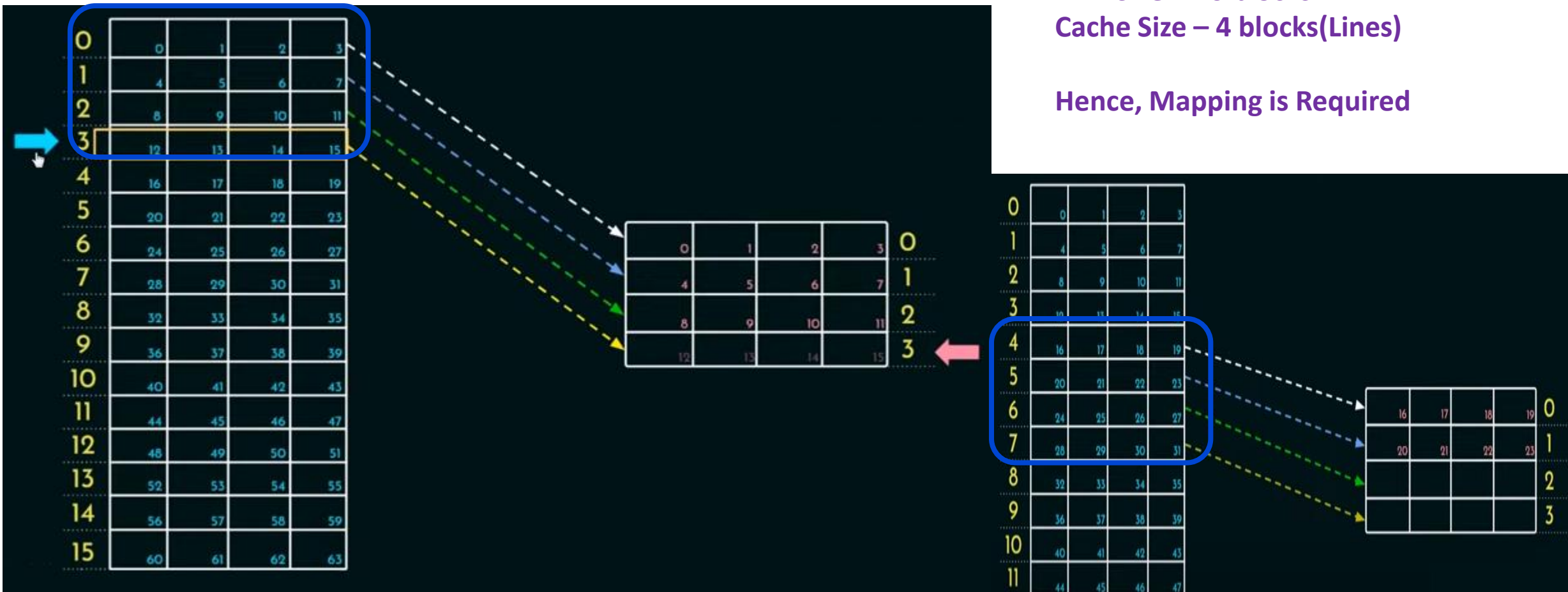
Mapping 64 words of Main memory on to 16 words of Cache

- Mapping – Direct Mapping

MM Size – 16 blocks

Cache Size – 4 blocks(Lines)

Hence, Mapping is Required



Mapping Techniques - Direct Mapping

Cache line number = (MM Block Address) % (Number of CM lines)

- Each address maps to one block
- Use modulo addressing
 - $0 \rightarrow (0 \bmod 8) = 0$
 - $1 \rightarrow (1 \bmod 8) = 1$
 - $2 \rightarrow (2 \bmod 8) = 2$
 - $8 \rightarrow (8 \bmod 8) = 0$
 - $9 \rightarrow (9 \bmod 8) = 1$
 - $18 \rightarrow (18 \bmod 8) = 2$



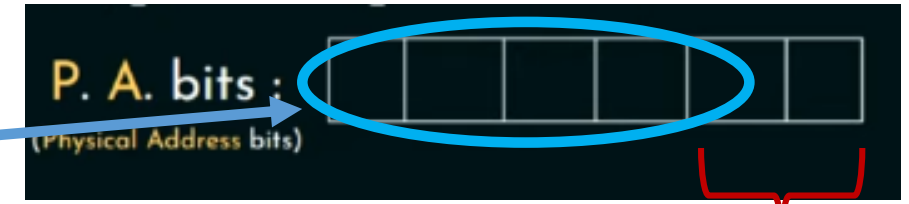
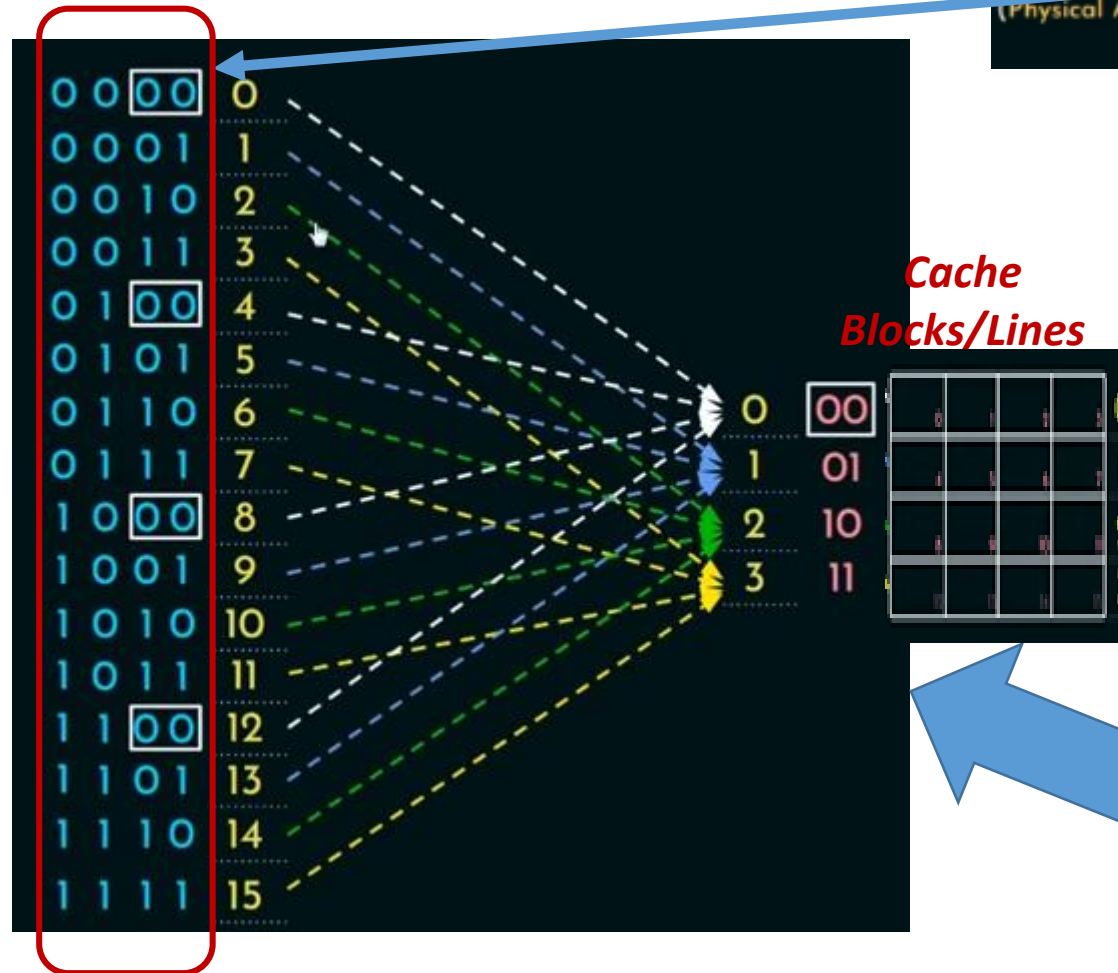
Mapping Techniques - Direct Mapping

Mapping 64 words of Main memory on to 16 words of Cache
Mapping – Direct Mapping

How a block is identified in cache with MM Address bits?

Block is identified by first 4 bits of PA

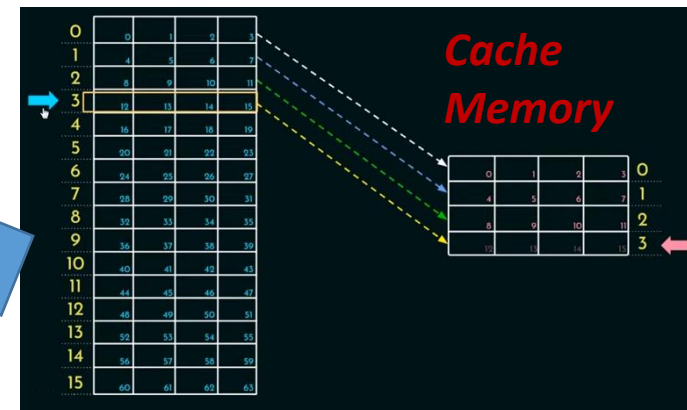
MM Blocks



2 bits identifies a word in a Block

Many-to-One Relationship

Main Memory



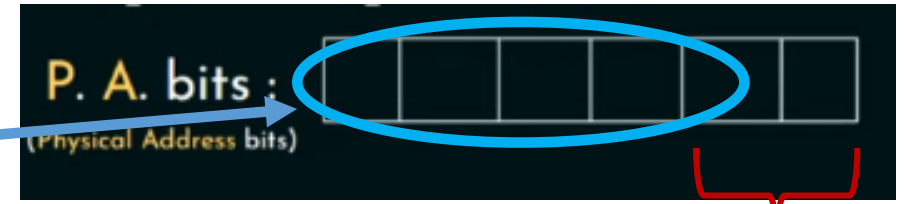
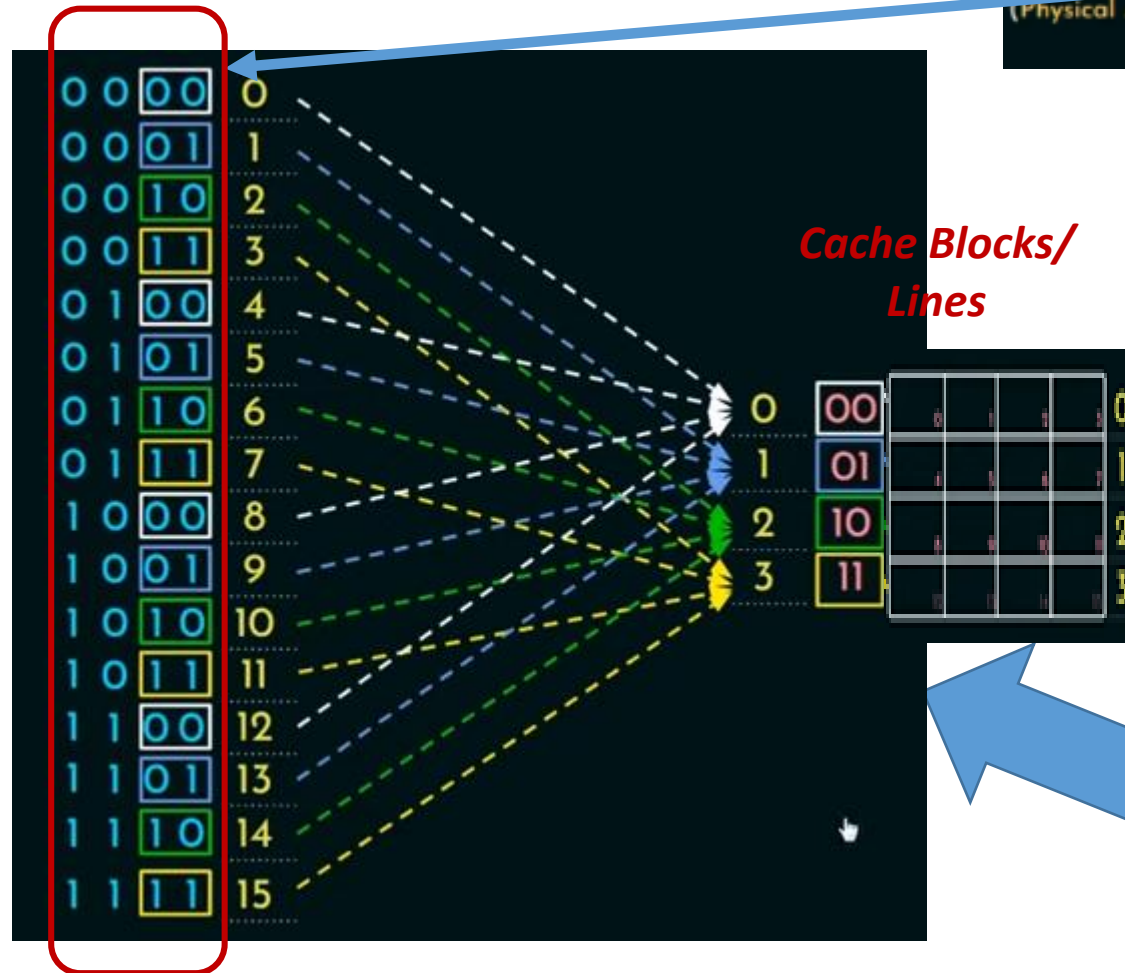
Mapping Techniques - Direct Mapping

Mapping 64 words of Main memory on to 16 words of Cache
Mapping – Direct Mapping

How a block is identified in cache with MM Address bits?

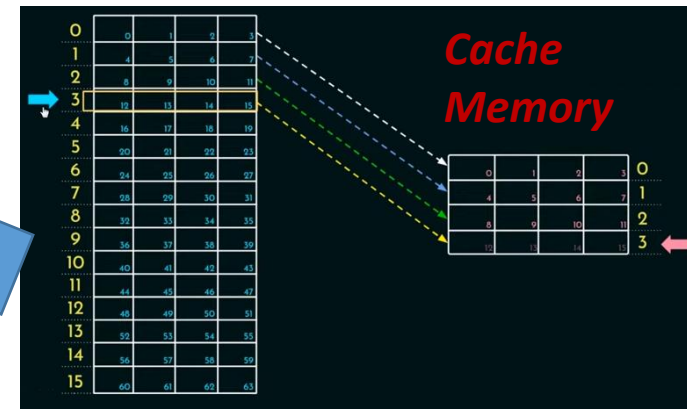
Block is identified by first 4 bits of PA

MM Blocks



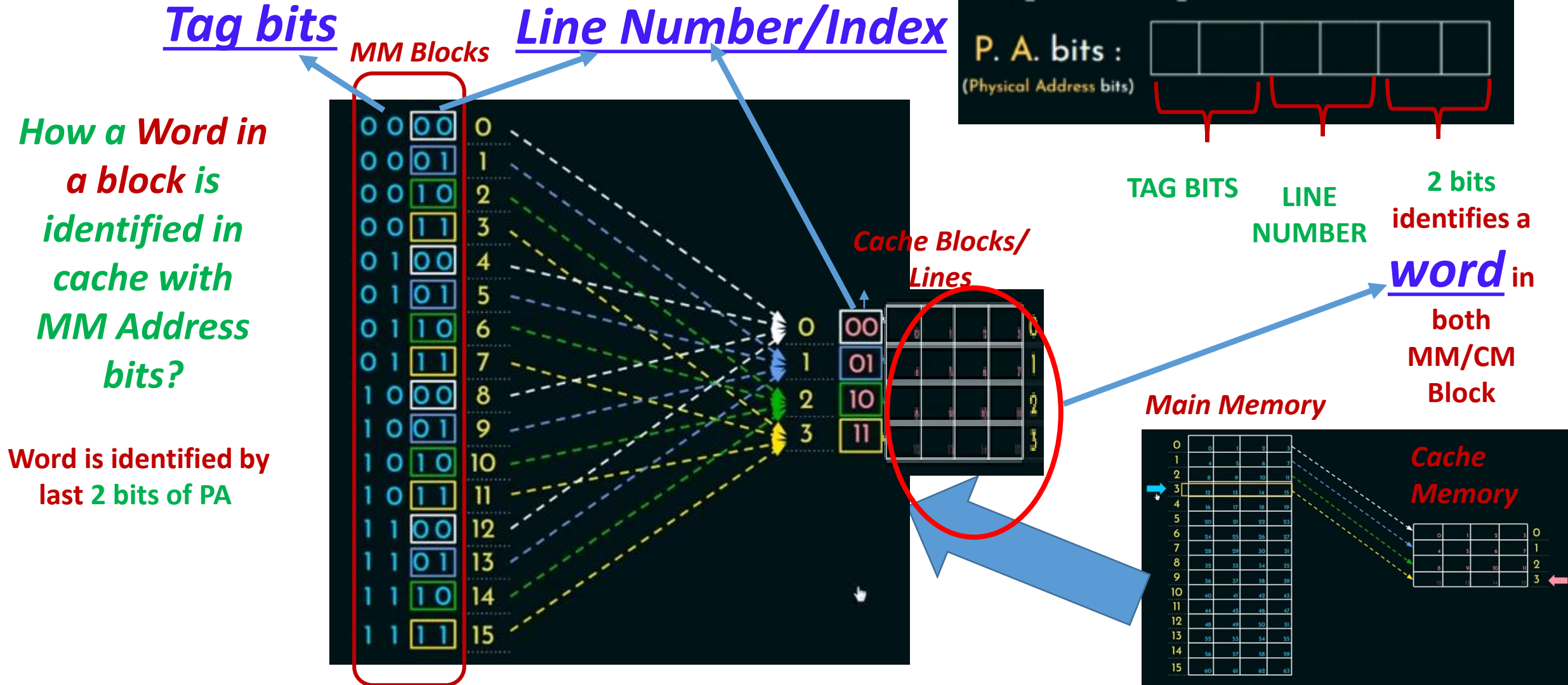
2 bits identifies a word in a Block

Main Memory



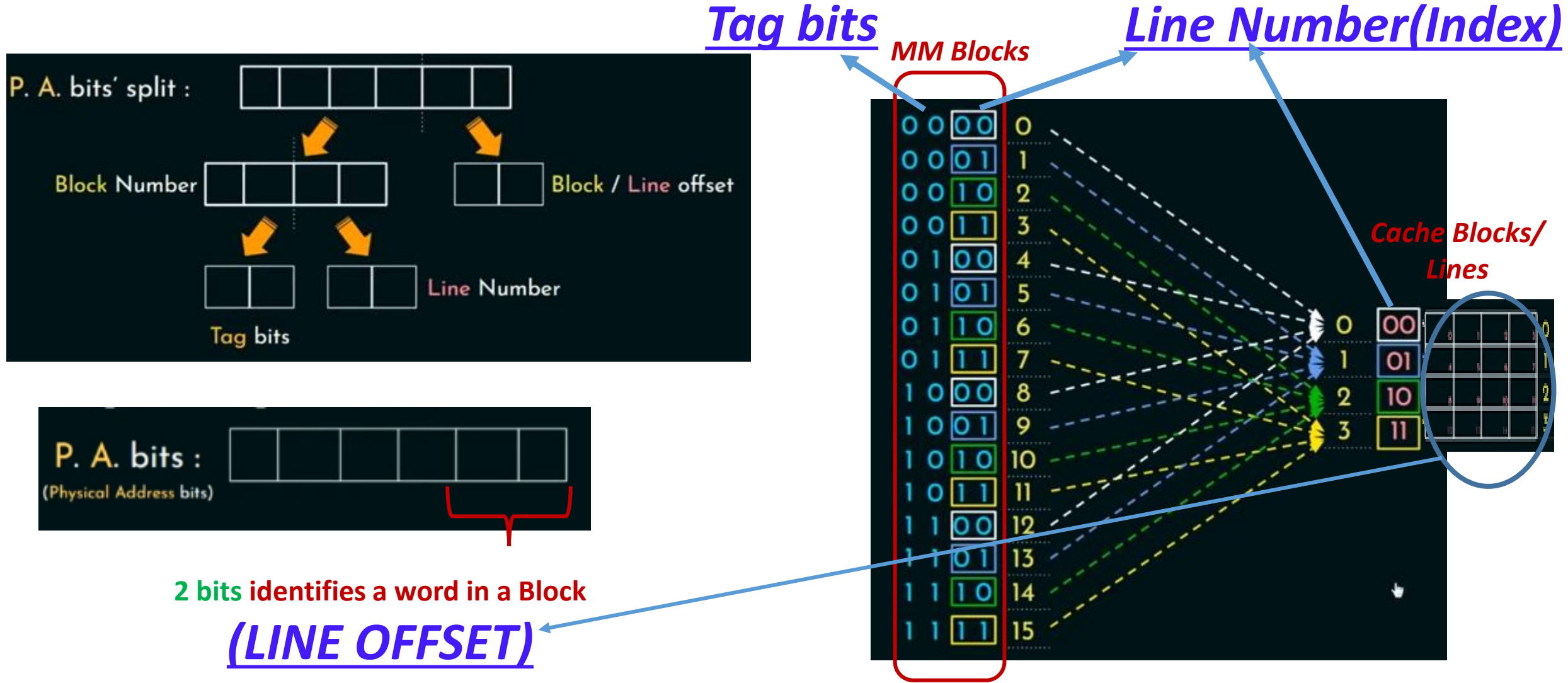
Mapping Techniques - Direct Mapping

Mapping 64 words of Main memory on to 16 words of Cache



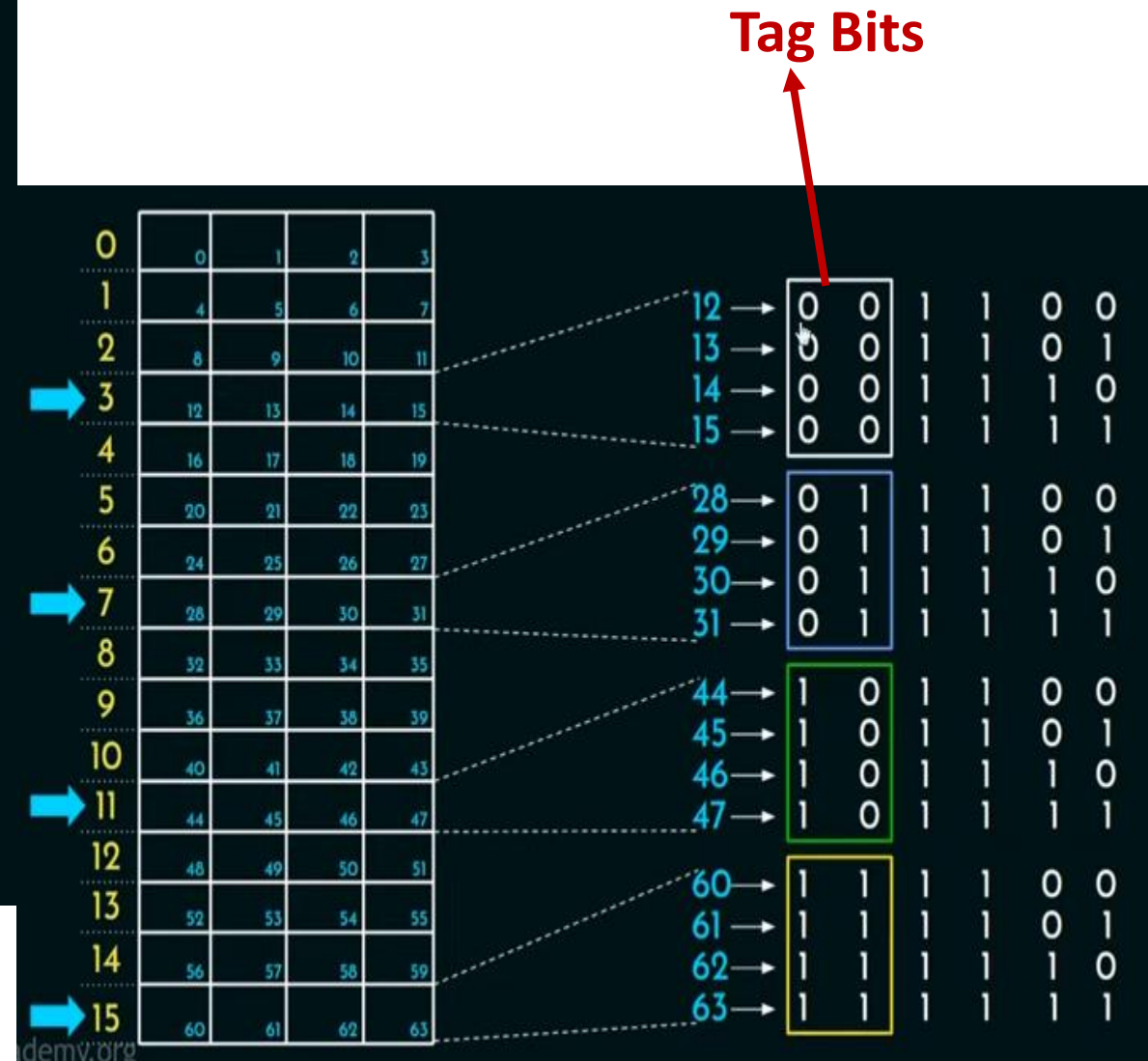
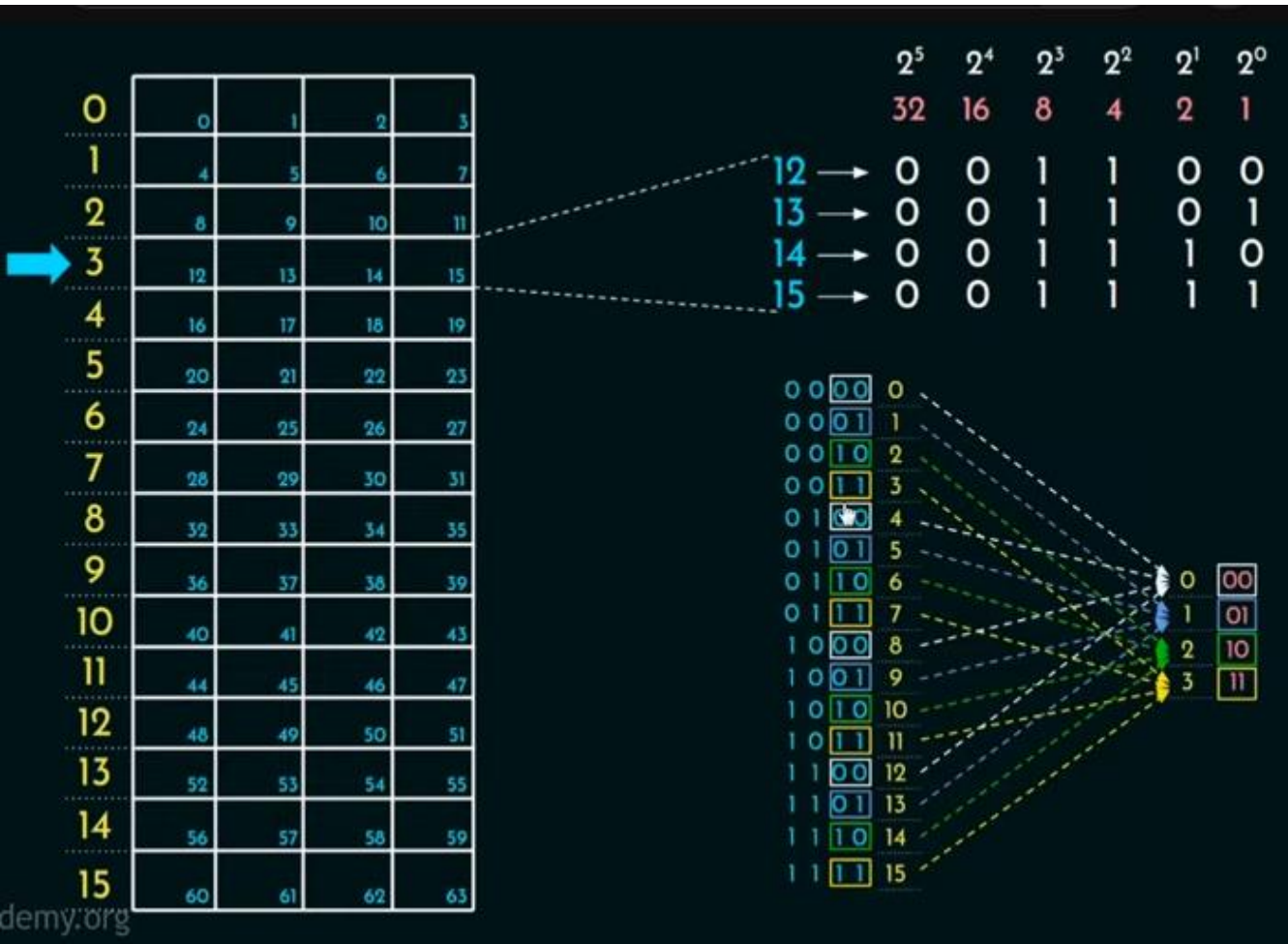
Mapping Techniques - Direct Mapping

Mapping 64 words of Main memory on to 16 words of Cache



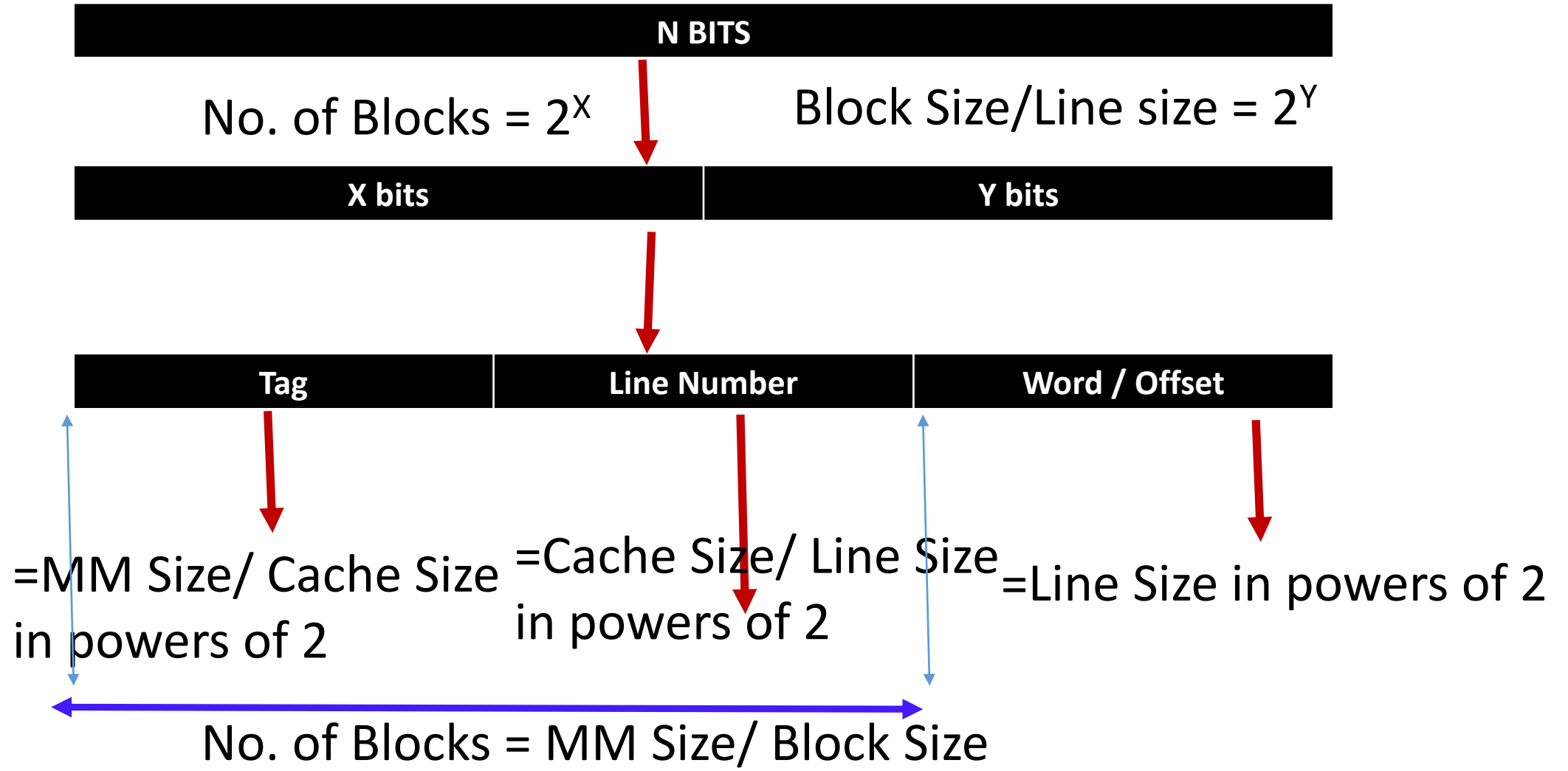
Mapping Techniques - Direct Mapping

Why called TAG bits?



Direct Mapping – Calculation of PA Bits needed

$$PA = MM \text{ Size} = 2^N$$



Mapping Techniques - Direct Mapping - Problems

Ex 1: MM Size: 4 GB
Cache Size: 1 MB
Block Size: 4 KB

1. P.A. bits' split?

1 Byte = 8 bits
1 KB = 1024 B(ytes) = 2^{10} B
1 MB = 1024 KB = 2^{20} B
1 GB = 1024 MB = 2^{30} B

Solution: Main Memory:

Calculation of PA bits:

$$\text{MM Size} = 4 \text{ GB} = 2^2 \times 2^{30} \text{ B} = 2^{(2+30)} \text{ B} = 2^{32} \text{ B}$$

$$\therefore \text{No. of P.A. bits} = \log_2 2^{32} = 32$$

Calculation of Block & Word(offset) bits:

$$\text{Block Size} = 4 \text{ KB} = 2^2 \times 2^{10} \text{ B} = 2^{12} \text{ B}$$

$$\text{No. of Blocks in MM} = 2^{32} / 2^{12} = 2^{20}$$

$$\therefore \text{Block number bits} = \log_2 2^{20} = 20$$

Cache Memory:

Calculation of Cache Block/
Line Number/Index bits:

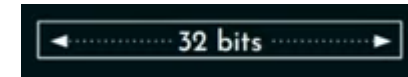
$$\text{Cache Size} = 1 \text{ MB} = 1 \times 2^{20} \text{ B} = 2^{20} \text{ B}$$

$$\text{No. of Lines in Cache} = 2^{20} / 2^{12} = 2^8$$

$$\therefore \text{Line number bits} = \log_2 2^8 = 8$$

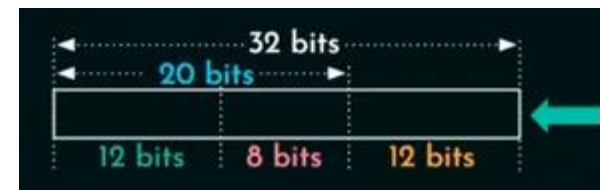
Calculation of TAG bits

$$\text{No. of Tag bits} : \text{P.A. bits} - (\text{Line no. bits} + \text{offset}) = 32 - (8 + 12) = 12$$



0	0	1
1	4	5
2	8	9
3	12	13
4	16	17
5	20	21
6	24	25
7	28	29
8	32	33
9	36	37
10	40	41
11	44	45
12	48	49
13	52	53
14	56	57
15	60	61

Physical Address Split:



Mapping Techniques - Direct Mapping - Problems

Ex 2: MM Size: 256 MB
Cache Size: 512 KB

• No. of tag bits?

Sol. Tag bits: Identifies the MM block residing in the Cache Line.

Block Size = Line Size

$\log_2 (\text{MM Size} : \text{Cache Size})$

$$\begin{aligned}\text{MM Size} &= 256 \text{ MB} \\ &= 2^8 \times 2^{20} \text{ B} \\ &= 2^{28} \text{ B}\end{aligned}$$

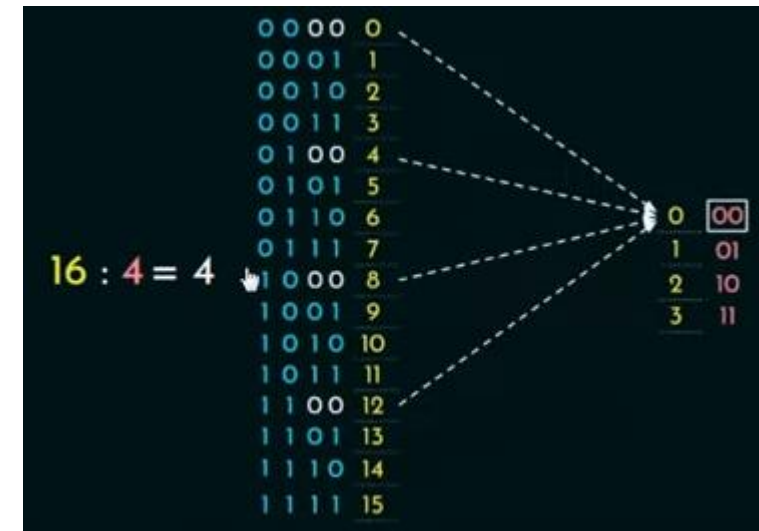
$$\begin{aligned}\text{Cache Size} &= 512 \text{ KB} \\ &= 2^9 \times 2^{10} \text{ B} \\ &= 2^{19} \text{ B}\end{aligned}$$

$$\begin{aligned}\therefore \text{No. of Tag bits} : \log_2 (2^{28} / 2^{19}) &= \text{Log}_2 2^{(28-19)} \\ &= \text{Log}_2 2^9 = 9 \quad \leftarrow\end{aligned}$$

Ex 2: MM Size: 256 MB
Cache Size: 512 KB
Block Size: 256 B
Word Size: 1 Byte

Logic:

Not Required



Mapping Techniques - Direct Mapping - Problems

Ex 3: Byte-addressable MM Size: 16 GB
Block Size: 16 KB
No. of Tag bits: 10

• Cache size?

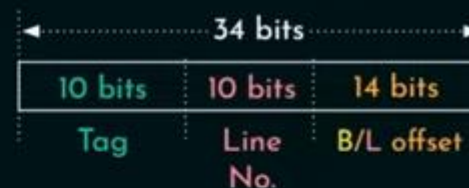
1 Byte	=	8 bits
1 KB	=	1024 B(ytes) = 2^{10} B
1 MB	=	1024 KB = 2^{20} B
1 GB	=	1024 MB = 2^{30} B

Sol. MM Size = 16 GB = $2^4 \times 2^{30}$ B = 2^{34} B

$\therefore$ No. of P.A. bits = $\log_2 2^{34} = 34$

Block Size = 16 KB = $2^4 \times 2^{10}$ B = 2^{14} B

$\therefore$ Block offset = $\log_2 2^{14} = 14$



No. of Line number bits : P.A. bits - (Tag bits + offset) = $34 - (10 + 14) = 10$

$\therefore$ No. of Cache Lines = 2^{10}

Line Size = 2^{14} B

Cache Size = $2^{10} \times 2^{14}$ B = $2^{(10+14)}$ B = 2^{24} B = $2^4 \times 2^{20}$ B = 16 MB ←

Parameters of Cache memory

- **Cache Hit**

- A referenced item is found in the cache by the processor

- **Cache Miss**

- A referenced item is not present in the cache

- **Hit ratio**

- Ratio of number of hits to total number of references = $\frac{\text{number of hits}}{\text{number of hits} + \text{number of Miss}}$

- **Miss penalty**

- Additional cycles required to serve the miss
- Time required for the cache miss depends on both the **latency and bandwidth**
- **Latency** – time to retrieve the first word of the block
- **Bandwidth** – time to retrieve the rest of this block

Types of Cache Miss

- Compulsory Miss / Cold Miss
- Conflict Miss/ Collision miss/ Interference miss
- Capacity Miss – Not because of mapping techniques ... because of size of cache.

➤ **Compulsory miss:** Any main memory block is missed for very first time is called Compulsory miss. It can be reduced by increasing the capacity of cache memory size.

➤ **Capacity miss:** if cache block is full then capacity miss occurs. It can be reduced by increasing the cache memory size.

➤ **Conflict miss:** if cache set is full and number of main memory block is mapped to same tag, then conflict miss occurs. It can be reduced by increasing the associativity. Conflict miss never occurs in fully-associative memory.

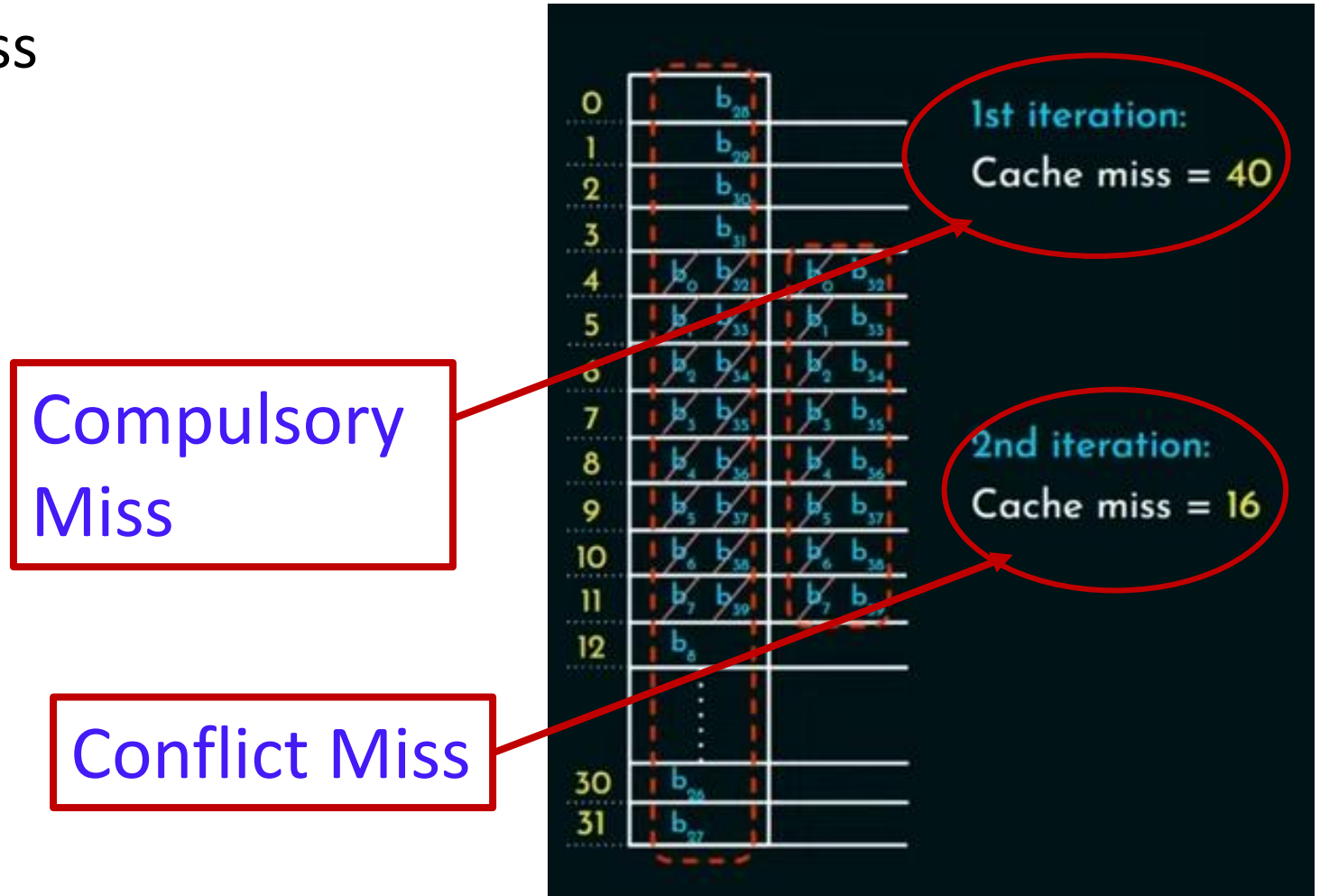
➤ **Note: -**

If number of set = 1, it is same as direct mapping.

Number of set = k implies only tag bits are present and same as fully-associative memory.

Mapping Techniques - Direct Mapping - Drawbacks

- Drawback – Conflict Miss

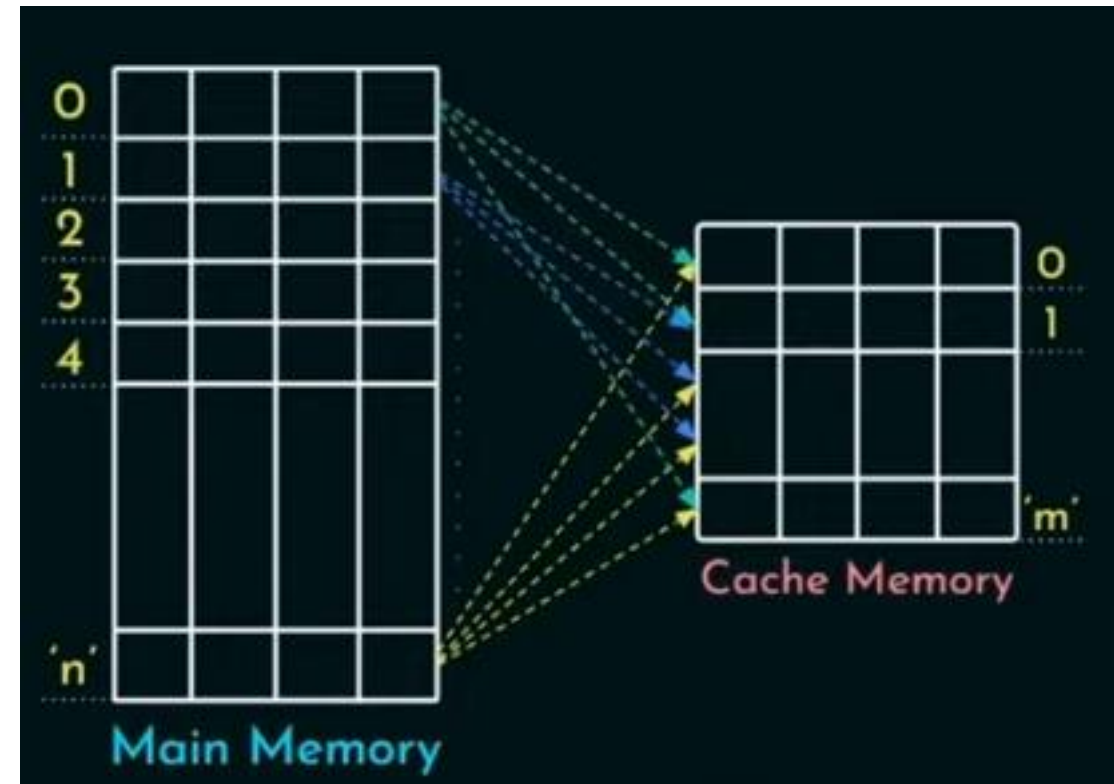


Mapping Techniques – (Fully) Associative Mapping

- To reduce Conflict Miss ----- Associative Mapping

- A block of main memory can **map to any line of the cache** that is freely available at that moment.
- This makes **fully associative mapping more flexible than direct mapping**.
- All the lines of cache are freely available.
- When **all the cache lines are occupied**, then one of the existing blocks will have to be **replaced**.

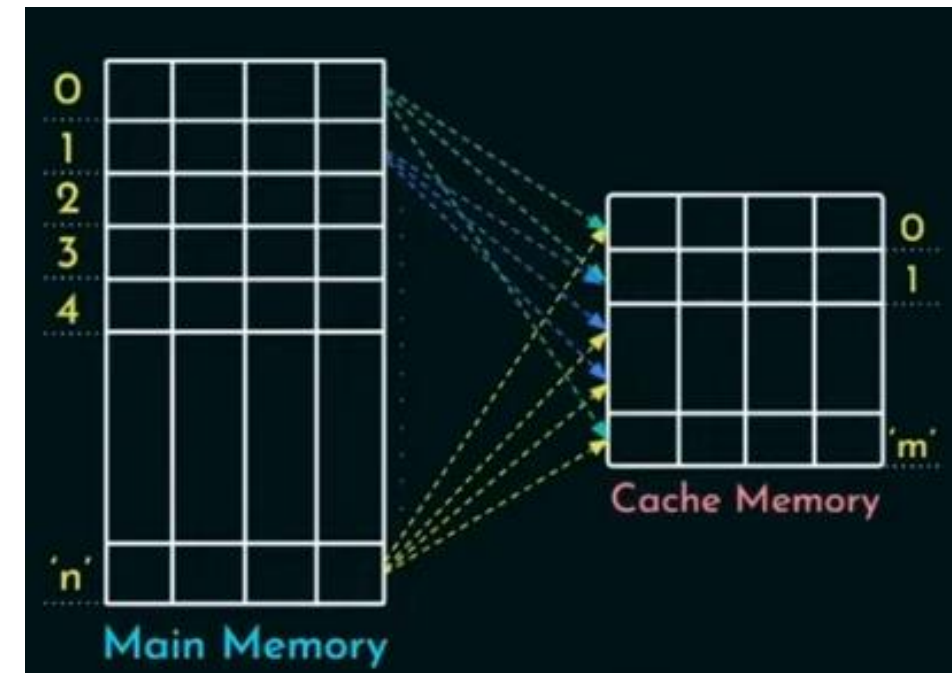
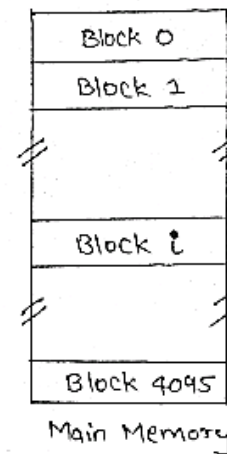
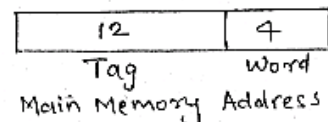
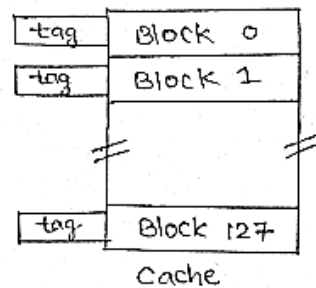
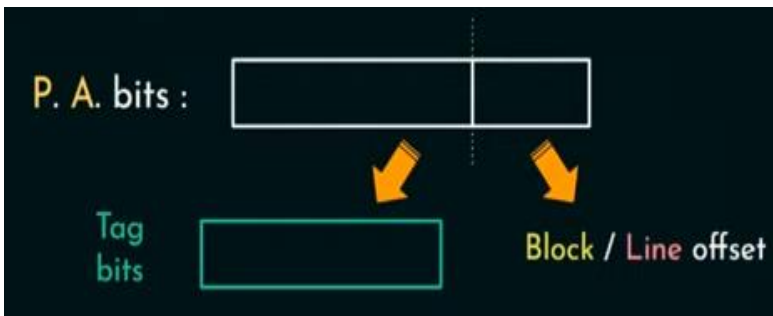
Many-to-Many Relationship



Mapping Techniques – (Fully) Associative Mapping

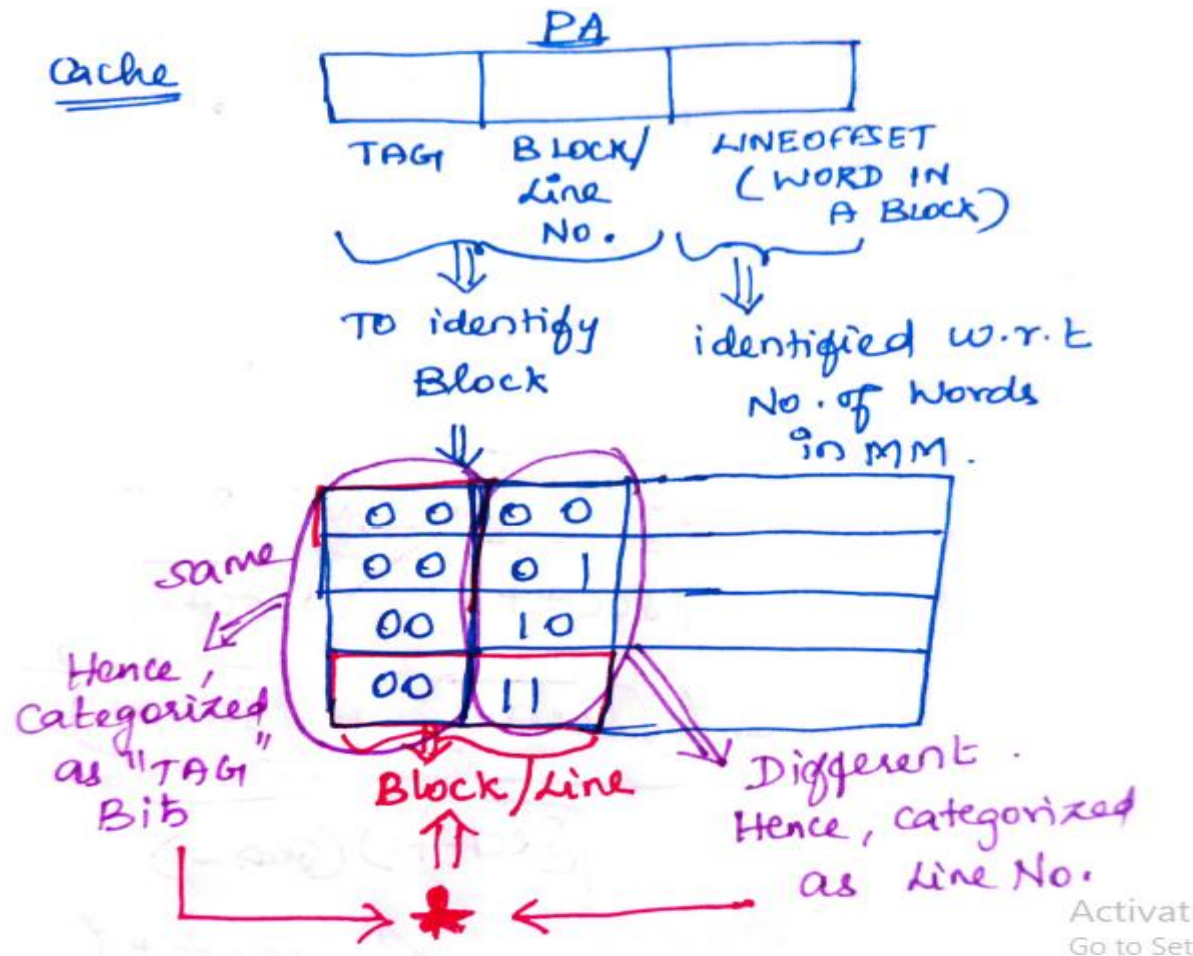
- Entire block number bits are used as TAG bits --- (Fully Associative)
 - No need of specifying the block number.

Many-to-Many Relationship

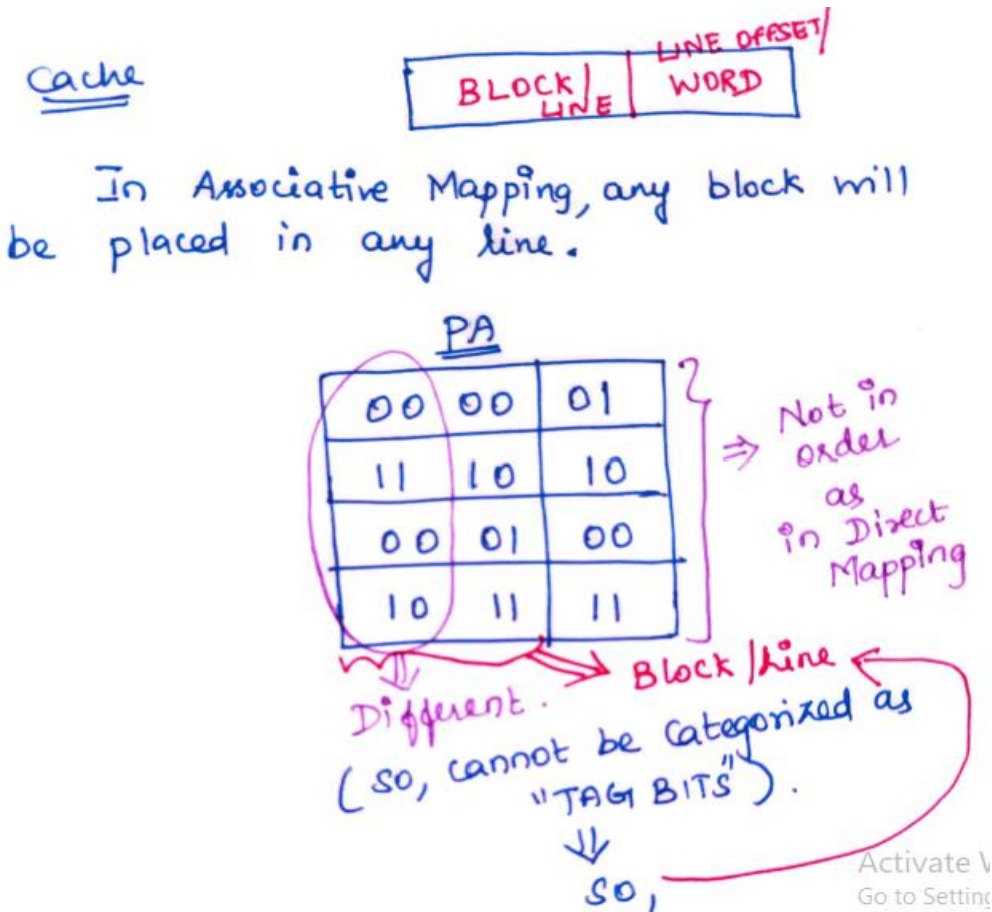


PA Address Calculation - Direct & Associative

Direct Mapping



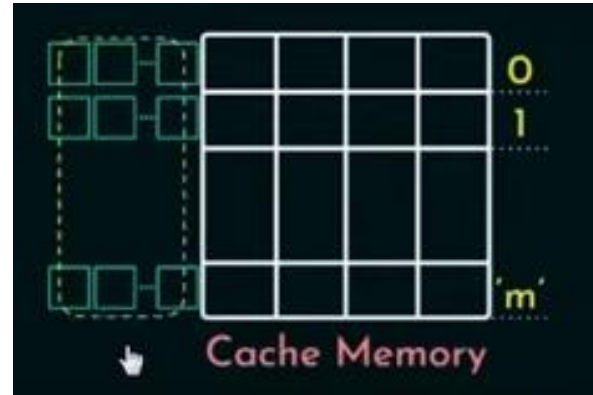
Associative Mapping



Mapping Techniques – (Fully) Associative Mapping

Disadvantages:

- A replacement algorithm is **required**.
- During retrieval, all the blocks need to be checked for the data.

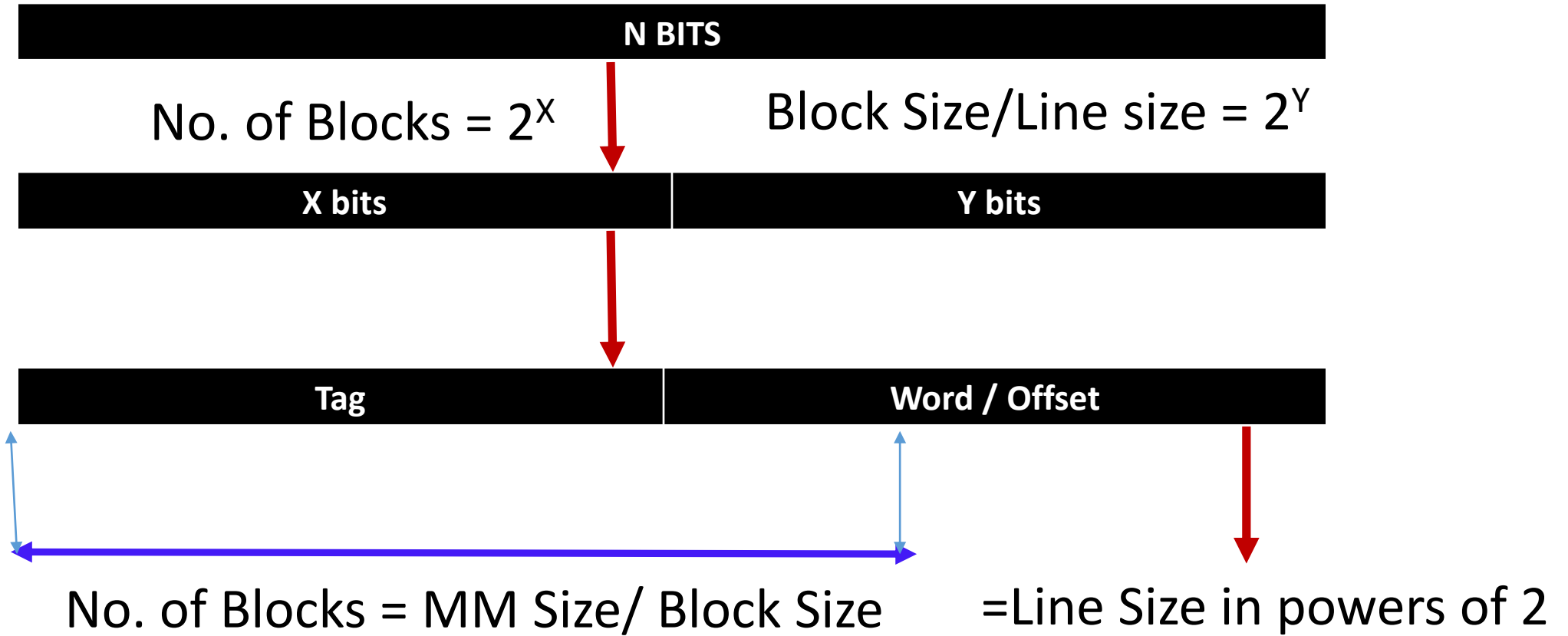


Need of Replacement Algorithm:

- A replacement algorithm is **required**.
- Replacement algorithm **suggests the block to be replaced if all the cache lines are occupied**.
- Thus, replacement algorithm like FCFS Algorithm, LRU Algorithm etc is employed.

Associative Mapping – Calculation of PA Bits needed

$$PA = MM \text{ Size} = 2^N$$



Mapping Techniques – (Fully) Associative Mapping

Example

Consider a fully associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB.

1. Number of bits in tag
2. Tag directory size

Given

Cache memory size = 16 KB

Block size = Frame size = Line size = 256 bytes

Main memory size = 128 KB

We consider that the memory is byte addressable.

Number of Bits in Physical Address-

Size of main memory

= 128 KB

= 2^{17} bytes

Thus, Number of bits in physical address = 17 bits



Mapping Techniques – (Fully) Associative Mapping

Number of Bits in Block Offset-

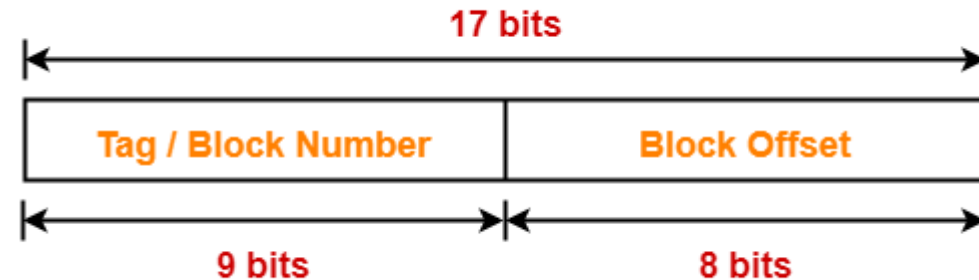
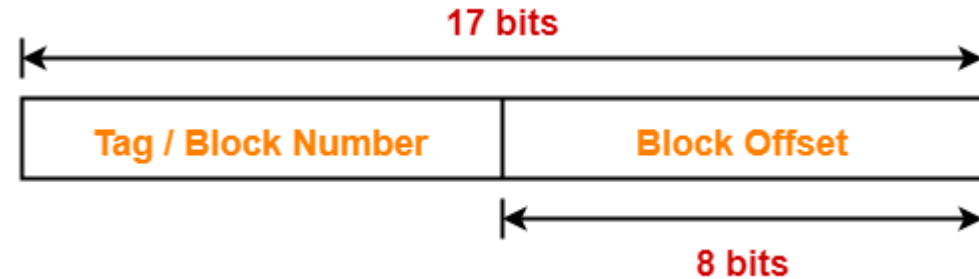
Block size
= 256 bytes
= 2^8 bytes

Thus, Number of bits in block offset = 8 bits

Number of Bits in Tag-

Number of bits in tag = Number of bits in phys
= 17 bits – 8 bits
= 9 bits

Thus, Number of bits in tag = 9 bits



Mapping Techniques – (Fully) Associative Mapping

Number of Lines in Cache

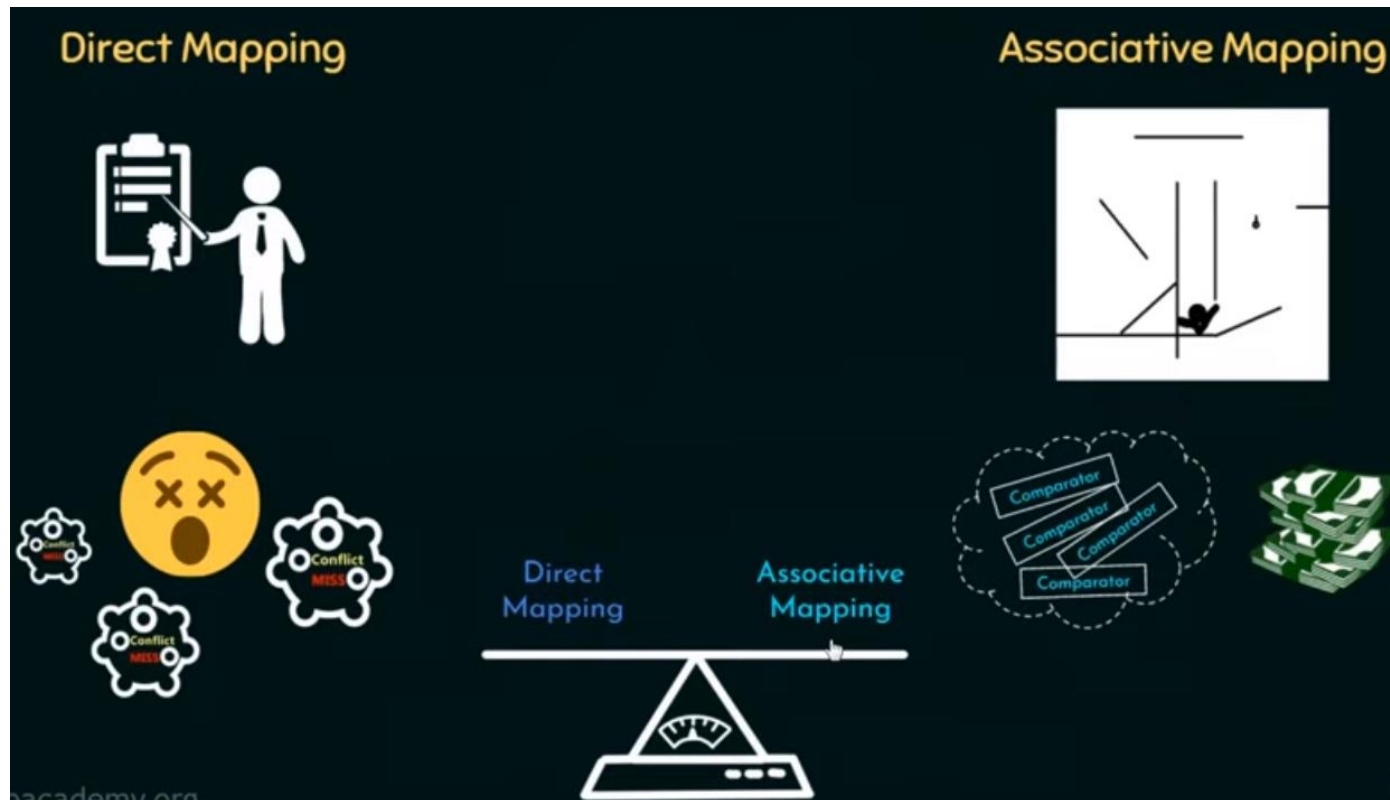
Total number of lines in cache = Cache size / Line size
= 16 KB / 256 bytes
= 16 x 1024 bytes / 256 bytes
= 64 lines

Tag Directory Size

= Number of lines in cache x Number of bits in tag
= 64 x 9 bits
= 576 bits
= 72 bytes
Thus, size of tag directory = 72 bytes

Mapping Techniques – Set Associative Mapping

- Combination of direct and associative mapping technique
- Cache blocks are grouped into sets and mapping allow block of main memory reside into any block of a specific set



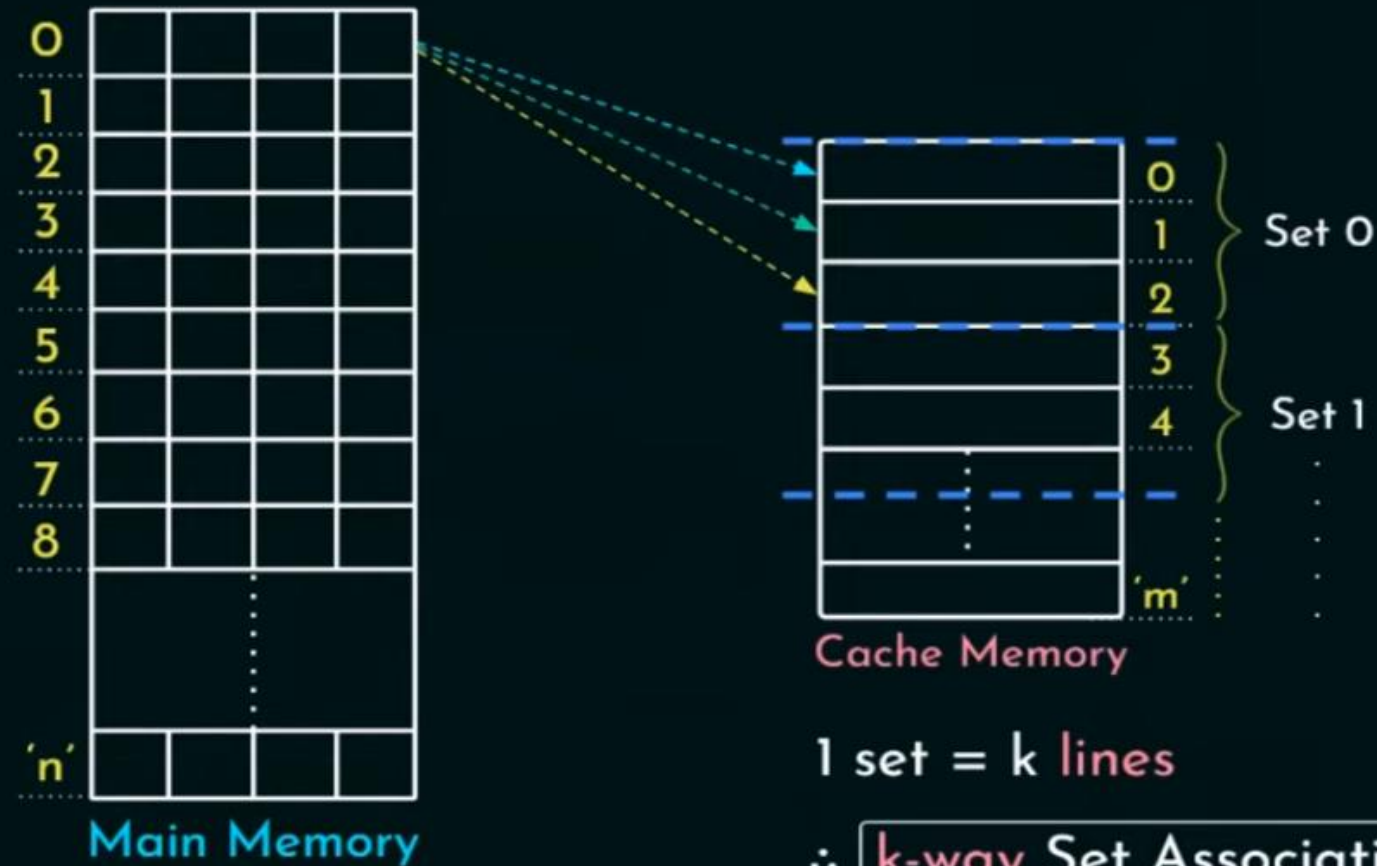
Mapping Techniques – Set Associative Mapping

- Hence contention problem of direct mapping is eased , at the same time , **hardware cost** is reduced by decreasing the size of associative **search**.
- For a cache with two blocks per set. In this case, memory block 0, 64, 128,.....,4032 map into cache set 0 and they can occupy any two block within this set.
- Having 64 sets means that the 6 bit set field of the address determines which set of the cache might contain the desired block.
- The tag bits of address must be associatively compared to the tags of the two blocks of the set to check if desired block is present. This is two way associative search.

$$\text{Cache set number} = (\text{MM Block Address}) \% (\text{Number of sets in Cache})$$

Mapping Techniques – Set Associative Mapping

Set Associative Mapping:



Mapping Techniques – Set Associative Mapping - Problems

Mapping using Set Associative

Block number (MM) % number of sets

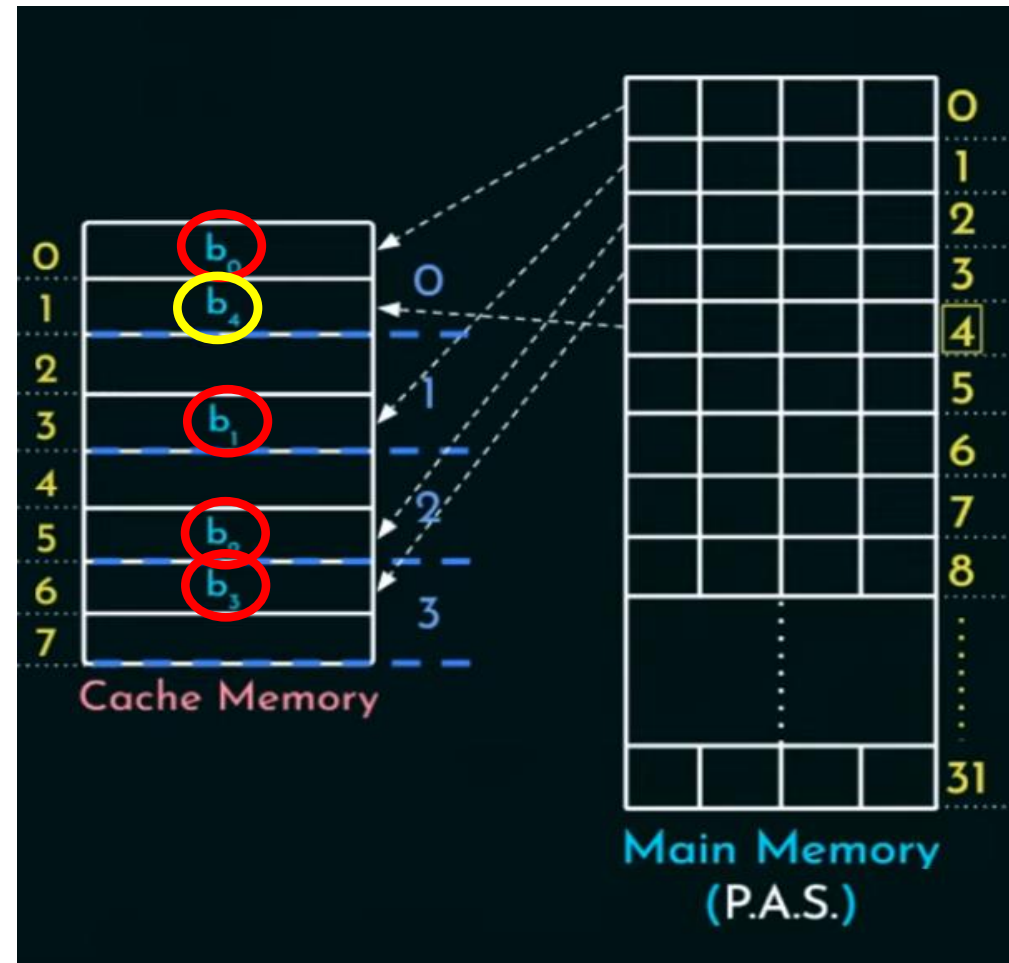
$$0 \% 4 = 0$$

$$1 \% 4 = 1$$

$$2 \% 4 = 2$$

$$3 \% 4 = 3$$

$$4 \% 4 = 0$$



Mapping Techniques – Set Associative Mapping

- $k = 2$ suggests that each set contains two cache lines. It is called as 2-way set associative mapping
- Since cache contains 6 lines, so number of sets in the cache = $6 / 2 = 3$ sets.
- Block 'j' of main memory can map to set number $(j \bmod 3)$ only of the cache.
- Within that set, block 'j' can map to any cache line that is freely available at that moment.
- If all the cache lines are occupied, then one of the existing blocks will have to be replaced.

Mapping Techniques – Set Associative Mapping

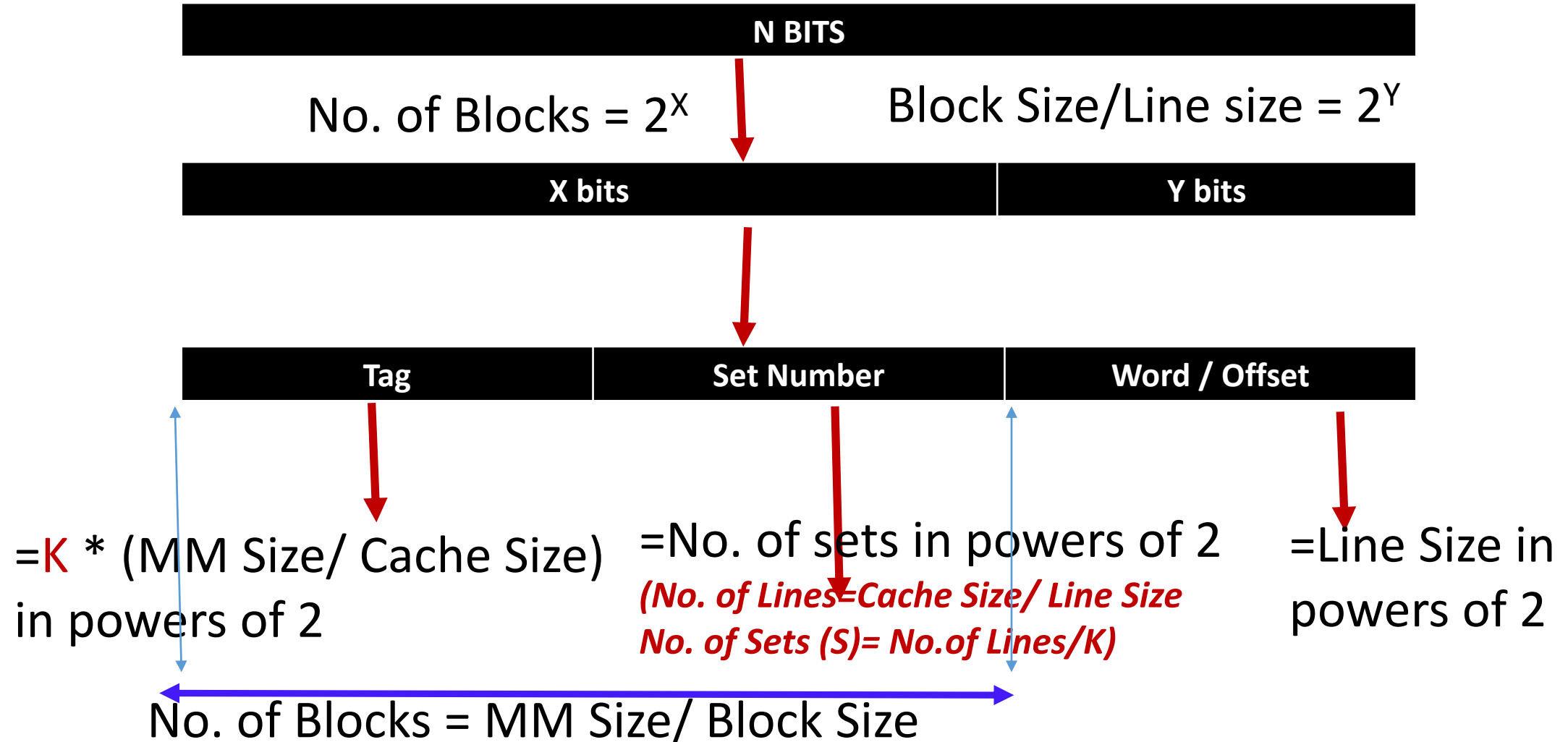
- If $k = 1$, then k -way set associative mapping becomes direct mapping i.e
- **1-way Set Associative Mapping $\equiv$ Direct Mapping**
- If $k = \text{Total number of lines in the cache}$, then k -way set associative mapping becomes **fully associative mapping**.

Need of Replacement Algorithm:

- Set associative mapping is a combination of direct mapping and fully associative mapping.
- It uses fully associative mapping within each set.
- Thus, set associative mapping requires a replacement algorithm.

Set Associative Mapping – Calculation of PA Bits needed

$$PA = MM \text{ Size} = 2^N$$



Mapping Techniques – Set Associative Mapping - Problems

Example 1

Byte addressable MM Size: 128 B
Cache Size: 32 B
Block Size: 4 B
2-way Set Associative 1 set = 2 lines

1 Byte = 8 bits
1 KB = 1024 B(ytes) = 2^{10} B
1 MB = 1024 KB = 2^{20} B
1 GB = 1024 MB = 2^{30} B

Solution: Main Memory:

Calculation of PA bits: $128 \text{ B} = 2^7 \text{ B} \therefore \text{P.A. bits} = 7$

7 bits

Calculation of Block & Word(offset) bits:

Block Size = $2^2 \text{ B} \therefore \text{offset} = 2$
 $\therefore \text{No. of MM Blocks} = 2^7 / 2^2$
 $= 2^5$

Cache Memory:

Number of lines and sets

Cache Size = 2^5 B
 $\therefore \text{No. of lines} = 2^5 / 2^2 = 2^3$
 $\therefore \text{No. of sets} = 2^3 / 2^1 = 2^2$

Calculation of TAG bits PA-(set no+offset) $7-(2+2)=3$

Physical Address Split:



Mapping Techniques – Set Associative Mapping - Problems

Example 1 - Elaborated

Ex : Byte addressable MM Size: 128 B
Cache Size: 32 B
Block Size: 4 B
2-way Set Associative

Mapping Techniques – Set Associative Mapping - Problems

Example 1 - Elaborated

Ex : Byte addressable MM Size: 128 B

Cache Size: 32 B

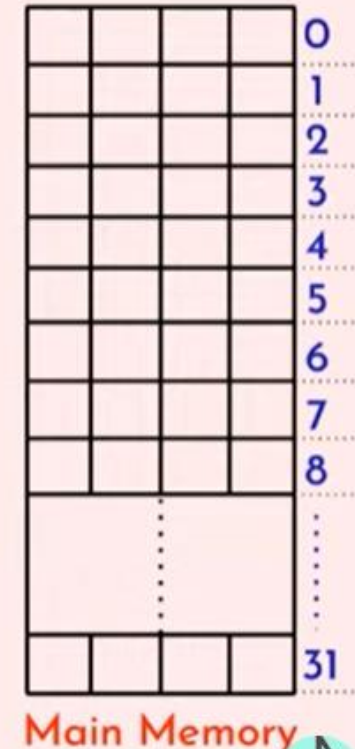
Block Size: 4 B

2-way Set Associative

P.A.S. = 128 B = 2^7 B $\therefore$ P.A. bits = 7

Block Size = 2^2 B $\therefore$ offset = 2

$\therefore$ No. of MM Blocks = $2^7 / 2^2$
 $= 2^5$



Mapping Techniques – Set Associative Mapping - Problems

Example 1 - Elaborated

Ex : Byte addressable MM Size: 128 B

Cache Size: 32 B

Block Size: 4 B

2-way Set Associative 1 set = 2 lines

P.A.S. = 128 B = 2^7 B $\therefore$ P.A. bits = 7

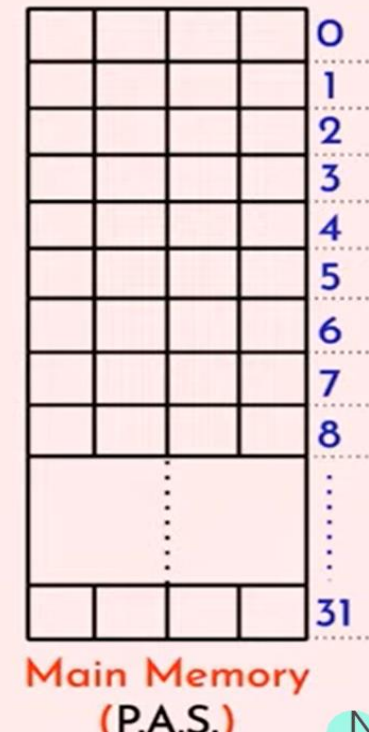
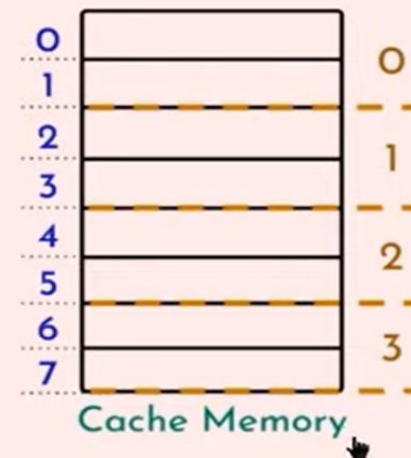
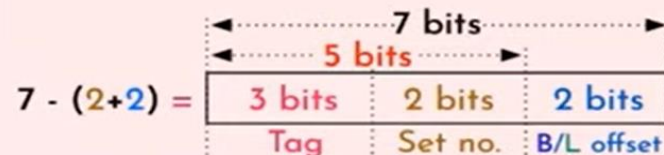
Block Size = 2^2 B $\therefore$ offset = 2

$\therefore$ No. of MM Blocks = $2^7 / 2^2$
 $= 2^5$

Cache Size = 2^5 B

$\therefore$ No. of lines = $2^5 / 2^2 = 2^3$

$\therefore$ No. of sets = $2^3 / 2^1 = 2^2$



Mapping Techniques – Set Associative Mapping - Problems

Example 1 - Elaborated

Ex : Byte addressable MM Size: 128 B

Cache Size: 32 B

Block Size: 4 B

2-way Set Associative 1 set = 2 lines

P.A.S. = 128 B = 2^7 B $\therefore$ P.A. bits = 7

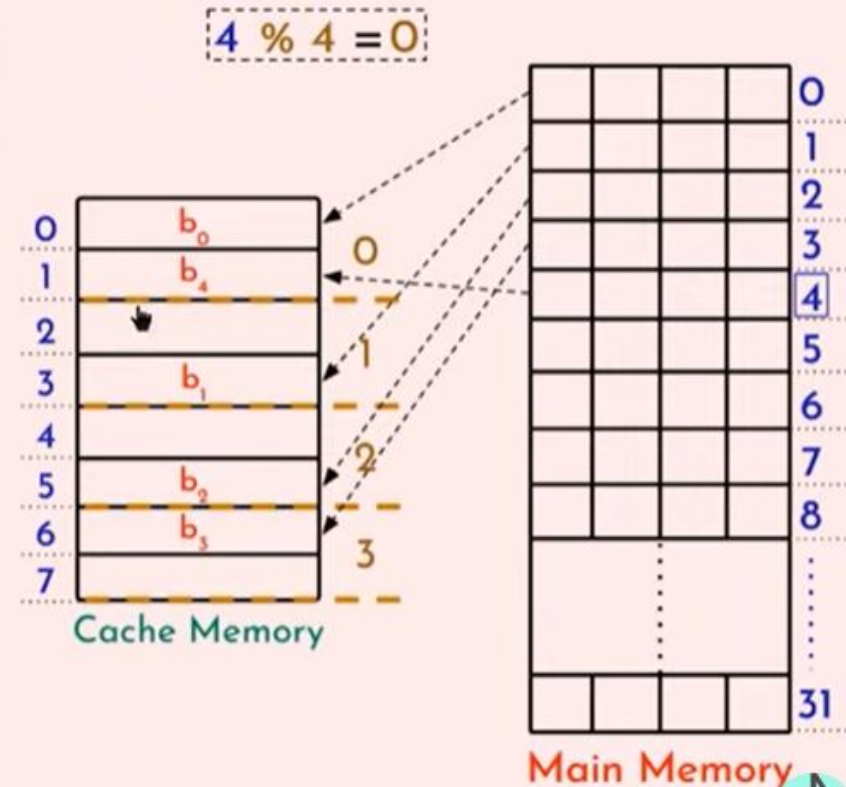
Block Size = 2^2 B $\therefore$ offset = 2

$\therefore$ No. of MM Blocks = $2^7 / 2^2$
 $= 2^5$

Cache Size = 2^5 B

$\therefore$ No. of lines = $2^5 / 2^2 = 2^3$

$\therefore$ No. of sets = $2^3 / 2^1 = 2^2$



Mapping Techniques – Set Associative Mapping - Problems

Example 2

Ex: MM Size: 128 kB
Cache Size: 16 kB
Block Size: 256 B

Byte addressable memory

- P.A. split?

2-way Set Associative

4-way Set Associative

8-way Set Associative

1 set = 2 lines

Sol. MM Size = 128 kB = 2^{17} B $\therefore$ P.A. bits = 17

Block Size = 256 B = 2^8 B $\therefore$ Offset = 8

$\therefore$ No. of MM Blocks = $2^{17} / 2^8 = 2^9$

Cache Size = 16 kB = 2^{14} B

$\therefore$ No. of Cache Lines = $2^{14} / 2^8 = 2^6$

"2-way": # sets = $2^6 / 2^1 = 2^5$



$$9 - 5 = 4$$

5



Mapping Techniques – Set Associative Mapping - Problems

Example 2

Ex: MM Size: 128 kB
Cache Size: 16 kB
Block Size: 256 B

Byte addressable memory

- P.A. split?

2-way Set Associative

1 set = 2 lines

4-way Set Associative

1 set = 4 lines

8-way Set Associative

1 set = 8 lines

Sol. MM Size = 128 kB = 2^{17} B $\therefore$ P.A. bits = 17

Block Size = 256 B = 2^8 B $\therefore$ Offset = 8

$\therefore$ No. of MM Blocks = $2^{17} / 2^8 = 2^9$

Cache Size = 16 kB = 2^{14} B

$\therefore$ No. of Cache Lines = $2^{14} / 2^8 = 2^6$

"2-way": # sets = $2^6 / 2^1 = 2^5$

"4-way": # sets = $2^6 / 2^2 = 2^4$

"8-way": # sets = $2^6 / 2^3 = 2^3$



9-5	=	4	5
9-4	=	5	4
9-3	=	6	3

Mapping Techniques – Set Associative Mapping - Problems

Example 3

Byte addressable MM Size: 256 MB
Cache Size: 1 MB
Block Size: 128 B
2-way Set Associative

1. P.A split?
2. Tag directory size?
3. Show the format of main memory address

1 Byte = 8 bits
1 KB = 1024 B(ytes) = 2^{10} B
1 MB = 1024 KB = 2^{20} B
1 GB = 1024 MB = 2^{30} B

Solution: Main Memory:

Calculation of PA bits: P.A.S. = 256 MB = $2^{(8+20)}$ B = 2^{28} B $\therefore$ P.A. bits = 28

Calculation of Block & Word(offset) bits:

Block Size = 128 B = 2^7 B $\therefore$ offset = 7
 $\therefore$ No. of MM Blocks = $2^{28} / 2^7 = 2^{21}$

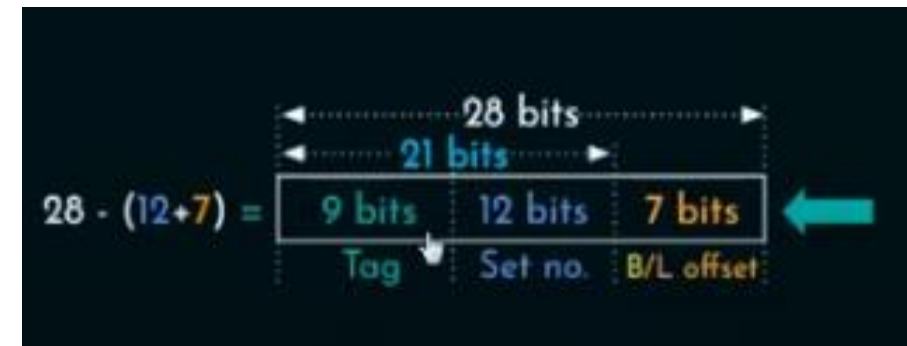
Cache Memory:

Number of lines and sets

Cache Size = 1 MB = 2^{20} B
 $\therefore$ No. of lines = $2^{20} / 2^7 = 2^{13}$
 $\therefore$ No. of sets = $2^{13} / 2^1 = 2^{12}$

Calculation of TAG bits PA-(set no+offset) $28-(12+7)=9$

Physical Address Split:



Tag directory Size = $2^{13} \times 9$ bits = 73728 bits

Mapping Techniques – Set Associative Mapping - Problems

Example 4

Byte addressable MM Size: 4 MB

Block Size: 64 B

Tag bits: 10

4-way Set Associative 1 set = 4 lines

1. P.A. split?

2. Cache Size?

1 Byte = 8 bits

1 KB = 1024 B(ytes) = 2^{10} B

1 MB = 1024 KB = 2^{20} B

1 GB = 1024 MB = 2^{30} B

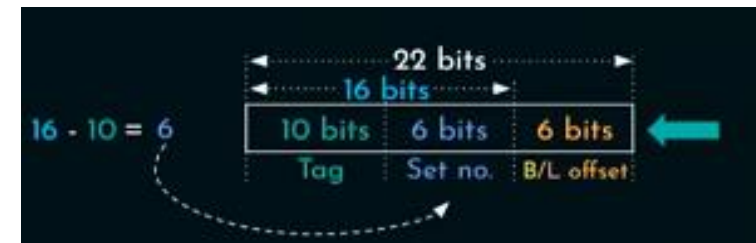
Solution:

Calculation of PA bits: P.A.S. = 4 MB = $2^{(2+20)}$ B = 2^{22} B $\therefore$ P.A. bits = 22

Calculation of Block & Word(offset) bits:

Block Size = 64 B = 2^6 B $\therefore$ offset = 6
 $\therefore$ No. of MM Blocks = $2^{22} / 2^6 = 2^{16}$

Physical Address Split:



Mapping Techniques – Set Associative Mapping - Problems

Example 4

Byte addressable MM Size: 4 MB

Block Size: 64 B

Tag bits: 10

4-way Set Associative 1 set = 4 lines

1. P.A. split?

2. Cache Size?

1 Byte = 8 bits

1 KB = 1024 B(ytes) = 2^{10} B

1 MB = 1024 KB = 2^{20} B

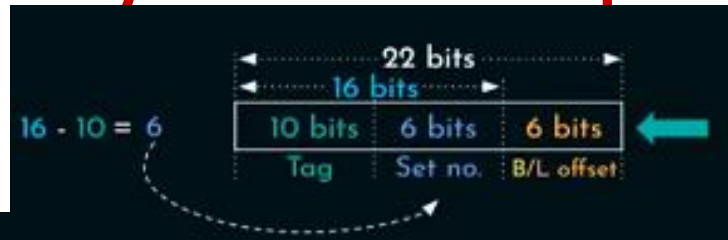
1 GB = 1024 MB = 2^{30} B

Solution:

Cache Memory:

Number of lines and sets

Physical Address Split:



Tag Bits = $K * (\text{MM Size} / \text{Cache Size})$
in powers of 2

Set Size = 4 = 2^2

Set no. bits = 6 $\therefore$ No. of sets = 2^6

$\therefore$ No. of lines = $2^6 \times 2^2 = 2^8$

Cache size $\rightarrow$

Cache Size = (No. of sets x Set Size) x Block Size

= $2^8 \times 2^6$ B

= 2^{14} B = 2^4 KB = 16 KB

(OR)

Tag bits given
 $2^{10} = 4 \times \frac{\text{MM Size}}{\text{Cache Size}}$

$2^{10} = 4 \times \frac{4 \times 2^{20}}{\text{Cache size}}$

Cache size = $\frac{4 \times 4 \times 2^{20}}{2^{10}}$
= $16 \times 2^{10} = 16 \text{ KB}$

Mapping Techniques – Set Associative Mapping - Problems

Example 5

Cache Size: 256 KB

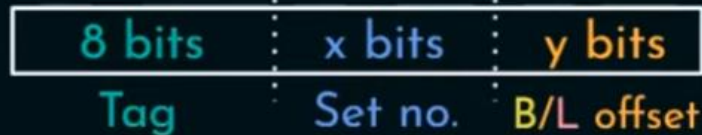
Tag bits: 8

8-way Set Associative

1 set = 8 lines

- Byte addressable MM Size?

Sol. Let, P.A. Split :



$$\text{Cache Size} = (\text{No. of sets} \times \text{Set Size}) \times \text{Block Size}$$

$$\therefore 256 \text{ KB} = 2^x \times 8 \times 2^y$$

$$\therefore 2^{(8+10)} / 8 = 2^x \times 2^y$$

$$\therefore 2^{x+y} = 2^{18} / 2^3 = 2^{15} \therefore x+y = 15$$

$$\text{P.A. bits} = 8 + 15 = 23 \therefore \text{MM Size} = 2^{23} \text{ B} = 2^3 \text{ MB} = 8 \text{ MB}$$

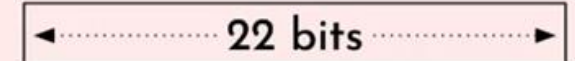
Mapping Techniques – Set Associative Mapping - Problems

Example 6

Q1: A block-set associative cache memory consists of 128 blocks divided into four block sets. The main memory consists of 16,384 blocks and each block contains 256 eight bit words.

1. How many bits are required for addressing the main memory?
- 2. How many bits are needed to represent the TAG, SET and WORD fields?

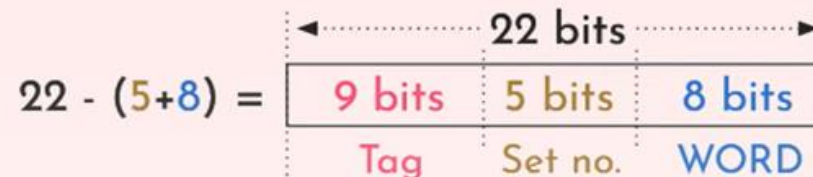
Sol. P.A. bits = 22



Block Size = 2^8 B $\therefore$ WORD = 8

Cache lines = 2^7

4-way Set Associative $\therefore$ # sets = $2^7 / 2^2 = 2^5$



Mapping Techniques – Set Associative Mapping - Problems

Example 7

- A set associative cache consists of 64 lines or slots, divided into four line sets. Main memory consists 4k blocks of 128 words each. Show the format of main memory addresses.

The cache is divided into 16 sets of 4 lines each. Therefore, 4 bits are needed to identify the set number. Main memory consists of $4K = 2^{12}$ blocks. Therefore, the set plus tag lengths must be 12 bits and therefore the tag length is 8 bits. Each block contains 128 words. Therefore, 7 bits are needed to specify the word.

	TAG	SET	WORD
Main memory address =	8	4	7

Mapping Techniques – Set Associative Mapping - Problems

Example 8

- A two-way set associative cache has lines of 16 bytes and a total size of 8k bytes. The 64-Mbyte main memory is byte addressable. Show the format of main memory addresses.

There are a total of $8 \text{ kbytes} / 16 \text{ bytes} = 512$ lines in the cache. Thus the cache consists of 256 sets of 2 lines each. Therefore 8 bits are needed to identify the set number. For the 64-Mbyte main memory, a 26-bit address is needed. Main memory consists of $64\text{-Mbyte} / 16 \text{ bytes} = 2^{22}$ blocks. Therefore, the set plus tag lengths must be 22 bits, so the tag length is 14 bits and the word field length is 4 bits.



Mapping Techniques – Set Associative Mapping - Problems

Example 9

- Consider a 32-bit microprocessor that has an on-chip 16-kB four-way set-associative cache. Assume that the cache has a line size of four 32-bit words. Draw a block diagram of this cache showing its organization and how the different address fields are used to determine a cache hit/miss. Identify the set number in cache for mapping the given memory address ABCDE8F8.

Mapping Techniques – Set Associative Mapping - Problems

Example 9 – Solution

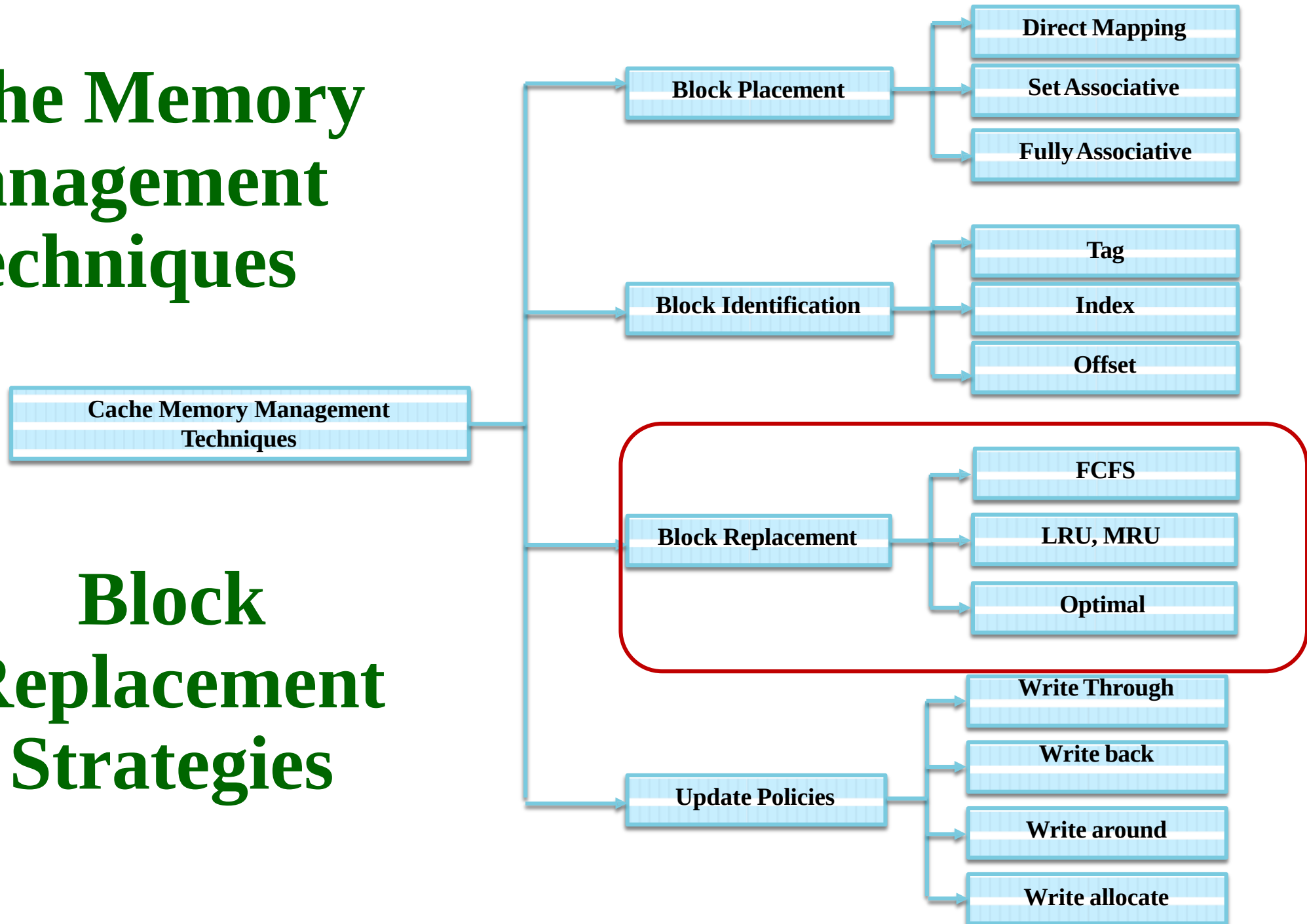
- Given: 16 Kb cache size. - 4 way set associative.
Hence, Line size = $4 \times 32 \text{ bit} = 16 \text{ bytes}$.
- Here, in the question "word" is mentioned and even "Where in the cache is the word from memory location" is asked. So, **word addressing is in use**.
So, offset bits = 2 for 4 words.
- No. of sets = no of lines / p (way)
- No. of lines = cache size / line size

tag(22 bit)	sets(8 bit)	block size(2 bit)
-------------	--------------	-------------------

- Identifying the Set Number: Address -- ABCDE8F8
its binary form is : 1010 1011 1100 1101 1110 1000 1111 1000
<1010 1011 1100 1101 1110 10> <00 1111 10> <00>
- it is mapped to set number 62 in cache.

Cache Memory Management Techniques

Block Replacement Strategies



Cache : Block Replacement Policies

- Cache memory size < main memory size
- Processor fetches data from cache memory to perform execution operation
- So, when required block is **not found** within cache, then main memory block is transferred to cache and previously present block is replaced---- Cache replacement policies are needed....
- **Replacement policies** – **used in**
 - Set-associative cache
 - Fully – Associative/ Associative Cache
- Cache Replacement Policies:
 - FIFO
 - Optimal Algorithm
 - LRU

Cache : Block Replacement Policies - FIFO

➤ Memory block which referenced first , that block is replaced first.

➤ Let main memory references are given:- 1, 2, 3, 4, 1, 2, 3, 5, 6, 7

➤ Number of blocks in cache memory=4

3				4	4	4	4	4	4	4
2			3	3	3	3	3	3	3	7
1		2	2	2	2	2	2	2	6	6
0	1	1	1	1	1	1	5	5	5	5
	1 miss	2 miss	3 miss	4 miss	1 hit	2 hit	3 hit	5 miss	6 miss	7 miss

Cache : Block Replacement Policies - FIFO

3				4	4	4	4	4	4	4
2			3	3	3	3	3	3	3	7
1		2	2	2	2	2	2	2	6	6
0	1	1	1	1	1	1	1	5	5	5
	1 miss	2 miss	3 miss	4 miss	1 hit	2 hit	3 hit	5 miss	6 miss	7 miss

- Total reference = 10
- Total number of hit = 3
- Total number of miss = 7
- Hit ratio = number of hit / total reference
 $= 3/10 = 0.3$
- %of hit ratio = $0.3 \times 100 = 30\%$
- Miss ratio = number of miss / total reference
 $= 7/10 = 0.7$
- %of miss ratio = $0.7 \times 100 = 70\%$
- Total probability = 1. If hit ratio = 0.3 then miss ratio = $1 - 0.3 = 0.7$

Cache : Block Replacement Policies - Optimal

➤ The page which is not referenced in near future, that page is replaced.

➤ It has no practical implementation.

➤ But gives more efficiency.

➤ Used as reference model.

➤ Let memory reference is given :- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 2

➤ Number of Cache block=4

3				2	2	2	2	2	2	2	2	2
2			1	1	1	1	1	4	4	4	4	4
1		0	0	0	0	0	0	0	0	0	0	0
0	7	7	7	7	7	3	3	3	3	3	3	3
	7 miss	0 miss	1 miss	2 miss	0 hit	3 miss	0 hit	4 miss	2 hit	3 hit	0 hit	2 hit

Cache : Block Replacement Policies - Optimal

3				2	2	2	2	2	2	2	2	2
2				1	1	1	1	4	4	4	4	4
1		1		0	0	0	0	0	0	0	0	0
0	7	7	7	7	7	3	3	3	3	3	3	3
	7 miss	0 miss	1 miss	2 miss	0 hit	3 miss	0 hit	4 miss	2 hit	3 hit	0 hit	2 hit

☞ Total reference = 12

☞ Total number of hit = 6

☞ Total number of miss = 6

☞ Hit ratio = number of hit / total reference
 $= 6/12 = 0.5$

☞ % of hit ratio = $0.5 \times 100 = 50\%$

☞ Miss ratio = number of miss / total reference
 $= 6/12 = 0.5$

☞ % of miss ratio = $0.5 \times 100 = 50\%$

Cache : Block Replacement Policies - LRU

- This algorithm replaces the cache memory block by main memory block, the block which is least recently used.
- Let memory reference is given :- 7, 0, 1, 2, 0, 3, 5, 4, 2, 3, 0
- Number of Cache block=4

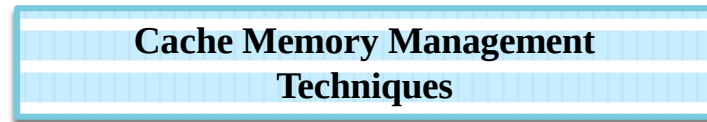
3				2	2	2	2	4	4	4	4
2			1	1	1	1	5	5	5	5	0
1		0	0	0	0	0	0	0	2	2	2
0	7	7	7	7	7	3	3	3	3	3	3
	7 miss	0 miss	1 miss	2 miss	0 hit	3 miss	5 miss	4 miss	2 miss	3 hit	0 miss

Cache : Block Replacement Policies - LRU

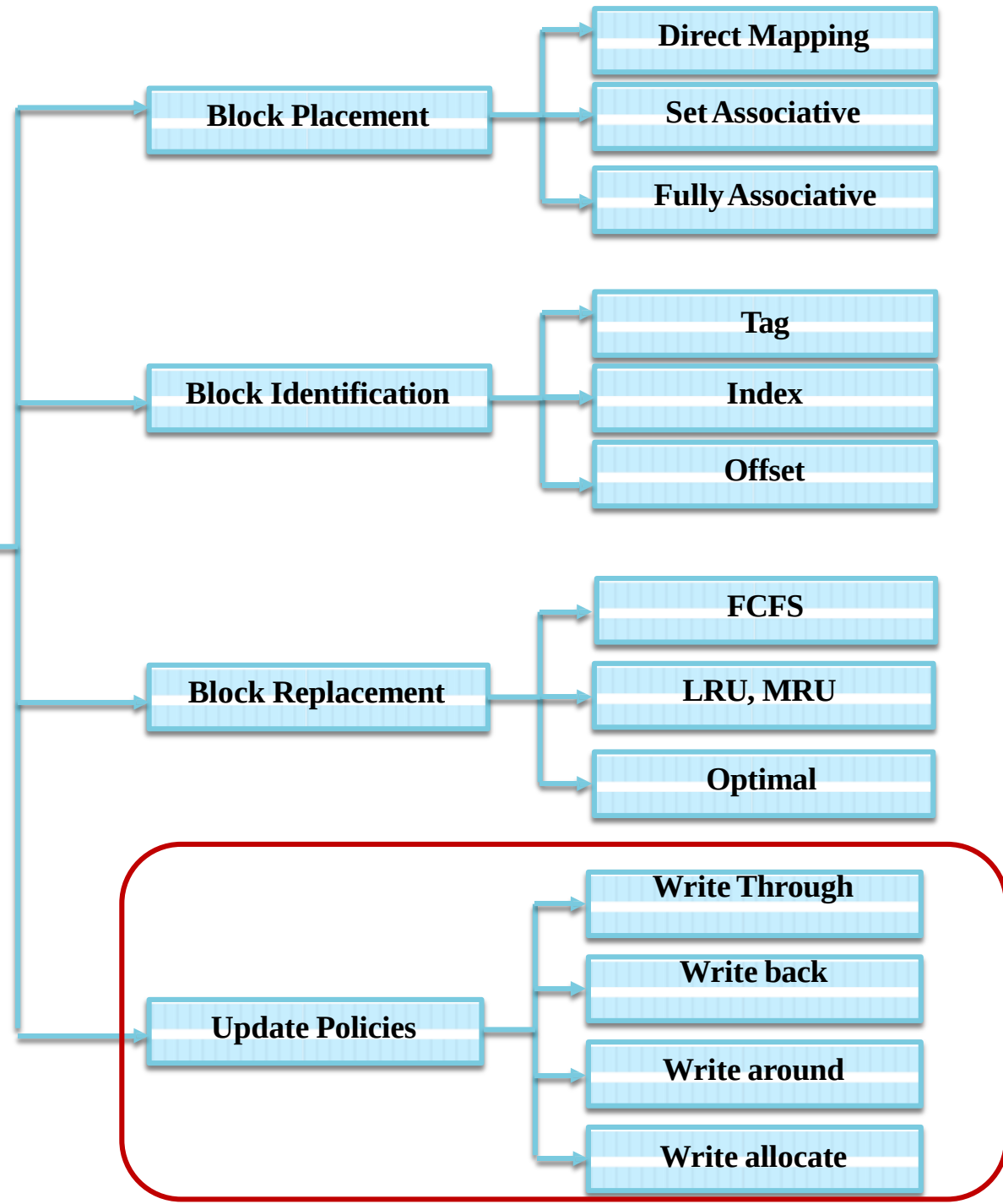
3				2	2	2	2	4	4	4	4
2				1	1	1	5	5	5	5	0
1		0	0	0	0	0	0	0	2	2	2
0	7	7	7	7	7	3	3	3	3	3	3
	7 miss	0 miss	1 miss	2 miss	0 hit	3 miss	5 miss	4 miss	2 miss	3 hit	0 miss

- Total reference = 11
- Total number of hit = 2
- Total number of miss = 9
- Hit ratio = number of hit / total reference
 $= 2/11 = .18$
- %of hit ratio = $0.18 \times 100 = 18\%$
- Miss ratio = number of miss / total reference
 $= 9/11 = 0.82$
- %of miss ratio = $0.82 \times 100 = 82\%$

Cache Memory Management Techniques



Update Policies



Cache Memory – Update Policies

- Update policy - determines how a cache & Main Memory is updated after an operation.

- Write through

- Write back

Used on Write HIT

- Write around

- Write Allocate

Used on Write MISS

Update Policy - Write-through

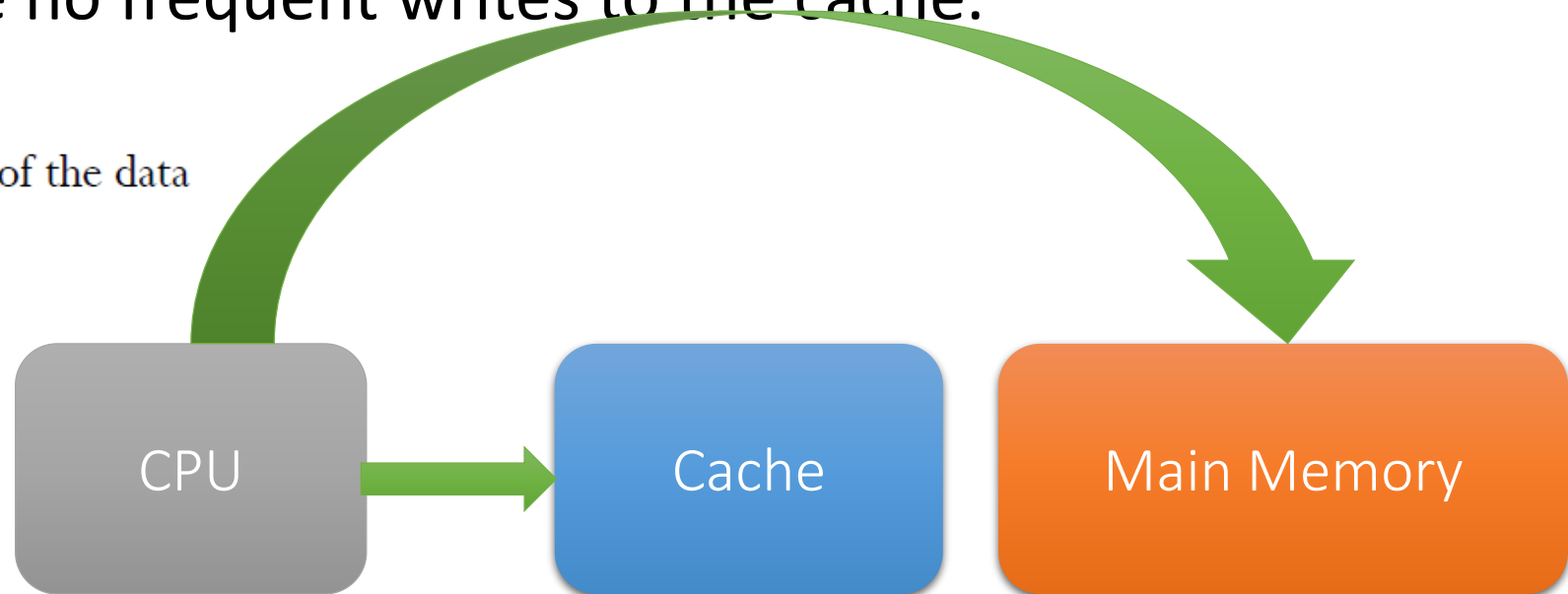
- Correspond to items currently in the cache (i.e. write Hit)
- Systems that write to main memory each time as well as to cache
- It's the easiest policy to implement, but it lowers the cache's performance.
- It's used when there are no frequent writes to the cache.

Advantage:

- easy to implement
- main memory always has the most current copy of the data (consistent)

Disadvantage:

- write is slower
- every write needs a main memory access
- as a result uses more memory bandwidth

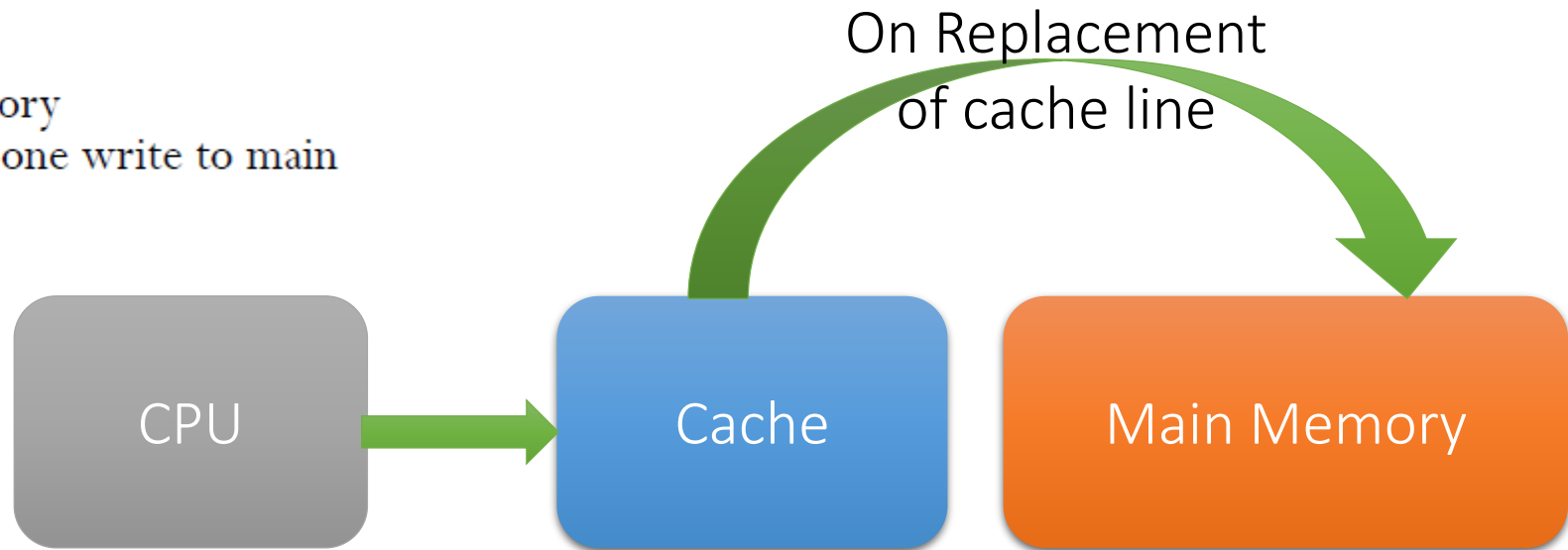


Update Policy - Write-Back/Copy Back

- Correspond to items currently in the cache (i.e. write Hit)
- Updating main memory until the block containing the altered item is removed from the cache

Advantage:

- writes occur at the speed of the cache memory
- multiple writes within a block require only one write to main memory
- as a result uses less memory bandwidth

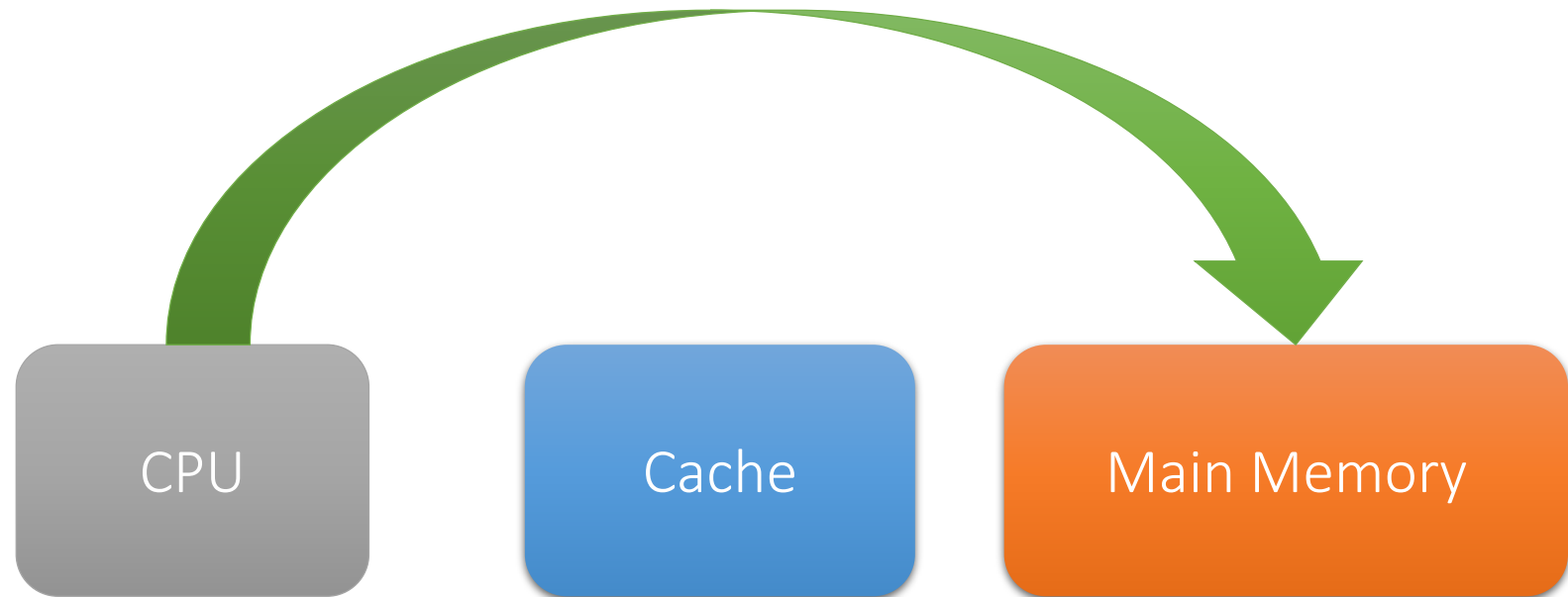


Disadvantage:

- harder to implement
- main memory is not always consistent with cache

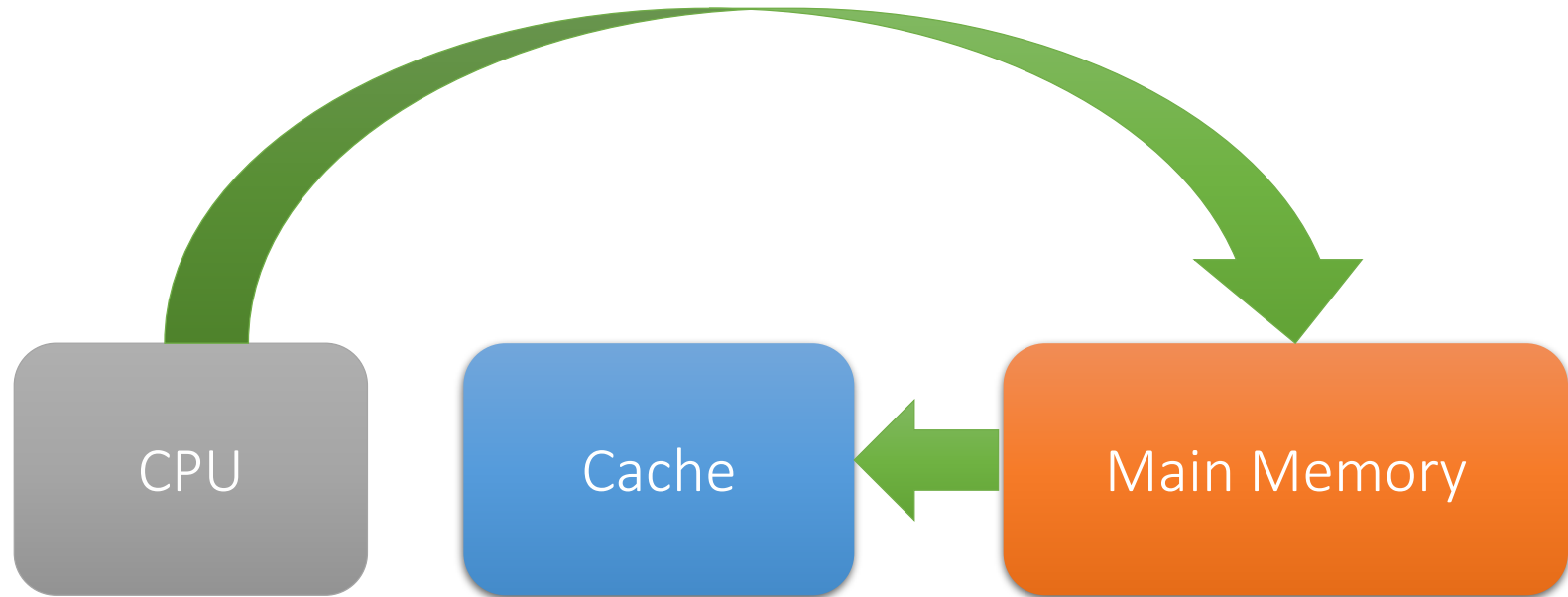
Update Policy - Write-Around

- Correspond to items not currently in the cache (i.e. write misses)
- The item could be updated in main memory only without affecting the cache.



Update Policy - Write-Allocate

- Correspond to items not currently in the cache (i.e. write misses)
- Update the item in main memory and bring the block containing the updated item into the cache.



Cache Update Policy - Summary

- Update policies
 - **Write Through**
 - On write hit, Update cache as well as Main Memory simultaneously
 - **Write Back**
 - On write hit, Update cache instantly and update main memory on cache line replacement
 - **Write –Around**
 - On write miss, Update main memory without affecting Cache
 - **Write-Allocate**
 - On write miss, Update main memory and copy updated block into Cache

Performance Metrics

- Cache Hit
- Hit Ratio
- Cache Miss
- Miss Ratio
- Miss Penalty
- Average Memory Access Time

Performance Metrics

- Hit Ratio:
 - Number of references found in the cache against Total number of references
 - Hit Ratio (h) = $\frac{\text{Number of references found in the Cache}}{\text{total number of memory references}}$
- Miss Ratio (m) = 1 - Hit Ratio
- Hit Ratio + Miss Ratio = 1
 - T_C is the average cache access time
 - T_M is Mean memory access time
 - T_A is average access time

Mean Memory Access Time (MMAT)

- Average Memory Access Time

= Hit ratio * Cache Memory Access Time

+

(1 – Hit ratio) * (Cache Memory Access Time + Time required to access a block of main memory)

$$\text{Average Access Time} = (h \times T_C) + (1 - h) \times (T_C + T_M)$$

Look through

- The cache is checked first for a hit, and if a miss occurs then the access to main memory is started

$$\text{Average Access Time} = (h \times T_C) + (1 - h) \times (T_C + T_M)$$

T_C is the average cache access time

T_M is Mean memory access time

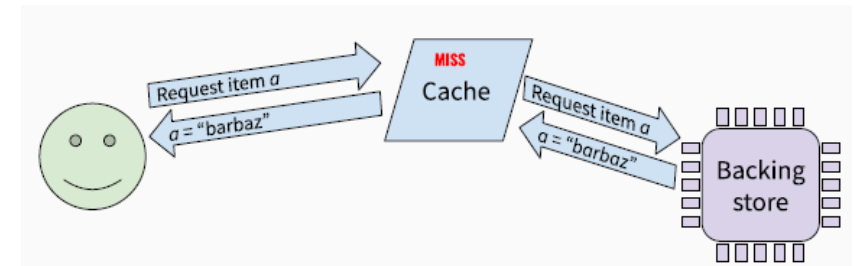
$$= (h \times T_C) + (T_C \times (1 - h)) + (T_M \times (1 - h))$$

$$= h \times T_C + T_C - h \times T_C + T_M - h \times T_M$$

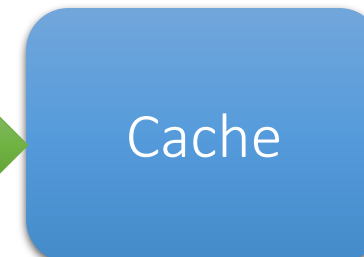
Now, we'll notice that the $h \times T_C$ term and the $-h \times T_C$ term cancel each other out:

$$= T_C + T_M - h \times T_M$$

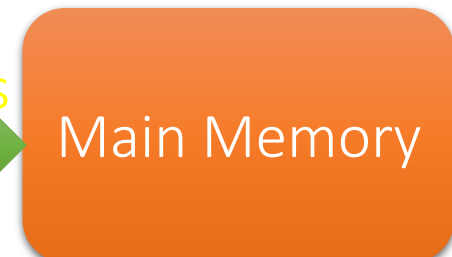
$$\text{Average Access Time} = T_C + (1 - h) \times T_M$$



$$T_{Avg} = (T_C) + (1-h) \times T_M$$



On Miss



Example

Assume that a computer system employs a cache with an access time of 20ns and a main memory with a cycle time of 200ns. Suppose that the hit ratio for reads is 90%,

- a) what would be the average access time for reads if the cache is a “look-through” cache?

The average read access time

$$T_{Avg} = (T_c) + (1-h) \times T_M$$

$$= 20\text{ns} + 0.10 * 200\text{ns} = 40\text{ns}$$

Look aside

- Access to main memory in parallel with the cache lookup

$$\text{Average Access Time} = (h \times T_C) + (1 - h) \times (T_C + T_M)$$

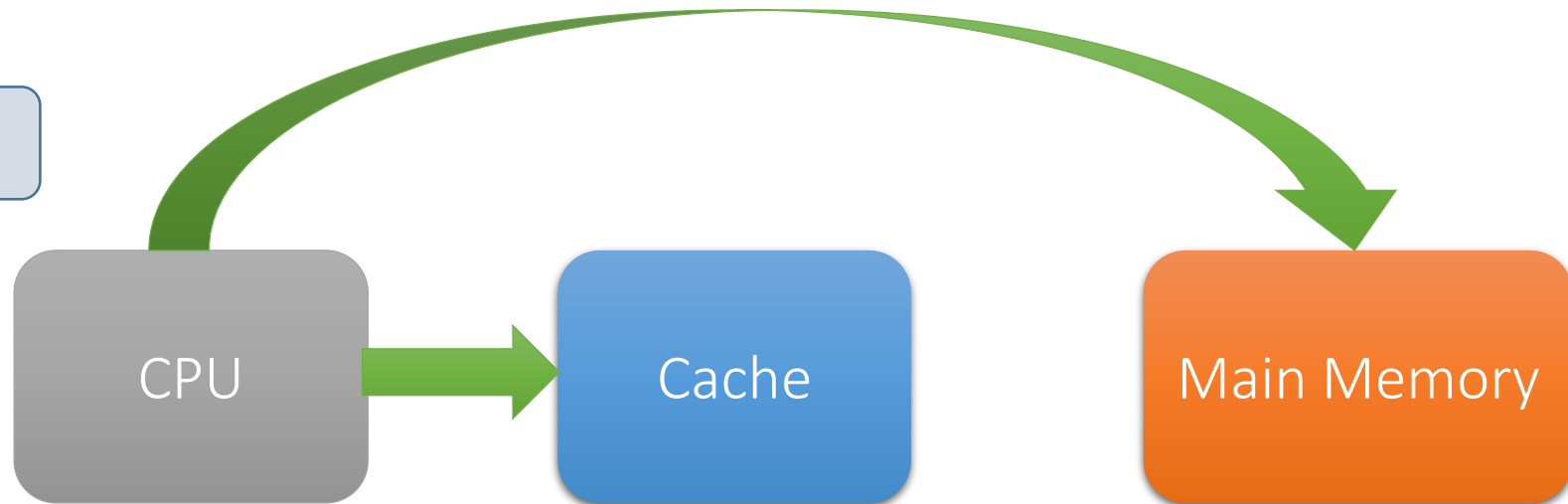
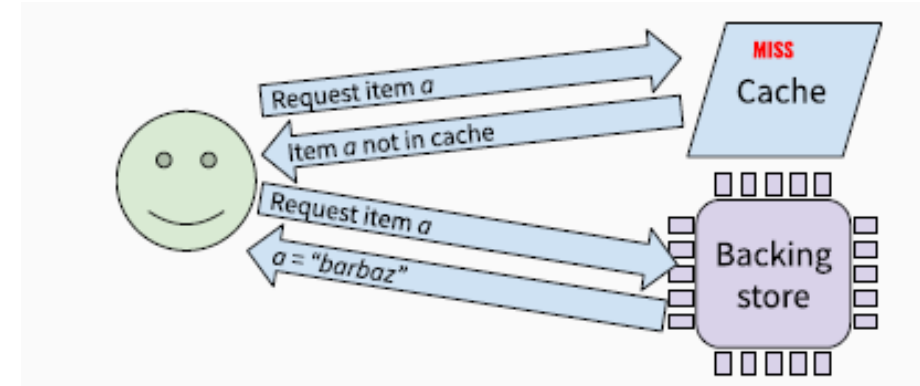
Cache Memory Access Time during Cache Hit (T_c) = T_c

Cache Memory Access Time during Cache miss (T_c) = 0

T_C is the average cache access time

T_M is Mean memory access time

$$T_{Avg} = (h \times T_C) + (1-h) \times T_M$$



Example

Assume that a computer system employs a cache with an access time of 20ns and a main memory with a cycle time of 200ns. Suppose that the hit ratio for reads is 90%,

b) what would be the average access time for reads if the cache is a “look-Aside” cache?

The average read access time in this case

$$T_{Avg} = (h \times T_c) + (1-h) \times T_M$$

$$= 0.9 \times 20\text{ns} + 0.10 \times 200\text{ns} = 38\text{ns}$$

A computer system employs a write-back cache with a 90% hit ratio for writes. The cache operates in "Look Through" and has a 80% read hit ratio. Reads account for 60% of all memory references and writes account for 40%. If the main memory cycle time is 300ns and the cache access time is 30ns, what would be the average access time for all references (reads as well as writes)?

TC	TM	Reads	Reads	Reads	Writes	Writes	Writes
ns	ns	%	Hit	Miss	%	Hit	Miss
30	300	60	80	20	40	90	10

Reads	Writes	Total
Look Through	Look Through	Look Through
$T_c + (m * T_M) =$	$T_c + (m * T_M) =$	
$30 + (0.2 * 300)$	$30 + (0.1 * 300)$	$((90 * 0.6) + (60 * 0.4))$
$) = 90$	$= 60$	$= 78$

A computer system employs a write-back cache with a 90% hit ratio for writes. The cache operates in "Look Aside" and has a 80% read hit ratio. Reads account for 60% of all memory references and writes account for 40%. If the main memory cycle time is 300ns and the cache access time is 30ns, what would be the average access time for all references (reads as well as writes)?

Ans:

TC	TM	Reads	Reads	Reads	Writes	Writes	Writes
ns	ns	%	Hit	Miss	%	Hit	Miss
30	300	60	80	20	40	90	10

Reads	R	Writes	Total
Look Aside	e		Look Aside
(h*Tc)+(m*	a		
TM)=	d		
	s		
			7
			3
(0.8*30)+(	8	(0.9*30)+(	5
0.2*300)=	4	0.1*300)=	7
			((84*0.6)+(
			57*0.4))=
			2